{
 "nbformat": 4,
 "nbformat_minor": 0,
 "metadata": {
  "colab": {
   "provenance": []
  },
  "kernelspec": {
   "name": "python3",
   "display_name": "Python 3"
  },
  "language_info": {
   "name": "python"
  }
 },
 "cells": [
  {
   "cell_type": "markdown",
   "source": [
    "**Breast Cancer Classification with a simple Neural Network (NN)**"
   ],
   "metadata": {
    "id": "RNC9RB0Non0k"
   }
  },
  {
   "cell_type": "markdown",
   "source": [
    "Previous videos:\n",
    "\n",
    "\n",
    "* Brest Cancer Classification with Logistic Regression video: https://youtu.be/bFh1umUDaGc\n",

"* EDA video: https://youtu.be/imdS1LlllSY\n",
"* Data Visualization video: https://youtu.be/gFSerq21wo8\n",
"\n"
],
"metadata": {
"id": "7GUR8W1lowy3"
}
},
{
"cell_type": "markdown",
"metadata": {
"id": "X3wT8l6lfj--"
},
"source": [
"**Importing the Dependencies**"
]
},
{
"cell_type": "code",
"metadata": {
"id": "XqsQmOXGXXTe"
},
"source": [
"import numpy as np\n",
"import pandas as pd\n",
"import matplotlib.pyplot as plt\n",
"import sklearn.datasets\n",
"from sklearn.model_selection import train_test_split"
],
"execution_count": 1,
"outputs": []
},
{

"cell_type": "markdown",
"metadata": {
"id": "pwJ9zLukg3Q_"
},
"source": [
"Data Collection & Processing"
]
},
{
"cell_type": "code",
"metadata": {
"id": "j6bMZMKUgz7L"
},
"source": [
"# loading the data from sklearn\n",
"breast_cancer_dataset = sklearn.datasets.load_breast_cancer()"
],
"execution_count": 2,
"outputs": []
},
{
"cell_type": "code",
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "xdY6i73KgkDG",
"outputId": "2a879655-8f19-4f31-9e45-888b0a8cffba"
},
"source": [
"print(breast_cancer_dataset)"
],
"execution_count": 3,

"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"{'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
\n",
" 1.189e-01],\n",
" [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,\n",
" 8.902e-02],\n",
" [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,\n",
" 8.758e-02],\n",
" ...,\n",
" [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,\n",
" 7.820e-02],\n",
" [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,\n",
" 1.240e-01],\n",
" [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,\n",
" 7.039e-02]]), 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1,\n",
" 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,\n",
" 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,\n",
" 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,\n",
" 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,\n",
" 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,\n",
" 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,\n",
" 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,\n",
" 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,\n",
" 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,\n",
" 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,\n",
" 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,\n",
" 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,\n",
" 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,\n",

" 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,\n",
" 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,\n",
" 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,\n",
" 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1,\n",
" 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,\n",
" 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,\n",
" 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,\n",
" 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,\n",
" 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,\n",
" 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,\n",
" 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,\n",
" 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1]), 'frame': None, 'target_names': array(['malignant', 'benign'], dtype='<U9'), 'DESCR': '.. _breast_cancer_dataset:\\n\\nBreast cancer wisconsin (diagnostic) dataset\\n--------------------------------------\\n\\n**Data Set Characteristics:**\\n\\n:Number of Instances: 569\\n\\n:Number of Attributes: 30 numeric, predictive attributes and the class\\n\\n:Attribute Information:\\n - radius (mean of distances from center to points on the perimeter)\\n - texture (standard deviation of gray-scale values)\\n - perimeter\\n - area\\n - smoothness (local variation in radius lengths)\\n - compactness (perimeter^2 / area - 1.0)\\n - concavity (severity of concave portions of the contour)\\n - concave points (number of concave portions of the contour)\\n - symmetry\\n - fractal dimension (\"coastline approximation\" - 1)\\n\\n The mean, standard error, and \"worst\" or largest (mean of the three\\n worst/largest values) of these features were computed for each image,\\n resulting in 30 features. For instance, field 0 is Mean Radius, field\\n 10 is Radius SE, field 20 is Worst Radius.\\n\\n - class:\\n   - WDBC-Malignant\\n   - WDBC-Benign\\n\\n:Summary Statistics:\\n\\n===================================== ========== ======\\n   Min   Max\\n===================================== ========== ======\\nradius (mean):   6.981 28.11\\ntexture (mean):   9.71 39.28\\nperimeter (mean):   43.79 188.5\\narea (mean):   143.5 2501.0\\nsmoothness (mean):   0.053 0.163\\ncompactness (mean):   0.019 0.345\\nconcavity (mean):   0.0   0.427\\nconcave points (mean):   0.0   0.201\\nsymmetry (mean):

0.106 0.304\\nfractal dimension (mean): 0.05 0.097\\nradius (standard error):
0.112 2.873\\ntexture (standard error): 0.36 4.885\\nperimeter (standard error): 0.757 21.98\\narea (standard error): 6.802 542.2\\nsmoothness (standard error): 0.002 0.031\\ncompactness (standard error): 0.002 0.135\\nconcavity (standard error): 0.0 0.396\\nconcave points (standard error): 0.0 0.053\\nsymmetry (standard error): 0.008 0.079\\nfractal dimension (standard error): 0.001 0.03\\nradius (worst): 7.93 36.04\\ntexture (worst): 12.02 49.54\\nperimeter (worst): 50.41 251.2\\narea (worst): 185.2 4254.0\\nsmoothness (worst): 0.071 0.223\\ncompactness (worst): 0.027 1.058\\nconcavity (worst): 0.0 1.252\\nconcave points (worst): 0.0 0.291\\nsymmetry (worst): 0.156 0.664\\nfractal dimension (worst): 0.055 0.208\\n========== ====== ======\\n\\n:Missing Attribute Values: None\\n\\n:Class Distribution: 212 - Malignant, 357 - Benign\\n\\n:Creator: Dr. William H. Wolberg, W. Nick Street, Olvi L. Mangasarian\\n\\n:Donor: Nick Street\\n\\n:Date: November, 1995\\n\\nThis is a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) datasets.\\nhttps://goo.gl/U2Uwz2\\n\\nFeatures are computed from a digitized image of a fine needle\\naspirate (FNA) of a breast mass. They describe\\ncharacteristics of the cell nuclei present in the image.\\n\\nSeparating plane described above was obtained using\\nMultisurface Method-Tree (MSM-T) [K. P. Bennett, \"Decision Tree\\nConstruction Via Linear Programming.\" Proceedings of the 4th\\nMidwest Artificial Intelligence and Cognitive Science Society,\\npp. 97-101, 1992], a classification method which uses linear\\nprogramming to construct a decision tree. Relevant features\\nwere selected using an exhaustive search in the space of 1-4\\nfeatures and 1-3 separating planes.\\n\\nThe actual linear program used to obtain the separating plane\\nin the 3-dimensional space is that described in:\\n[K. P. Bennett and O. L. Mangasarian: \"Robust Linear\\nProgramming Discrimination of Two Linearly Inseparable Sets\",\\nOptimization Methods and Software 1, 1992, 23-34].\\n\\nThis database is also available through the UW CS ftp server:\\n\\nftp ftp.cs.wisc.edu\\ncd math-prog/cpo-dataset/machine-learn/WDBC/\\n\\n.. dropdown:: References\\n\\n - W.N. Street, W.H. Wolberg and O.L. Mangasarian. Nuclear feature extraction\\n for

breast tumor diagnosis. IS&T/SPIE 1993 International Symposium on\\n Electro
nic Imaging: Science and Technology, volume 1905, pages 861-870,\\n San Jos
e, CA, 1993.\\n - O.L. Mangasarian, W.N. Street and W.H. Wolberg. Breast cancer
 diagnosis and\\n prognosis via linear programming. Operations Research, 43(4
), pages 570-577,\\n July-August 1995.\\n - W.H. Wolberg, W.N. Street, and O.
L. Mangasarian. Machine learning techniques\\n to diagnose breast cancer from
 fine-needle aspirates. Cancer Letters 77 (1994)\\n 163-171.\\n', 'feature_names':
 array(['mean radius', 'mean texture', 'mean perimeter', 'mean area',\n",
" 'mean smoothness', 'mean compactness', 'mean concavity',\n",
" 'mean concave points', 'mean symmetry', 'mean fractal dimension',\n",
" 'radius error', 'texture error', 'perimeter error', 'area error',\n",
" 'smoothness error', 'compactness error', 'concavity error',\n",
" 'concave points error', 'symmetry error',\n",
" 'fractal dimension error', 'worst radius', 'worst texture',\n",
" 'worst perimeter', 'worst area', 'worst smoothness',\n",
" 'worst compactness', 'worst concavity', 'worst concave points',\n",
" 'worst symmetry', 'worst fractal dimension'], dtype='<U23'), 'filename': '
breast_cancer.csv', 'data_module': 'sklearn.datasets.data'}\n"
]
}
]
},
{
"cell_type": "code",
"metadata": {
"id": "yFamhVTThMdK"
},
"source": [
"# loading the data to a data frame\n",
"data_frame = pd.DataFrame(breast_cancer_dataset.data, columns = breast
_cancer_dataset.feature_names)"
],
"execution_count": 4,

"outputs": []
},
{
"cell_type": "code",
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/",
"height": 290
},
"id": "xLiv1maYiGVH",
"outputId": "28083e03-d74f-42a2-fcce-a81069aff8ae"
},
"source": [
"# print the first 5 rows of the dataframe\n",
"data_frame.head()"
],
"execution_count": 5,
"outputs": [
{
"output_type": "execute_result",
"data": {
"text/plain": [
"   mean radius mean texture mean perimeter mean area mean smoothne
ss \\\n",
"0   17.99   10.38   122.80   1001.0   0.11840 \n",
"1   20.57   17.77   132.90   1326.0   0.08474 \n",
"2   19.69   21.25   130.00   1203.0   0.10960 \n",
"3   11.42   20.38   77.58   386.1   0.14250 \n",
"4   20.29   14.34   135.10   1297.0   0.10030 \n",
"\n",
"   mean compactness mean concavity mean concave points mean symm
etry \\\n",
"0   0.27760   0.3001   0.14710   0.2419 \n",

"1 0.07864 0.0869 0.07017 0.1812 \n",
"2 0.15990 0.1974 0.12790 0.2069 \n",
"3 0.28390 0.2414 0.10520 0.2597 \n",
"4 0.13280 0.1980 0.10430 0.1809 \n",
"\n",
" mean fractal dimension ... worst radius worst texture worst perimeter \\
\n",
"0 0.07871 ... 25.38 17.33 184.60 \n",
"1 0.05667 ... 24.99 23.41 158.80 \n",
"2 0.05999 ... 23.57 25.53 152.50 \n",
"3 0.09744 ... 14.91 26.50 98.87 \n",
"4 0.05883 ... 22.54 16.67 152.20 \n",
"\n",
" worst area worst smoothness worst compactness worst concavity \\\n
",
"0 2019.0 0.1622 0.6656 0.7119 \n",
"1 1956.0 0.1238 0.1866 0.2416 \n",
"2 1709.0 0.1444 0.4245 0.4504 \n",
"3 567.7 0.2098 0.8663 0.6869 \n",
"4 1575.0 0.1374 0.2050 0.4000 \n",
"\n",
" worst concave points worst symmetry worst fractal dimension \n",
"0 0.2654 0.4601 0.11890 \n",
"1 0.1860 0.2750 0.08902 \n",
"2 0.2430 0.3613 0.08758 \n",
"3 0.2575 0.6638 0.17300 \n",
"4 0.1625 0.2364 0.07678 \n",
"\n",
"[5 rows x 30 columns]"
],
"text/html": [
"\n",
" <div id=\"df-ee582482-5841-46cc-ac4d-fdcc684d1bcf\" class=\"colab-

df-container\">\n",
" \n",
"<style scoped>\n",
" .dataframe tbody tr th:only-of-type {\n",
"   vertical-align: middle;\n",
" }\n",
"\n",
" .dataframe tbody tr th {\n",
"   vertical-align: top;\n",
" }\n",
"\n",
" .dataframe thead th {\n",
"   text-align: right;\n",
" }\n",
"</style>\n",
"<table border=\"1\" class=\"dataframe\">\n",
" <thead>\n",
"   <tr style=\"text-align: right;\">\n",
"   <th></th>\n",
"   <th>mean radius</th>\n",
"   <th>mean texture</th>\n",
"   <th>mean perimeter</th>\n",
"   <th>mean area</th>\n",
"   <th>mean smoothness</th>\n",
"   <th>mean compactness</th>\n",
"   <th>mean concavity</th>\n",
"   <th>mean concave points</th>\n",
"   <th>mean symmetry</th>\n",
"   <th>mean fractal dimension</th>\n",
"   <th>...</th>\n",
"   <th>worst radius</th>\n",
"   <th>worst texture</th>\n",
"   <th>worst perimeter</th>\n",

"   <th>worst area</th>\n",
"   <th>worst smoothness</th>\n",
"   <th>worst compactness</th>\n",
"   <th>worst concavity</th>\n",
"   <th>worst concave points</th>\n",
"   <th>worst symmetry</th>\n",
"   <th>worst fractal dimension</th>\n",
"   </tr>\n",
" </thead>\n",
" <tbody>\n",
"   <tr>\n",
"   <th>0</th>\n",
"   <td>17.99</td>\n",
"   <td>10.38</td>\n",
"   <td>122.80</td>\n",
"   <td>1001.0</td>\n",
"   <td>0.11840</td>\n",
"   <td>0.27760</td>\n",
"   <td>0.3001</td>\n",
"   <td>0.14710</td>\n",
"   <td>0.2419</td>\n",
"   <td>0.07871</td>\n",
"   <td>...</td>\n",
"   <td>25.38</td>\n",
"   <td>17.33</td>\n",
"   <td>184.60</td>\n",
"   <td>2019.0</td>\n",
"   <td>0.1622</td>\n",
"   <td>0.6656</td>\n",
"   <td>0.7119</td>\n",
"   <td>0.2654</td>\n",
"   <td>0.4601</td>\n",
"   <td>0.11890</td>\n",

" </tr>\n",
" <tr>\n",
"   <th>1</th>\n",
"   <td>20.57</td>\n",
"   <td>17.77</td>\n",
"   <td>132.90</td>\n",
"   <td>1326.0</td>\n",
"   <td>0.08474</td>\n",
"   <td>0.07864</td>\n",
"   <td>0.0869</td>\n",
"   <td>0.07017</td>\n",
"   <td>0.1812</td>\n",
"   <td>0.05667</td>\n",
"   <td>...</td>\n",
"   <td>24.99</td>\n",
"   <td>23.41</td>\n",
"   <td>158.80</td>\n",
"   <td>1956.0</td>\n",
"   <td>0.1238</td>\n",
"   <td>0.1866</td>\n",
"   <td>0.2416</td>\n",
"   <td>0.1860</td>\n",
"   <td>0.2750</td>\n",
"   <td>0.08902</td>\n",
" </tr>\n",
" <tr>\n",
"   <th>2</th>\n",
"   <td>19.69</td>\n",
"   <td>21.25</td>\n",
"   <td>130.00</td>\n",
"   <td>1203.0</td>\n",
"   <td>0.10960</td>\n",
"   <td>0.15990</td>\n",

"   <td>0.1974</td>\n",
"   <td>0.12790</td>\n",
"   <td>0.2069</td>\n",
"   <td>0.05999</td>\n",
"   <td>...</td>\n",
"   <td>23.57</td>\n",
"   <td>25.53</td>\n",
"   <td>152.50</td>\n",
"   <td>1709.0</td>\n",
"   <td>0.1444</td>\n",
"   <td>0.4245</td>\n",
"   <td>0.4504</td>\n",
"   <td>0.2430</td>\n",
"   <td>0.3613</td>\n",
"   <td>0.08758</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>3</th>\n",
"   <td>11.42</td>\n",
"   <td>20.38</td>\n",
"   <td>77.58</td>\n",
"   <td>386.1</td>\n",
"   <td>0.14250</td>\n",
"   <td>0.28390</td>\n",
"   <td>0.2414</td>\n",
"   <td>0.10520</td>\n",
"   <td>0.2597</td>\n",
"   <td>0.09744</td>\n",
"   <td>...</td>\n",
"   <td>14.91</td>\n",
"   <td>26.50</td>\n",
"   <td>98.87</td>\n",
"   <td>567.7</td>\n",

"   <td>0.2098</td>\n",
"   <td>0.8663</td>\n",
"   <td>0.6869</td>\n",
"   <td>0.2575</td>\n",
"   <td>0.6638</td>\n",
"   <td>0.17300</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>4</th>\n",
"   <td>20.29</td>\n",
"   <td>14.34</td>\n",
"   <td>135.10</td>\n",
"   <td>1297.0</td>\n",
"   <td>0.10030</td>\n",
"   <td>0.13280</td>\n",
"   <td>0.1980</td>\n",
"   <td>0.10430</td>\n",
"   <td>0.1809</td>\n",
"   <td>0.05883</td>\n",
"   <td>...</td>\n",
"   <td>22.54</td>\n",
"   <td>16.67</td>\n",
"   <td>152.20</td>\n",
"   <td>1575.0</td>\n",
"   <td>0.1374</td>\n",
"   <td>0.2050</td>\n",
"   <td>0.4000</td>\n",
"   <td>0.1625</td>\n",
"   <td>0.2364</td>\n",
"   <td>0.07678</td>\n",
"   </tr>\n",
" </tbody>\n",
"</table>\n",

" 5 rows × 30 columns \n",
" \n",
" <div class=\"colab-df-buttons\">\n",
"\n",
" <div class=\"colab-df-container\">\n",
" <button class=\"colab-df-convert\" onclick=\"convertToInteractive('df-ee582482-5841-46cc-ac4d-fdcc684d1bcf')\"\n",
" title=\"Convert this dataframe to an interactive table.\"\n",
" style=\"display:none;\">\n",
"\n",
" <svg xmlns=\"http://www.w3.org/2000/svg\" height=\"24px\" viewBox=\"0 -960 960 960\">\n",
" <path d=\"M120-120v-720h720v720H120Zm60-500h600v-160H180v160Zm220 220h160v-160H400v160Zm0 220h160v-160H400v160ZM180-400h160v-160H180v160Zm440 0h160v-160H620v160ZM180-180h160v-160H180v160Zm440 0h160v-160H620v160Z\"/>\n",
" </svg>\n",
" </button>\n",
"\n",
" <style>\n",
" .colab-df-container {\n",
" display:flex;\n",
" gap: 12px;\n",
" }\n",
"\n",
" .colab-df-convert {\n",
" background-color: #E8F0FE;\n",
" border: none;\n",
" border-radius: 50%;\n",
" cursor: pointer;\n",
" display: none;\n",
" fill: #1967D2;\n",
" height: 32px;\n",

```
"     padding: 0 0 0 0;\n",
"     width: 32px;\n",
"   }\n",
"\n",
"   .colab-df-convert:hover {\n",
"     background-color: #E2EBFA;\n",
"     box-shadow: 0px 1px 2px rgba(60, 64, 67, 0.3), 0px 1px 3px 1px rgba(6
0, 64, 67, 0.15);\n",
"     fill: #174EA6;\n",
"   }\n",
"\n",
"   .colab-df-buttons div {\n",
"     margin-bottom: 4px;\n",
"   }\n",
"\n",
"   [theme=dark] .colab-df-convert {\n",
"     background-color: #3B4455;\n",
"     fill: #D2E3FC;\n",
"   }\n",
"\n",
"   [theme=dark] .colab-df-convert:hover {\n",
"     background-color: #434B5C;\n",
"     box-shadow: 0px 1px 3px 1px rgba(0, 0, 0, 0.15);\n",
"     filter: drop-shadow(0px 1px 2px rgba(0, 0, 0, 0.3));\n",
"     fill: #FFFFFF;\n",
"   }\n",
"  </style>\n",
"\n",
"   <script>\n",
"     const buttonEl =\n",
"       document.querySelector('#df-ee582482-5841-46cc-ac4d-fdcc684
d1bcf button.colab-df-convert');\n",
"     buttonEl.style.display =\n",
```

"   google.colab.kernel.accessAllowed ? 'block' : 'none';\n",
"\n",
"   async function convertToInteractive(key) {\n",
"   const element = document.querySelector('#df-ee582482-5841-46cc-ac4d-fdcc684d1bcf');\n",
"   const dataTable =\n",
"   await google.colab.kernel.invokeFunction('convertToInteractive',\n",
"   [key], {});\n",
"   if (!dataTable) return;\n",
"\n",
"   const docLinkHtml = 'Like what you see? Visit the ' +\n",
"   '<a target=\"_blank\" href=https://colab.research.google.com/notebooks/data_table.ipynb>data table notebook</a>'\n",
"   + ' to learn more about interactive tables.';\n",
"   element.innerHTML = '';\n",
"   dataTable['output_type'] = 'display_data';\n",
"   await google.colab.output.renderOutput(dataTable, element);\n",
"   const docLink = document.createElement('div');\n",
"   docLink.innerHTML = docLinkHtml;\n",
"   element.appendChild(docLink);\n",
"   }\n",
"   </script>\n",
" \n",
"\n",
"\n",
"   \n",
" \n"
],
"application/vnd.google.colaboratory.intrinsic+json": {
"type": "dataframe",
"variable_name": "data_frame"
}
},

"metadata": {},
"execution_count": 5
}
]
},
{
"cell_type": "code",
"metadata": {
"id": "UEvD_aTDiNLF"
},
"source": [
"# adding the 'target' column to the data frame\n",
"data_frame['label'] = breast_cancer_dataset.target"
],
"execution_count": 6,
"outputs": []
},
{
"cell_type": "code",
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/",
"height": 290
},
"id": "f_kmjEA5io2v",
"outputId": "a6fdc13a-f9b5-4118-d2c4-a66afeb7a791"
},
"source": [
"# print last 5 rows of the dataframe\n",
"data_frame.tail()"
],
"execution_count": 7,
"outputs": [

{
"output_type": "execute_result",
"data": {
"text/plain": [
" mean radius mean texture mean perimeter mean area mean smoothne
ss \\\n",
"564   21.56   22.39   142.00   1479.0   0.11100 \n",
"565   20.13   28.25   131.20   1261.0   0.09780 \n",
"566   16.60   28.08   108.30   858.1   0.08455 \n",
"567   20.60   29.33   140.10   1265.0   0.11780 \n",
"568   7.76   24.54   47.92   181.0   0.05263 \n",
"\n",
" mean compactness mean concavity mean concave points mean sym
metry \\\n",
"564   0.11590   0.24390   0.13890   0.1726 \n",
"565   0.10340   0.14400   0.09791   0.1752 \n",
"566   0.10230   0.09251   0.05302   0.1590 \n",
"567   0.27700   0.35140   0.15200   0.2397 \n",
"568   0.04362   0.00000   0.00000   0.1587 \n",
"\n",
" mean fractal dimension ... worst texture worst perimeter worst area \\
\n",
"564   0.05623 ...   26.40   166.10   2027.0 \n",
"565   0.05533 ...   38.25   155.00   1731.0 \n",
"566   0.05648 ...   34.12   126.70   1124.0 \n",
"567   0.07016 ...   39.42   184.60   1821.0 \n",
"568   0.05884 ...   30.37   59.16   268.6 \n",
"\n",
" worst smoothness worst compactness worst concavity \\\n",
"564   0.14100   0.21130   0.4107 \n",
"565   0.11660   0.19220   0.3215 \n",
"566   0.11390   0.30940   0.3403 \n",
"567   0.16500   0.86810   0.9387 \n",

"568 0.08996 0.06444 0.0000 \n",
"\n",
" worst concave points worst symmetry worst fractal dimension label \n",
"564 0.2216 0.2060 0.07115 0 \n",
"565 0.1628 0.2572 0.06637 0 \n",
"566 0.1418 0.2218 0.07820 0 \n",
"567 0.2650 0.4087 0.12400 0 \n",
"568 0.0000 0.2871 0.07039 1 \n",
"\n",
"[5 rows x 31 columns]"
],
"text/html": [
"\n",
" <div id=\"df-592a60ac-edcd-4e40-a3ab-d2652adf84bb\" class=\"colab-df-container\">\n",
" \n",
"<style scoped>\n",
" .dataframe tbody tr th:only-of-type {\n",
" vertical-align: middle;\n",
" }\n",
"\n",
" .dataframe tbody tr th {\n",
" vertical-align: top;\n",
" }\n",
"\n",
" .dataframe thead th {\n",
" text-align: right;\n",
" }\n",
"</style>\n",
"<table border=\"1\" class=\"dataframe\">\n",
" <thead>\n",
" <tr style=\"text-align: right;\">\n",

```
"     <th></th>\n",
"     <th>mean radius</th>\n",
"     <th>mean texture</th>\n",
"     <th>mean perimeter</th>\n",
"     <th>mean area</th>\n",
"     <th>mean smoothness</th>\n",
"     <th>mean compactness</th>\n",
"     <th>mean concavity</th>\n",
"     <th>mean concave points</th>\n",
"     <th>mean symmetry</th>\n",
"     <th>mean fractal dimension</th>\n",
"     <th>...</th>\n",
"     <th>worst texture</th>\n",
"     <th>worst perimeter</th>\n",
"     <th>worst area</th>\n",
"     <th>worst smoothness</th>\n",
"     <th>worst compactness</th>\n",
"     <th>worst concavity</th>\n",
"     <th>worst concave points</th>\n",
"     <th>worst symmetry</th>\n",
"     <th>worst fractal dimension</th>\n",
"     <th>label</th>\n",
"   </tr>\n",
"  </thead>\n",
"  <tbody>\n",
"   <tr>\n",
"     <th>564</th>\n",
"     <td>21.56</td>\n",
"     <td>22.39</td>\n",
"     <td>142.00</td>\n",
"     <td>1479.0</td>\n",
"     <td>0.11100</td>\n",
"     <td>0.11590</td>\n",
```

"   <td>0.24390</td>\n",
"   <td>0.13890</td>\n",
"   <td>0.1726</td>\n",
"   <td>0.05623</td>\n",
"   <td>...</td>\n",
"   <td>26.40</td>\n",
"   <td>166.10</td>\n",
"   <td>2027.0</td>\n",
"   <td>0.14100</td>\n",
"   <td>0.21130</td>\n",
"   <td>0.4107</td>\n",
"   <td>0.2216</td>\n",
"   <td>0.2060</td>\n",
"   <td>0.07115</td>\n",
"   <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>565</th>\n",
"   <td>20.13</td>\n",
"   <td>28.25</td>\n",
"   <td>131.20</td>\n",
"   <td>1261.0</td>\n",
"   <td>0.09780</td>\n",
"   <td>0.10340</td>\n",
"   <td>0.14400</td>\n",
"   <td>0.09791</td>\n",
"   <td>0.1752</td>\n",
"   <td>0.05533</td>\n",
"   <td>...</td>\n",
"   <td>38.25</td>\n",
"   <td>155.00</td>\n",
"   <td>1731.0</td>\n",
"   <td>0.11660</td>\n",

"   <td>0.19220</td>\n",
"   <td>0.3215</td>\n",
"   <td>0.1628</td>\n",
"   <td>0.2572</td>\n",
"   <td>0.06637</td>\n",
"   <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>566</th>\n",
"   <td>16.60</td>\n",
"   <td>28.08</td>\n",
"   <td>108.30</td>\n",
"   <td>858.1</td>\n",
"   <td>0.08455</td>\n",
"   <td>0.10230</td>\n",
"   <td>0.09251</td>\n",
"   <td>0.05302</td>\n",
"   <td>0.1590</td>\n",
"   <td>0.05648</td>\n",
"   <td>...</td>\n",
"   <td>34.12</td>\n",
"   <td>126.70</td>\n",
"   <td>1124.0</td>\n",
"   <td>0.11390</td>\n",
"   <td>0.30940</td>\n",
"   <td>0.3403</td>\n",
"   <td>0.1418</td>\n",
"   <td>0.2218</td>\n",
"   <td>0.07820</td>\n",
"   <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>567</th>\n",

"   <td>20.60</td>\n",
"   <td>29.33</td>\n",
"   <td>140.10</td>\n",
"   <td>1265.0</td>\n",
"   <td>0.11780</td>\n",
"   <td>0.27700</td>\n",
"   <td>0.35140</td>\n",
"   <td>0.15200</td>\n",
"   <td>0.2397</td>\n",
"   <td>0.07016</td>\n",
"   <td>...</td>\n",
"   <td>39.42</td>\n",
"   <td>184.60</td>\n",
"   <td>1821.0</td>\n",
"   <td>0.16500</td>\n",
"   <td>0.86810</td>\n",
"   <td>0.9387</td>\n",
"   <td>0.2650</td>\n",
"   <td>0.4087</td>\n",
"   <td>0.12400</td>\n",
"   <td>0</td>\n",
" </tr>\n",
" <tr>\n",
"   <th>568</th>\n",
"   <td>7.76</td>\n",
"   <td>24.54</td>\n",
"   <td>47.92</td>\n",
"   <td>181.0</td>\n",
"   <td>0.05263</td>\n",
"   <td>0.04362</td>\n",
"   <td>0.00000</td>\n",
"   <td>0.00000</td>\n",
"   <td>0.1587</td>\n",

"   <td>0.05884</td>\n",
"   <td>...</td>\n",
"   <td>30.37</td>\n",
"   <td>59.16</td>\n",
"   <td>268.6</td>\n",
"   <td>0.08996</td>\n",
"   <td>0.06444</td>\n",
"   <td>0.0000</td>\n",
"   <td>0.0000</td>\n",
"   <td>0.2871</td>\n",
"   <td>0.07039</td>\n",
"   <td>1</td>\n",
"   </tr>\n",
" </tbody>\n",
"</table>\n",
" 5 rows × 31 columns \n",
" \n",
"   <div class=\"colab-df-buttons\">\n",
"\n",
" <div class=\"colab-df-container\">\n",
"   <button class=\"colab-df-convert\" onclick=\"convertToInteractive('df-592a60ac-edcd-4e40-a3ab-d2652adf84bb')\"\n",
"   title=\"Convert this dataframe to an interactive table.\"\n",
"   style=\"display:none;\">\n",
"\n",
" <svg xmlns=\"http://www.w3.org/2000/svg\" height=\"24px\" viewBox=\"0 -960 960 960\">\n",
"   <path d=\"M120-120v-720h720v720H120Zm60-500h600v-160H180v160Zm220 220h160v-160H400v160Zm0 220h160v-160H400v160ZM180-400h160v-160H180v160Zm440 0h160v-160H620v160ZM180-180h160v-160H180v160Zm440 0h160v-160H620v160Z\"/>\n",
" </svg>\n",
"   </button>\n",

```
"\n",
"  <style>\n",
"    .colab-df-container {\n",
"      display:flex;\n",
"      gap: 12px;\n",
"    }\n",
"\n",
"    .colab-df-convert {\n",
"      background-color: #E8F0FE;\n",
"      border: none;\n",
"      border-radius: 50%;\n",
"      cursor: pointer;\n",
"      display: none;\n",
"      fill: #1967D2;\n",
"      height: 32px;\n",
"      padding: 0 0 0 0;\n",
"      width: 32px;\n",
"    }\n",
"\n",
"    .colab-df-convert:hover {\n",
"      background-color: #E2EBFA;\n",
"      box-shadow: 0px 1px 2px rgba(60, 64, 67, 0.3), 0px 1px 3px 1px rgba(6
0, 64, 67, 0.15);\n",
"      fill: #174EA6;\n",
"    }\n",
"\n",
"    .colab-df-buttons div {\n",
"      margin-bottom: 4px;\n",
"    }\n",
"\n",
"    [theme=dark] .colab-df-convert {\n",
"      background-color: #3B4455;\n",
"      fill: #D2E3FC;\n",
```

```
"   }\n",
"\n",
"   [theme=dark] .colab-df-convert:hover {\n",
"     background-color: #434B5C;\n",
"     box-shadow: 0px 1px 3px 1px rgba(0, 0, 0, 0.15);\n",
"     filter: drop-shadow(0px 1px 2px rgba(0, 0, 0, 0.3));\n",
"     fill: #FFFFFF;\n",
"   }\n",
"  </style>\n",
"\n",
"   <script>\n",
"     const buttonEl =\n",
"       document.querySelector('#df-592a60ac-edcd-4e40-a3ab-d2652ad
f84bb button.colab-df-convert');\n",
"     buttonEl.style.display =\n",
"       google.colab.kernel.accessAllowed ? 'block' : 'none';\n",
"\n",
"     async function convertToInteractive(key) {\n",
"       const element = document.querySelector('#df-592a60ac-edcd-4e40
-a3ab-d2652adf84bb');\n",
"       const dataTable =\n",
"         await google.colab.kernel.invokeFunction('convertToInteractive',\n",
"                                                  [key], {});\n",
"       if (!dataTable) return;\n",
"\n",
"       const docLinkHtml = 'Like what you see? Visit the ' +\n",
"         '<a target=\"_blank\" href=https://colab.research.google.com/noteb
ooks/data_table.ipynb>data table notebook</a>'\n",
"         + ' to learn more about interactive tables.';\n",
"       element.innerHTML = '';\n",
"       dataTable['output_type'] = 'display_data';\n",
"       await google.colab.output.renderOutput(dataTable, element);\n",
"       const docLink = document.createElement('div');\n",
```

"   docLink.innerHTML = docLinkHtml;\n",
"   element.appendChild(docLink);\n",
"   }\n",
"   </script>\n",
" \n",
"\n",
"\n",
"   \n",
" \n"
],
"application/vnd.google.colaboratory.intrinsic+json": {
"type": "dataframe"
}
},
"metadata": {},
"execution_count": 7
}
]
},
{
"cell_type": "code",
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "sw3wjdK6iwK4",
"outputId": "08e2b301-ddcf-4ecd-eb8b-5d14b0a22305"
},
"source": [
"# number of rows and columns in the dataset\n",
"data_frame.shape"
],
"execution_count": 8,

"outputs": [
{
"output_type": "execute_result",
"data": {
"text/plain": [
"(569, 31)"
]
},
"metadata": {},
"execution_count": 8
}
]
},
{
"cell_type": "code",
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "AWOjMuyBi77M",
"outputId": "e2408f6e-5e41-497c-f2cf-c776ab5cb4a3"
},
"source": [
"# getting some information about the data\n",
"data_frame.info()"
],
"execution_count": 9,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"<class 'pandas.core.frame.DataFrame'>\n",

```
"RangeIndex: 569 entries, 0 to 568\n",
"Data columns (total 31 columns):\n",
" #   Column                  Non-Null Count  Dtype  \n",
"---  ------                  --------------  -----  \n",
" 0   mean radius            569 non-null   float64\n",
" 1   mean texture           569 non-null   float64\n",
" 2   mean perimeter         569 non-null   float64\n",
" 3   mean area              569 non-null   float64\n",
" 4   mean smoothness        569 non-null   float64\n",
" 5   mean compactness       569 non-null   float64\n",
" 6   mean concavity         569 non-null   float64\n",
" 7   mean concave points    569 non-null   float64\n",
" 8   mean symmetry          569 non-null   float64\n",
" 9   mean fractal dimension  569 non-null   float64\n",
" 10  radius error           569 non-null   float64\n",
" 11  texture error          569 non-null   float64\n",
" 12  perimeter error        569 non-null   float64\n",
" 13  area error             569 non-null   float64\n",
" 14  smoothness error       569 non-null   float64\n",
" 15  compactness error      569 non-null   float64\n",
" 16  concavity error        569 non-null   float64\n",
" 17  concave points error   569 non-null   float64\n",
" 18  symmetry error         569 non-null   float64\n",
" 19  fractal dimension error 569 non-null   float64\n",
" 20  worst radius           569 non-null   float64\n",
" 21  worst texture          569 non-null   float64\n",
" 22  worst perimeter        569 non-null   float64\n",
" 23  worst area             569 non-null   float64\n",
" 24  worst smoothness       569 non-null   float64\n",
" 25  worst compactness      569 non-null   float64\n",
" 26  worst concavity        569 non-null   float64\n",
" 27  worst concave points   569 non-null   float64\n",
" 28  worst symmetry         569 non-null   float64\n",
```

```
      " 29  worst fractal dimension  569 non-null    float64\n",
      " 30  label                    569 non-null    int64  \n",
      "dtypes: float64(30), int64(1)\n",
      "memory usage: 137.9 KB\n"
     ]
    }
   ]
  },
  {
   "cell_type": "code",
   "metadata": {
    "colab": {
     "base_uri": "https://localhost:8080/",
     "height": 1000
    },
    "id": "RuoVlUTYjLpk",
    "outputId": "f279c9fb-33ac-4489-a5c9-5c1e5b7d16ae"
   },
   "source": [
    "# checking for missing values\n",
    "data_frame.isnull().sum()"
   ],
   "execution_count": 10,
   "outputs": [
    {
     "output_type": "execute_result",
     "data": {
      "text/plain": [
       "mean radius              0\n",
       "mean texture             0\n",
       "mean perimeter           0\n",
       "mean area                0\n",
       "mean smoothness          0\n",
```

```
  "mean compactness         0\n",
  "mean concavity           0\n",
  "mean concave points      0\n",
  "mean symmetry            0\n",
  "mean fractal dimension    0\n",
  "radius error             0\n",
  "texture error            0\n",
  "perimeter error          0\n",
  "area error               0\n",
  "smoothness error         0\n",
  "compactness error        0\n",
  "concavity error          0\n",
  "concave points error     0\n",
  "symmetry error           0\n",
  "fractal dimension error   0\n",
  "worst radius             0\n",
  "worst texture            0\n",
  "worst perimeter          0\n",
  "worst area               0\n",
  "worst smoothness         0\n",
  "worst compactness        0\n",
  "worst concavity          0\n",
  "worst concave points     0\n",
  "worst symmetry           0\n",
  "worst fractal dimension   0\n",
  "label                    0\n",
  "dtype: int64"
 ],
 "text/html": [
  "<div>\n",
  "<style scoped>\n",
  "    .dataframe tbody tr th:only-of-type {\n",
  "        vertical-align: middle;\n",
```

"   }\n",
"\n",
"   .dataframe tbody tr th {\n",
"   vertical-align: top;\n",
"   }\n",
"\n",
"   .dataframe thead th {\n",
"   text-align: right;\n",
"   }\n",
"</style>\n",
"<table border=\"1\" class=\"dataframe\">\n",
" <thead>\n",
"   <tr style=\"text-align: right;\">\n",
"   <th></th>\n",
"   <th>0</th>\n",
"   </tr>\n",
" </thead>\n",
" <tbody>\n",
"   <tr>\n",
"   <th>mean radius</th>\n",
"   <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>mean texture</th>\n",
"   <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>mean perimeter</th>\n",
"   <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>mean area</th>\n",
"   <td>0</td>\n",

```
"   </tr>\n",
"   <tr>\n",
"     <th>mean smoothness</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>mean compactness</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>mean concavity</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>mean concave points</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>mean symmetry</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>mean fractal dimension</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>radius error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>texture error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
```

```
"   <tr>\n",
"     <th>perimeter error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>area error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>smoothness error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>compactness error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>concavity error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>concave points error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>symmetry error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
"     <th>fractal dimension error</th>\n",
"     <td>0</td>\n",
"   </tr>\n",
"   <tr>\n",
```

```
"     <th>worst radius</th>\n",
"     <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"     <th>worst texture</th>\n",
"     <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"     <th>worst perimeter</th>\n",
"     <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"     <th>worst area</th>\n",
"     <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"     <th>worst smoothness</th>\n",
"     <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"     <th>worst compactness</th>\n",
"     <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"     <th>worst concavity</th>\n",
"     <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"     <th>worst concave points</th>\n",
"     <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"     <th>worst symmetry</th>\n",
```

" <td>0</td>\n",
" </tr>\n",
" <tr>\n",
" <th>worst fractal dimension</th>\n",
" <td>0</td>\n",
" </tr>\n",
" <tr>\n",
" <th>label</th>\n",
" <td>0</td>\n",
" </tr>\n",
" </tbody>\n",
"</table>\n",
"
<label>dtype: int64</label>"
]
},
"metadata": {},
"execution_count": 10
}
]
},
{
"cell_type": "code",
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/",
"height": 384
},
"id": "oLMuXI33jlkq",
"outputId": "67e440e7-e247-4816-df69-5d888638c2df"
},
"source": [
"# statistical measures about the data\n",
"data_frame.describe()"

```
  ],
  "execution_count": 11,
  "outputs": [
   {
    "output_type": "execute_result",
    "data": {
     "text/plain": [
      "       mean radius  mean texture  mean perimeter   mean area  \\\n",
      "count  569.000000   569.000000    569.000000  569.000000  \n",
      "mean    14.127292    19.289649     91.969033  654.889104  \n",
      "std      3.524049     4.301036     24.298981  351.914129  \n",
      "min      6.981000     9.710000     43.790000  143.500000  \n",
      "25%     11.700000    16.170000     75.170000  420.300000  \n",
      "50%     13.370000    18.840000     86.240000  551.100000  \n",
      "75%     15.780000    21.800000    104.100000  782.700000  \n",
      "max     28.110000    39.280000    188.500000 2501.000000  \n",
      "\n",
      "       mean smoothness  mean compactness  mean concavity  mean concav
e points  \\\n",
      "count     569.000000      569.000000    569.000000        569.000000
\n",
      "mean       0.096360        0.104341      0.088799          0.048919  \n",
      "std        0.014064        0.052813      0.079720          0.038803  \n",
      "min        0.052630        0.019380      0.000000          0.000000  \n",
      "25%        0.086370        0.064920      0.029560          0.020310  \n",
      "50%        0.095870        0.092630      0.061540          0.033500  \n",
      "75%        0.105300        0.130400      0.130700          0.074000  \n",
      "max        0.163400        0.345400      0.426800          0.201200  \n",
      "\n",
      "       mean symmetry  mean fractal dimension  ...  worst texture  \\\n",
      "count    569.000000           569.000000 ...    569.000000  \n",
      "mean       0.181162             0.062798 ...     25.677223  \n",
      "std        0.027414             0.007060 ...      6.146258  \n",
```

```
"min      0.106000          0.049960 ...    12.020000  \n",
"25%      0.161900          0.057700 ...    21.080000  \n",
"50%      0.179200          0.061540 ...    25.410000  \n",
"75%      0.195700          0.066120 ...    29.720000  \n",
"max      0.304000          0.097440 ...    49.540000  \n",
"\n",
"     worst perimeter  worst area  worst smoothness  worst compactness  \\
\n",
"count     569.000000  569.000000     569.000000      569.000000  \n"
,
"mean      107.261213  880.583128      0.132369        0.254265  \n",
"std       33.602542  569.356993      0.022832        0.157336  \n",
"min       50.410000  185.200000      0.071170        0.027290  \n",
"25%       84.110000  515.300000      0.116600        0.147200  \n",
"50%       97.660000  686.500000      0.131300        0.211900  \n",
"75%      125.400000 1084.000000      0.146000        0.339100  \n",
"max      251.200000 4254.000000      0.222600        1.058000  \n",
"\n",
"     worst concavity  worst concave points  worst symmetry  \\\n",
"count     569.000000        569.000000    569.000000  \n",
"mean       0.272188          0.114606      0.290076  \n",
"std       0.208624          0.065732      0.061867  \n",
"min       0.000000          0.000000      0.156500  \n",
"25%       0.114500          0.064930      0.250400  \n",
"50%       0.226700          0.099930      0.282200  \n",
"75%       0.382900          0.161400      0.317900  \n",
"max       1.252000          0.291000      0.663800  \n",
"\n",
"     worst fractal dimension     label  \n",
"count          569.000000  569.000000  \n",
"mean            0.083946   0.627417  \n",
"std             0.018061   0.483918  \n",
"min             0.055040   0.000000  \n",
```

"25% 0.071460 0.000000 \n",
"50% 0.080040 1.000000 \n",
"75% 0.092080 1.000000 \n",
"max 0.207500 1.000000 \n",
"\n",
"[8 rows x 31 columns]"
],
"text/html": [
"\n",
" <div id=\"df-0a350776-4d36-41b9-8912-d12b1c6ebcae\" class=\"colab-df-container\">\n",
"   \n",
"<style scoped>\n",
"   .dataframe tbody tr th:only-of-type {\n",
"   vertical-align: middle;\n",
"   }\n",
"\n",
"   .dataframe tbody tr th {\n",
"   vertical-align: top;\n",
"   }\n",
"\n",
"   .dataframe thead th {\n",
"   text-align: right;\n",
"   }\n",
"</style>\n",
"<table border=\"1\" class=\"dataframe\">\n",
" <thead>\n",
"   <tr style=\"text-align: right;\">\n",
"   <th></th>\n",
"   <th>mean radius</th>\n",
"   <th>mean texture</th>\n",
"   <th>mean perimeter</th>\n",
"   <th>mean area</th>\n",

```
"     <th>mean smoothness</th>\n",
"     <th>mean compactness</th>\n",
"     <th>mean concavity</th>\n",
"     <th>mean concave points</th>\n",
"     <th>mean symmetry</th>\n",
"     <th>mean fractal dimension</th>\n",
"     <th>...</th>\n",
"     <th>worst texture</th>\n",
"     <th>worst perimeter</th>\n",
"     <th>worst area</th>\n",
"     <th>worst smoothness</th>\n",
"     <th>worst compactness</th>\n",
"     <th>worst concavity</th>\n",
"     <th>worst concave points</th>\n",
"     <th>worst symmetry</th>\n",
"     <th>worst fractal dimension</th>\n",
"     <th>label</th>\n",
"   </tr>\n",
"  </thead>\n",
"  <tbody>\n",
"   <tr>\n",
"     <th>count</th>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>569.000000</td>\n",
"     <td>...</td>\n",
```

" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" <td>569.000000</td>\n",
" </tr>\n",
" <tr>\n",
" <th>mean</th>\n",
" <td>14.127292</td>\n",
" <td>19.289649</td>\n",
" <td>91.969033</td>\n",
" <td>654.889104</td>\n",
" <td>0.096360</td>\n",
" <td>0.104341</td>\n",
" <td>0.088799</td>\n",
" <td>0.048919</td>\n",
" <td>0.181162</td>\n",
" <td>0.062798</td>\n",
" <td>...</td>\n",
" <td>25.677223</td>\n",
" <td>107.261213</td>\n",
" <td>880.583128</td>\n",
" <td>0.132369</td>\n",
" <td>0.254265</td>\n",
" <td>0.272188</td>\n",
" <td>0.114606</td>\n",
" <td>0.290076</td>\n",
" <td>0.083946</td>\n",

" <td>0.627417</td>\n",
" </tr>\n",
" <tr>\n",
" <th>std</th>\n",
" <td>3.524049</td>\n",
" <td>4.301036</td>\n",
" <td>24.298981</td>\n",
" <td>351.914129</td>\n",
" <td>0.014064</td>\n",
" <td>0.052813</td>\n",
" <td>0.079720</td>\n",
" <td>0.038803</td>\n",
" <td>0.027414</td>\n",
" <td>0.007060</td>\n",
" <td>...</td>\n",
" <td>6.146258</td>\n",
" <td>33.602542</td>\n",
" <td>569.356993</td>\n",
" <td>0.022832</td>\n",
" <td>0.157336</td>\n",
" <td>0.208624</td>\n",
" <td>0.065732</td>\n",
" <td>0.061867</td>\n",
" <td>0.018061</td>\n",
" <td>0.483918</td>\n",
" </tr>\n",
" <tr>\n",
" <th>min</th>\n",
" <td>6.981000</td>\n",
" <td>9.710000</td>\n",
" <td>43.790000</td>\n",
" <td>143.500000</td>\n",
" <td>0.052630</td>\n",

"   <td>0.019380</td>\n",
"   <td>0.000000</td>\n",
"   <td>0.000000</td>\n",
"   <td>0.106000</td>\n",
"   <td>0.049960</td>\n",
"   <td>...</td>\n",
"   <td>12.020000</td>\n",
"   <td>50.410000</td>\n",
"   <td>185.200000</td>\n",
"   <td>0.071170</td>\n",
"   <td>0.027290</td>\n",
"   <td>0.000000</td>\n",
"   <td>0.000000</td>\n",
"   <td>0.156500</td>\n",
"   <td>0.055040</td>\n",
"   <td>0.000000</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>25%</th>\n",
"   <td>11.700000</td>\n",
"   <td>16.170000</td>\n",
"   <td>75.170000</td>\n",
"   <td>420.300000</td>\n",
"   <td>0.086370</td>\n",
"   <td>0.064920</td>\n",
"   <td>0.029560</td>\n",
"   <td>0.020310</td>\n",
"   <td>0.161900</td>\n",
"   <td>0.057700</td>\n",
"   <td>...</td>\n",
"   <td>21.080000</td>\n",
"   <td>84.110000</td>\n",
"   <td>515.300000</td>\n",

"   <td>0.116600</td>\n",
"   <td>0.147200</td>\n",
"   <td>0.114500</td>\n",
"   <td>0.064930</td>\n",
"   <td>0.250400</td>\n",
"   <td>0.071460</td>\n",
"   <td>0.000000</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>50%</th>\n",
"   <td>13.370000</td>\n",
"   <td>18.840000</td>\n",
"   <td>86.240000</td>\n",
"   <td>551.100000</td>\n",
"   <td>0.095870</td>\n",
"   <td>0.092630</td>\n",
"   <td>0.061540</td>\n",
"   <td>0.033500</td>\n",
"   <td>0.179200</td>\n",
"   <td>0.061540</td>\n",
"   <td>...</td>\n",
"   <td>25.410000</td>\n",
"   <td>97.660000</td>\n",
"   <td>686.500000</td>\n",
"   <td>0.131300</td>\n",
"   <td>0.211900</td>\n",
"   <td>0.226700</td>\n",
"   <td>0.099930</td>\n",
"   <td>0.282200</td>\n",
"   <td>0.080040</td>\n",
"   <td>1.000000</td>\n",
"   </tr>\n",
"   <tr>\n",

"   <th>75%</th>\n",
"   <td>15.780000</td>\n",
"   <td>21.800000</td>\n",
"   <td>104.100000</td>\n",
"   <td>782.700000</td>\n",
"   <td>0.105300</td>\n",
"   <td>0.130400</td>\n",
"   <td>0.130700</td>\n",
"   <td>0.074000</td>\n",
"   <td>0.195700</td>\n",
"   <td>0.066120</td>\n",
"   <td>...</td>\n",
"   <td>29.720000</td>\n",
"   <td>125.400000</td>\n",
"   <td>1084.000000</td>\n",
"   <td>0.146000</td>\n",
"   <td>0.339100</td>\n",
"   <td>0.382900</td>\n",
"   <td>0.161400</td>\n",
"   <td>0.317900</td>\n",
"   <td>0.092080</td>\n",
"   <td>1.000000</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>max</th>\n",
"   <td>28.110000</td>\n",
"   <td>39.280000</td>\n",
"   <td>188.500000</td>\n",
"   <td>2501.000000</td>\n",
"   <td>0.163400</td>\n",
"   <td>0.345400</td>\n",
"   <td>0.426800</td>\n",
"   <td>0.201200</td>\n",

"   <td>0.304000</td>\n",
"   <td>0.097440</td>\n",
"   <td>...</td>\n",
"   <td>49.540000</td>\n",
"   <td>251.200000</td>\n",
"   <td>4254.000000</td>\n",
"   <td>0.222600</td>\n",
"   <td>1.058000</td>\n",
"   <td>1.252000</td>\n",
"   <td>0.291000</td>\n",
"   <td>0.663800</td>\n",
"   <td>0.207500</td>\n",
"   <td>1.000000</td>\n",
"   </tr>\n",
" </tbody>\n",
"</table>\n",
" 8 rows × 31 columns \n",
" \n",
"   <div class=\"colab-df-buttons\">\n",
"\n",
" <div class=\"colab-df-container\">\n",
"   <button class=\"colab-df-convert\" onclick=\"convertToInteractive('df-0a350776-4d36-41b9-8912-d12b1c6ebcae')\"\n",
"   title=\"Convert this dataframe to an interactive table.\"\n",
"   style=\"display:none;\">\n",
"\n",
" <svg xmlns=\"http://www.w3.org/2000/svg\" height=\"24px\" viewBox=\"0 -960 960 960\">\n",
"   <path d=\"M120-120v-720h720v720H120Zm60-500h600v-160H180v160Zm220 220h160v-160H400v160Zm0 220h160v-160H400v160ZM180-400h160v-160H180v160Zm440 0h160v-160H620v160ZM180-180h160v-160H180v160Zm440 0h160v-160H620v160Z\"/>\n",
" </svg>\n",

```
"   </button>\n",
"\n",
"  <style>\n",
"    .colab-df-container {\n",
"      display:flex;\n",
"      gap: 12px;\n",
"    }\n",
"\n",
"    .colab-df-convert {\n",
"      background-color: #E8F0FE;\n",
"      border: none;\n",
"      border-radius: 50%;\n",
"      cursor: pointer;\n",
"      display: none;\n",
"      fill: #1967D2;\n",
"      height: 32px;\n",
"      padding: 0 0 0 0;\n",
"      width: 32px;\n",
"    }\n",
"\n",
"    .colab-df-convert:hover {\n",
"      background-color: #E2EBFA;\n",
"      box-shadow: 0px 1px 2px rgba(60, 64, 67, 0.3), 0px 1px 3px 1px rgba(6
0, 64, 67, 0.15);\n",
"      fill: #174EA6;\n",
"    }\n",
"\n",
"    .colab-df-buttons div {\n",
"      margin-bottom: 4px;\n",
"    }\n",
"\n",
"    [theme=dark] .colab-df-convert {\n",
"      background-color: #3B4455;\n",
```

```
"     fill: #D2E3FC;\n",
"   }\n",
"\n",
"   [theme=dark] .colab-df-convert:hover {\n",
"     background-color: #434B5C;\n",
"     box-shadow: 0px 1px 3px 1px rgba(0, 0, 0, 0.15);\n",
"     filter: drop-shadow(0px 1px 2px rgba(0, 0, 0, 0.3));\n",
"     fill: #FFFFFF;\n",
"   }\n",
"  </style>\n",
"\n",
"   <script>\n",
"     const buttonEl =\n",
"       document.querySelector('#df-0a350776-4d36-41b9-8912-d12b1c6e
bcae button.colab-df-convert');\n",
"     buttonEl.style.display =\n",
"       google.colab.kernel.accessAllowed ? 'block' : 'none';\n",
"\n",
"     async function convertToInteractive(key) {\n",
"       const element = document.querySelector('#df-0a350776-4d36-41b9
-8912-d12b1c6ebcae');\n",
"       const dataTable =\n",
"         await google.colab.kernel.invokeFunction('convertToInteractive',\n",
"                                                  [key], {});\n",
"       if (!dataTable) return;\n",
"\n",
"       const docLinkHtml = 'Like what you see? Visit the ' +\n",
"         '<a target=\"_blank\" href=https://colab.research.google.com/noteb
ooks/data_table.ipynb>data table notebook</a>'\n",
"         + ' to learn more about interactive tables.';\n",
"       element.innerHTML = '';\n",
"       dataTable['output_type'] = 'display_data';\n",
"       await google.colab.output.renderOutput(dataTable, element);\n",
```

"   const docLink = document.createElement('div');\n",
"   docLink.innerHTML = docLinkHtml;\n",
"   element.appendChild(docLink);\n",
"   }\n",
"   </script>\n",
" \n",
"\n",
"\n",
"   \n",
" \n"
],
"application/vnd.google.colaboratory.intrinsic+json": {
"type": "dataframe"
}
},
"metadata": {},
"execution_count": 11
}
]
},
{
"cell_type": "code",
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/",
"height": 178
},
"id": "tC8Yii4Yjzer",
"outputId": "7c1258fa-87ed-4684-e81c-d5a0211b0c49"
},
"source": [
"# checking the distribution of Target Varibale\n",
"data_frame['label'].value_counts()"

```
],
"execution_count": 12,
"outputs": [
 {
  "output_type": "execute_result",
  "data": {
   "text/plain": [
    "label\n",
    "1   357\n",
    "0   212\n",
    "Name: count, dtype: int64"
   ],
   "text/html": [
    "<div>\n",
    "<style scoped>\n",
    "    .dataframe tbody tr th:only-of-type {\n",
    "        vertical-align: middle;\n",
    "    }\n",
    "\n",
    "    .dataframe tbody tr th {\n",
    "        vertical-align: top;\n",
    "    }\n",
    "\n",
    "    .dataframe thead th {\n",
    "        text-align: right;\n",
    "    }\n",
    "</style>\n",
    "<table border=\"1\" class=\"dataframe\">\n",
    "  <thead>\n",
    "    <tr style=\"text-align: right;\">\n",
    "      <th></th>\n",
    "      <th>count</th>\n",
    "    </tr>\n",
```

" <tr>\n",
"   <th>label</th>\n",
"   <th></th>\n",
" </tr>\n",
" </thead>\n",
" <tbody>\n",
" <tr>\n",
"   <th>1</th>\n",
"   <td>357</td>\n",
" </tr>\n",
" <tr>\n",
"   <th>0</th>\n",
"   <td>212</td>\n",
" </tr>\n",
" </tbody>\n",
"</table>\n",
"
<label>dtype: int64</label>"
]
},
"metadata": {},
"execution_count": 12
}
]
},
{
"cell_type": "markdown",
"metadata": {
"id": "HbbkzWeFkjqc"
},
"source": [
"1 --> Benign\n",
"\n",
"0 --> Malignant"

```
    ]
  },
  {
    "cell_type": "code",
    "metadata": {
      "colab": {
        "base_uri": "https://localhost:8080/",
        "height": 228
      },
      "id": "YGWHjrVSkN5c",
      "outputId": "1067a27d-e8e6-46a1-8aeb-c6b22214ba3b"
    },
    "source": [
      "data_frame.groupby('label').mean()"
    ],
    "execution_count": 13,
    "outputs": [
      {
        "output_type": "execute_result",
        "data": {
          "text/plain": [
            "       mean radius  mean texture  mean perimeter   mean area  mean smoothness  \\\n",
            "label                                                                          \n",
            "0      17.462830    21.604906     115.365377  978.376415       0.102898  \n",
            "1      12.146524    17.914762      78.075406  462.790196       0.092478  \n",
            "\n",
            "       mean compactness  mean concavity  mean concave points  mean symmetry  \\\n",
            "label                                                                          \n",
            "0          0.145188       0.160775            0.087990       0.192909  \n",
            "1          0.080085       0.046058            0.025717       0.174186  \n",
```

"\n",
" mean fractal dimension ... worst radius worst texture \\\n",
"label   ...   \n",
"0   0.062680 ...   21.134811   29.318208 \n",
"1   0.062867 ...   13.379801   23.515070 \n",
"\n",
" worst perimeter worst area worst smoothness worst compactness \\
\n",
"label   \n",
"0   141.370330 1422.286321   0.144845   0.374824 \n",
"1   87.005938   558.899440   0.124959   0.182673 \n",
"\n",
" worst concavity worst concave points worst symmetry \\\n",
"label   \n",
"0   0.450606   0.182237   0.323468 \n",
"1   0.166238   0.074444   0.270246 \n",
"\n",
" worst fractal dimension \n",
"label   \n",
"0   0.091530 \n",
"1   0.079442 \n",
"\n",
"[2 rows x 30 columns]"
],
"text/html": [
"\n",
" <div id=\"df-7c93aae1-98fe-4be6-b751-c2a55ea7328a\" class=\"colab-
df-container\">\n",
" \n",
"<style scoped>\n",
" .dataframe tbody tr th:only-of-type {\n",
"   vertical-align: middle;\n",
" }\n",

```
"\n",
"   .dataframe tbody tr th {\n",
"      vertical-align: top;\n",
"   }\n",
"\n",
"   .dataframe thead th {\n",
"      text-align: right;\n",
"   }\n",
"</style>\n",
"<table border=\"1\" class=\"dataframe\">\n",
"  <thead>\n",
"    <tr style=\"text-align: right;\">\n",
"      <th></th>\n",
"      <th>mean radius</th>\n",
"      <th>mean texture</th>\n",
"      <th>mean perimeter</th>\n",
"      <th>mean area</th>\n",
"      <th>mean smoothness</th>\n",
"      <th>mean compactness</th>\n",
"      <th>mean concavity</th>\n",
"      <th>mean concave points</th>\n",
"      <th>mean symmetry</th>\n",
"      <th>mean fractal dimension</th>\n",
"      <th>...</th>\n",
"      <th>worst radius</th>\n",
"      <th>worst texture</th>\n",
"      <th>worst perimeter</th>\n",
"      <th>worst area</th>\n",
"      <th>worst smoothness</th>\n",
"      <th>worst compactness</th>\n",
"      <th>worst concavity</th>\n",
"      <th>worst concave points</th>\n",
"      <th>worst symmetry</th>\n",
```

"   <th>worst fractal dimension</th>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>label</th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   <th></th>\n",
"   </tr>\n",
" </thead>\n",
" <tbody>\n",
"   <tr>\n",
"   <th>0</th>\n",
"   <td>17.462830</td>\n",
"   <td>21.604906</td>\n",
"   <td>115.365377</td>\n",

"   <td>978.376415</td>\n",
"   <td>0.102898</td>\n",
"   <td>0.145188</td>\n",
"   <td>0.160775</td>\n",
"   <td>0.087990</td>\n",
"   <td>0.192909</td>\n",
"   <td>0.062680</td>\n",
"   <td>...</td>\n",
"   <td>21.134811</td>\n",
"   <td>29.318208</td>\n",
"   <td>141.370330</td>\n",
"   <td>1422.286321</td>\n",
"   <td>0.144845</td>\n",
"   <td>0.374824</td>\n",
"   <td>0.450606</td>\n",
"   <td>0.182237</td>\n",
"   <td>0.323468</td>\n",
"   <td>0.091530</td>\n",
"   </tr>\n",
"   <tr>\n",
"   <th>1</th>\n",
"   <td>12.146524</td>\n",
"   <td>17.914762</td>\n",
"   <td>78.075406</td>\n",
"   <td>462.790196</td>\n",
"   <td>0.092478</td>\n",
"   <td>0.080085</td>\n",
"   <td>0.046058</td>\n",
"   <td>0.025717</td>\n",
"   <td>0.174186</td>\n",
"   <td>0.062867</td>\n",
"   <td>...</td>\n",
"   <td>13.379801</td>\n",

"   <td>23.515070</td>\n",
"   <td>87.005938</td>\n",
"   <td>558.899440</td>\n",
"   <td>0.124959</td>\n",
"   <td>0.182673</td>\n",
"   <td>0.166238</td>\n",
"   <td>0.074444</td>\n",
"   <td>0.270246</td>\n",
"   <td>0.079442</td>\n",
"   </tr>\n",
" </tbody>\n",
"</table>\n",
" 2 rows × 30 columns \n",
" \n",
"   <div class=\"colab-df-buttons\">\n",
"\n",
" <div class=\"colab-df-container\">\n",
"   <button class=\"colab-df-convert\" onclick=\"convertToInteractive('df-7c93aae1-98fe-4be6-b751-c2a55ea7328a')\"\n",
"   title=\"Convert this dataframe to an interactive table.\"\n",
"   style=\"display:none;\">\n",
"\n",
" <svg xmlns=\"http://www.w3.org/2000/svg\" height=\"24px\" viewBox=\"0 -960 960 960\">\n",
"   <path d=\"M120-120v-720h720v720H120Zm60-500h600v-160H180v160Zm220 220h160v-160H400v160Zm0 220h160v-160H400v160ZM180-400h160v-160H180v160Zm440 0h160v-160H620v160ZM180-180h160v-160H180v160Zm440 0h160v-160H620v160Z\"/>\n",
" </svg>\n",
"   </button>\n",
"\n",
" <style>\n",
"   .colab-df-container {\n",

```
"    display:flex;\n",
"    gap: 12px;\n",
"  }\n",
"\n",
"  .colab-df-convert {\n",
"    background-color: #E8F0FE;\n",
"    border: none;\n",
"    border-radius: 50%;\n",
"    cursor: pointer;\n",
"    display: none;\n",
"    fill: #1967D2;\n",
"    height: 32px;\n",
"    padding: 0 0 0 0;\n",
"    width: 32px;\n",
"  }\n",
"\n",
"  .colab-df-convert:hover {\n",
"    background-color: #E2EBFA;\n",
"    box-shadow: 0px 1px 2px rgba(60, 64, 67, 0.3), 0px 1px 3px 1px rgba(6
0, 64, 67, 0.15);\n",
"    fill: #174EA6;\n",
"  }\n",
"\n",
"  .colab-df-buttons div {\n",
"    margin-bottom: 4px;\n",
"  }\n",
"\n",
"  [theme=dark] .colab-df-convert {\n",
"    background-color: #3B4455;\n",
"    fill: #D2E3FC;\n",
"  }\n",
"\n",
"  [theme=dark] .colab-df-convert:hover {\n",
```

```
"     background-color: #434B5C;\n",
"     box-shadow: 0px 1px 3px 1px rgba(0, 0, 0, 0.15);\n",
"     filter: drop-shadow(0px 1px 2px rgba(0, 0, 0, 0.3));\n",
"     fill: #FFFFFF;\n",
"   }\n",
"  </style>\n",
"\n",
"   <script>\n",
"     const buttonEl =\n",
"       document.querySelector('#df-7c93aae1-98fe-4be6-b751-c2a55ea7
328a button.colab-df-convert');\n",
"     buttonEl.style.display =\n",
"       google.colab.kernel.accessAllowed ? 'block' : 'none';\n",
"\n",
"     async function convertToInteractive(key) {\n",
"       const element = document.querySelector('#df-7c93aae1-98fe-4be6-
b751-c2a55ea7328a');\n",
"       const dataTable =\n",
"         await google.colab.kernel.invokeFunction('convertToInteractive',\n",
"                                                  [key], {});\n",
"       if (!dataTable) return;\n",
"\n",
"       const docLinkHtml = 'Like what you see? Visit the ' +\n",
"         '<a target=\"_blank\" href=https://colab.research.google.com/noteb
ooks/data_table.ipynb>data table notebook</a>'\n",
"         + ' to learn more about interactive tables.';\n",
"       element.innerHTML = '';\n",
"       dataTable['output_type'] = 'display_data';\n",
"       await google.colab.output.renderOutput(dataTable, element);\n",
"       const docLink = document.createElement('div');\n",
"       docLink.innerHTML = docLinkHtml;\n",
"       element.appendChild(docLink);\n",
"     }\n",
```

" </script>\n",
" \n",
"\n",
"\n",
" \n",
" \n"
],
"application/vnd.google.colaboratory.intrinsic+json": {
"type": "dataframe"
}
},
"metadata": {},
"execution_count": 13
}
]
},
{
"cell_type": "markdown",
"metadata": {
"id": "tUPYps4DlVFR"
},
"source": [
"Separating the features and target"
]
},
{
"cell_type": "code",
"metadata": {
"id": "Z5pD8rP5kzKD"
},
"source": [
"X = data_frame.drop(columns='label', axis=1)\n",
"Y = data_frame['label']"

```
    ],
    "execution_count": 14,
    "outputs": []
   },
   {
    "cell_type": "code",
    "metadata": {
     "colab": {
      "base_uri": "https://localhost:8080/"
     },
     "id": "Z-GWJHpAlpWJ",
     "outputId": "490c01f0-b1c7-4253-fefa-b2c82b53aef5"
    },
    "source": [
     "print(X)"
    ],
    "execution_count": 15,
    "outputs": [
     {
      "output_type": "stream",
      "name": "stdout",
      "text": [
       "     mean radius  mean texture  mean perimeter  mean area  mean smoothne
ss  \\\n",
       "0       17.99       10.38       122.80    1001.0       0.11840  \n",
       "1       20.57       17.77       132.90    1326.0       0.08474  \n",
       "2       19.69       21.25       130.00    1203.0       0.10960  \n",
       "3       11.42       20.38        77.58     386.1       0.14250  \n",
       "4       20.29       14.34       135.10    1297.0       0.10030  \n",
       "..        ...         ...         ...       ...          ...  \n",
       "564     21.56       22.39       142.00    1479.0       0.11100  \n",
       "565     20.13       28.25       131.20    1261.0       0.09780  \n",
       "566     16.60       28.08       108.30     858.1       0.08455  \n",
```

```
"567      20.60       29.33       140.10    1265.0       0.11780  \n",
"568       7.76       24.54        47.92     181.0       0.05263  \n",
"\n",
"    mean compactness  mean concavity  mean concave points  mean symm
etry  \\\n",
"0         0.27760      0.30010          0.14710      0.2419  \n",
"1         0.07864      0.08690          0.07017      0.1812  \n",
"2         0.15990      0.19740          0.12790      0.2069  \n",
"3         0.28390      0.24140          0.10520      0.2597  \n",
"4         0.13280      0.19800          0.10430      0.1809  \n",
"..            ...          ...              ...          ...  \n",
"564       0.11590      0.24390          0.13890      0.1726  \n",
"565       0.10340      0.14400          0.09791      0.1752  \n",
"566       0.10230      0.09251          0.05302      0.1590  \n",
"567       0.27700      0.35140          0.15200      0.2397  \n",
"568       0.04362      0.00000          0.00000      0.1587  \n",
"\n",
"    mean fractal dimension  ...  worst radius  worst texture  \\\n",
"0              0.07871  ...      25.380       17.33  \n",
"1              0.05667  ...      24.990       23.41  \n",
"2              0.05999  ...      23.570       25.53  \n",
"3              0.09744  ...      14.910       26.50  \n",
"4              0.05883  ...      22.540       16.67  \n",
"..                 ...  ...         ...         ...  \n",
"564            0.05623  ...      25.450       26.40  \n",
"565            0.05533  ...      23.690       38.25  \n",
"566            0.05648  ...      18.980       34.12  \n",
"567            0.07016  ...      25.740       39.42  \n",
"568            0.05884  ...       9.456       30.37  \n",
"\n",
"    worst perimeter  worst area  worst smoothness  worst compactness  \\\n
",
"0         184.60     2019.0        0.16220          0.66560  \n",
```

```
"1        158.80    1956.0       0.12380        0.18660  \n",
"2        152.50    1709.0       0.14440        0.42450  \n",
"3         98.87     567.7       0.20980        0.86630  \n",
"4        152.20    1575.0       0.13740        0.20500  \n",
"..          ...       ...          ...            ...  \n",
"564       166.10    2027.0       0.14100        0.21130  \n",
"565       155.00    1731.0       0.11660        0.19220  \n",
"566       126.70    1124.0       0.11390        0.30940  \n",
"567       184.60    1821.0       0.16500        0.86810  \n",
"568        59.16     268.6       0.08996        0.06444  \n",
"\n",
"    worst concavity  worst concave points  worst symmetry  \\\n",
"0         0.7119            0.2654       0.4601  \n",
"1         0.2416            0.1860       0.2750  \n",
"2         0.4504            0.2430       0.3613  \n",
"3         0.6869            0.2575       0.6638  \n",
"4         0.4000            0.1625       0.2364  \n",
"..           ...               ...          ...  \n",
"564       0.4107            0.2216       0.2060  \n",
"565       0.3215            0.1628       0.2572  \n",
"566       0.3403            0.1418       0.2218  \n",
"567       0.9387            0.2650       0.4087  \n",
"568       0.0000            0.0000       0.2871  \n",
"\n",
"    worst fractal dimension  \n",
"0              0.11890  \n",
"1              0.08902  \n",
"2              0.08758  \n",
"3              0.17300  \n",
"4              0.07678  \n",
"..                ...  \n",
"564            0.07115  \n",
"565            0.06637  \n",
```

"566 0.07820 \n",
"567 0.12400 \n",
"568 0.07039 \n",
"\n",
"[569 rows x 30 columns]\n"
]
}
]
},
{
"cell_type": "code",
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "r5rs8pColqsn",
"outputId": "ec8191ce-1a74-4ead-a692-dd4d3d22055f"
},
"source": [
"print(Y)"
],
"execution_count": 16,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"0 0\n",
"1 0\n",
"2 0\n",
"3 0\n",
"4 0\n",
" ..\n",

"564   0\n",
"565   0\n",
"566   0\n",
"567   0\n",
"568   1\n",
"Name: label, Length: 569, dtype: int64\n"
]
}
]
},
{
"cell_type": "markdown",
"metadata": {
"id": "B5yk9d-Nl4VV"
},
"source": [
"Splitting the data into training data & Testing data"
]
},
{
"cell_type": "code",
"metadata": {
"id": "vZtU30bPluG_"
},
"source": [
"X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)"
],
"execution_count": 17,
"outputs": []
},
{
"cell_type": "code",

"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "zWaeuX3amqYH",
"outputId": "d6e5916c-ce1e-45f9-d05c-ecdf762d4deb"
},
"source": [
"print(X.shape, X_train.shape, X_test.shape)"
],
"execution_count": 18,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"(569, 30) (455, 30) (114, 30)\n"
]
}
]
},
{
"cell_type": "markdown",
"source": [
"Standardize the data"
],
"metadata": {
"id": "0g1NEndK7UB6"
}
},
{
"cell_type": "code",
"source": [

"from sklearn.preprocessing import StandardScaler"
],
"metadata": {
"id": "C5EP9f_B7UXU"
},
"execution_count": 19,
"outputs": []
},
{
"cell_type": "code",
"source": [
"scaler = StandardScaler()\n",
"\n",
"X_train_std = scaler.fit_transform(X_train)\n",
"\n",
"X_test_std = scaler.transform(X_test)"
],
"metadata": {
"id": "sCGC8dku7cge"
},
"execution_count": 20,
"outputs": []
},
{
"cell_type": "markdown",
"source": [
"**Building the Neural Network**"
],
"metadata": {
"id": "VxyxgeK8qkDj"
}
},
{

"cell_type": "markdown",
"source": [
"![download.png](
AAAO0AAADUCAMAAABH5ITYAAABCFBMVEX/////z54AAAD/06H/0aD/1KL19f
Xu7u7/1qP5+fn8/PykpKRPT0+ysrLx8fHc3Nzl5eXb29tqampERETQ0NDDw8NJ
SUk7Ozurq6tXV1fV1dWgoKCcnJw1NTUqKiqQkJDHx8d5eXlkZGR1dXWEhIS6ur
qMjIyDg4MhISHhtopmZmbyxJUxMTGph2RUVFQeHh7MpHu9mHKZelkTExPFn
3hRQjNxW0TpvI5VQCnXroNfTDeui2hQRz+UdVQ9REpoUDeHa09bTD04IwMz
LihDNScyKyRHNB5DPDVKUFU7LiJoVUEpHAo9KhRkTDJ0WTxDMyEqMTglHxg
XICVGS1AxHAAbCgBONxojFwhT5eFrAAAf00lEQVR4nO19i3eiSNM3AoIgNwUF
vHETjddobiYxmThPspuZ2Zns7rvP+73//3/ydQMqIA2dTLI5M2fqnJ1N1LRd3dXV
dfIVQRC/6Bf9oI/0i37RT0jSgAH/8gOWIPpK+BI74MlfWp13m9YbkUICPgmOrBH
EaSt8SSaZ8AeRfLdpvRFtuZX3L/3U3AaSDLkIwd62zOlYgy81mtOBAbnlLb3rgh
Vp1xt6t/re0/1OUsiOUTWqAbcawZLTjncKuNXlecckTwGzDjnodFWWsEgVvGK/9
3y/jxTyCFLEbecI7K8HuDUBf4wA9lYjbYaTyQZhHdcI9qj93vP9PlLIQAFH3Dpj8H
MLcNuD6hhKcoMMyCesJnil+4Nr6ZiWgty64BcbcDu1wA/tgNuaAoghLlH42bitg
k0EAswQ3nGLkHvg3NZOh+ADLeXn4rYWcMsfk6auA25l8sjs9qBOrpJdUwCLM
FDBL70fnFvWgTcQ5wA7Cv7HdPpV1gFHWbH6fq0PP6FZ/YHPET5UUB37XSf7i
37RL0KQohmeoD/qgmdoyntP5o1J8z5cn5wtrq6uFmcn1x887b0n9HbENbqXy1
KZoilINFUuLS+7Nvfe03obUszzRYmiS3uiqdLi3Pwp5dn4bUnHWY0Yppe/Ge89t
den9u+j8gGvkMpXv//grs8h9a+B2GYTRV/333t6r0vVb6VDKd5Jc+nbu0QtmDf
SGP55DrOQ3XP/bb44l4z8wJhivWxY6XGEEuNImEeP0suG/h4q4FZ7YZTQWec
zC9hdOy8b+nslcitXObFqg19sXzGq0NjhO8BJVYDiHJJG9QUip33M1sYJdj/++2
YV5NYn3SMShkMHpHBMngLuGOiMN8BbJNk7rj9/VG9RtLXg6C6812engEJuL
ULuAsGyyA7BCl0JcCuH3NZeJMnK5SRPRUXcTi7/dZsq5Bb84lEtHKgcPKrsntuX
nVtrVry1QJRPXqgCX047bh3ILQyMKaTyvdwKo+KtBZs7El6Xl2JKcnskwzAo3Ns
GPMUv5LZ2U6yjlJVv5OLBXpUS3FqkKlrkkCc4/dQfkCQMfA/9Z6tO8QKT2wvx
DTjKo/25daEkV0myDmOGdo8U4Fsc4PnZ8lY9weT25F2TXoP56wxzhqOkgJo6
G7zK972Q+q+jNpwrTG6vhq/yfS8k8fuuBJE87c49q+phqWSglFdkh3+lqb8DnQb
pu04fe2/J03fbXammiVWrl2ot9oUjBNlKl7CWmNwuB6YrvPTLvots59Pj3cl6uT5
5Ov/giS+RsNoYMHtUw9fJt4ZmVrv/uv3lGr9dLyc0HYRCabq0uPg8eObNz9fma
kMkSeA4+V8qWNxWvvjEUNSm9pvwhCS/+3RFxSOhVHl10ms/Z39b464PLupj
E/yRcoMpyTcKoahS7V/FmrDu+YpKz4+mJhdTXBnjZVP1g7XxbfiviaWmqlUJPit6

hOlO/jXVLOuzUtbkaOrsCM+Rb9V1Pxb/ZxodrINLhaaU2SC48fhNWMuY6t9nFO
J6LI/+LDZk+ZrbjPHKK4OjunKO5d+eB5GpFnQzLaz8Aa8oynepcHmaI3XU5KGlX
a2ux
ve1MQRq2SL6GHdQeRnFlDsBAKVbK5wr2w+u80Qaycj2WbJfltRFntBRo4dc1V
wbN31m/yvjBOaFQcg3ufHVYGtLW3+PUyEeUpsWHRvOJDuKPCATAZ5pNnBD
z7QKnVn+rMpXKjoQCvY1dS8rXsgt0bkt2lz6djdPO3BGatMCORKh0w1RKi1CgX+
rgc/XjsdGW5KrjBgEC234j9Ei5CPXaB/lvP8lly2VoMoMZTvX6k2RSb/lQ3bBLLh
mrsyAZVw09xLpBFkwppmvmk03+AtWH4RBhoEOHdbTHqn4pHl8Cm/6lXRrTq
vQYu+R6aPBNFeFAoellYF9zUrU2U1fJeGZqd2v8naXWt3HJsPowXdwTj0vor6V
zroXchuA6XSo131yyHHOaY3owzvtCLzUzBBw47ZYdVLLjECoDGT4YF8BSboNr
u9g6o3zHHap1e+NxESiKK6l5qhcNeLWdeLcdgJuwWQk0t5zqx5yy98Ubi2gyk0
r9Xe2l7mvUI/EXvceVyhhLq8+J88o70bM+z10XKjeDBZY6VmH3Ml3wLHO47bx
BcclKM+Sfyl7eua+AursA+1KvW//sShnrSZdXvzRSP2lokebaqNVs00G1kifVlloMOc
Ke241+LZNDJo8ocCPqYfaZogR0YeBUD12w4HzirRqte2UCcYgyT5R+7BZVdL8
0pXV5sPh3drZJnSVJjKkMCBd0RCgNlKOm36dhKrcnlo+55PTaps0GaCdHLEJ7
wR3avhJHSCpGBZPoKd2h0n2uqh9BcqytxV5WQCa2eEJqf95XYq7G8DFKq0/
9zN0kaRulaFiojx8Tjwiya4NFTdwtvpDCJRs9UhS9kmfJOGRh1eCD3QyUesFqZ4
YKZeFJkBAldvoNCL08JbM7aYbgY1RhxKheNPNYlKullEnCf6dLG6nXrYiau1vpKH
5TPPQx4h6axscXuHBDaRM8XrofQVUNbe3JevDkl0bflgxml/vNrPlcjnb3H0VDK
Q97O1X0mo+z8PH4bYzwwsx0GcuWHpXzwfCNHqxX1hdIOf7DdKqfcdx+tXcO
Dzb2/9Bo5dWY7mkYWQLB1i5KRgsc1lvKhak0hPz6wwk4/i5ERgxNmet99pAh
WFh/jza21GgcnKJj9+0hDJ9kV9mxi5hKduufzn1cbm9OiWbZid3qzpm/LfBy2ba0
mO/SO7wVQMaFua5pRZCXdesZtOQUd/f6sY3U1ZT2oxR5FqtJit5Sg5SO3H39
HPcr+TALAZyUiz0yiJul06DNAnOHnTrRua4+5s2oGHC/uD8vqk/nF+eP+hm38+
dF68mxjGmuc61ZPQ/BQM/6qZVeMyVb1jMlsonHcKcu2Bfec1RTftQpN0EkE/WY
3vY8h4ulpNopMny4sFLW91x0oSE9DSOkFqca7nxgWcXj06+VmQfsWypUump
RTRcJ5Qy1q9P+6mFrLqJOcYuTtn7c7YqU5ETTdNUeTX708vZMSd5zclThJnac
v9cTrYD02Dgymj9m5OnG7kxHmBg9cAB78aub50DznCbjrw/U7aeOl/7/eHb0HB
MfgcNTccqUv8wzeRSZAdfuf7ns8OBJ7PHvOhH9RYnpk8FSJ+Gy6j7La0Zc7UT
SRkDfNoY8fXtxxj37tApCNyCOxd5fMUUXomruwfsKp82k4w4KV25uhyg9YJSH
CsDVLklpm822HgKg2OtudBRwOD1JPp2Z+8q5glieLp0ggyo8vWUFcUfBF+V3h
oRX6JKGxet9x2MQCi9EoIBNBM4Fkkx0yzVNQYpq82MtpZpzjl2drsLsyZqVspB+
KKqQ8VW3f6B0kOHvMA6otlVrov1FLUJDwPv2uCEppaZqbnkPOFl2pGZwbm5

dik1G6NkrnMAT7B1H/hV0Spy3WWOmUBTGzR8clBoYNCLrZVkC+BUpbeEVRs
Np+vsGDZMO/yhv8nXCZUNCo/NCQfXjqwOSDJcBN7LtwDp0h3Se2FvCoC1N
H1ubz/s+jAWk9gSPlDUbMNtei34Ro2McCqtlqmhJ9eoi9c2D16CafDjQML9LwW
Hj56gUVh2fsaGLj3tL7wWPMBO4
pi2twqTNdypZRMdMoxwc8VoOHokoGS5nw6hKSoMD8CRmcfCwCG1qCMvu
M51TvycLp/EL4RxA57H2FT2gSjwhlJtmjpJBkakuCk2wcsb1O3IptWDEARD4JZ3
TjBukTu0uWZ9m6BkI63htMBH2QcvWT11wgLcxVQhuOKoPFT2TdTm+m7yd6nR
seq9U5jbxrgzqcXhUdjR4BLBLlV+MpNKCZ5cQuna0a/1tGMXJr1MXnsqnBKkJ6
QV7B4a+ZzSkrBkBuK8c2xx8X6dUctCVxZ/ptFMSuCDKb2Q3U4aps2RpOpYVZ
moY4FqqCUS581m5wz4OZatW0bmriC1mnejSuL4AvN9sskIloRWod+DW17rpe
2hqlsL8zk3mKHbG6Sd18m8n5g/8QZembmetPjPt7NJuRwYn9BTmSw2f2WtT2
0a/M8H2kk6yGGw3ejWtT9iYmo+2qgJ8WaWlDc2WB45PXnl9/4Y3/3P0wxGfi
ulq/XmdxURCa2HetSaE95BpqW6jTwYBZbFjtsNOozZStxPWngPWphArNJTMfz
J7tc/fdA/fBo7OfmA6M7pHB2cOW4n2R3MgBe1zumlMli/55OB+sfTBxC/glUWy
UkKmx88ilx1jTwQ9P1R62Pll+BVkYPYleKKCl5twFd3MQfOV1PPlDklkiuq7aaWT+n
tnHjvChPDOvpL9EUE+Z6we8/vh3BRAXNgav1aYFEHHiHXAjop6Yjuo6o8NmJ3
9BfZN6oIMkx39154kQ9UzlExueUYSSqIVkpdlmhDS1RJKGUGnFpWVsBB4DEzp
YEkywLaEJD13XUPJXmsYEsyVSzJvN3pu5/++cd0LD8vkDus1sKMjRa/clcd6AG
Rp7o7bFvP0FKKgM747JQ81FLwph+/kpbiNO/xy8nZBNJidnFjohmW1W0gyt9b7
7XgOl8DgRPF59xATFoBxKgZfRFhW8E2F7vjET3lQs1889t6UoERSxitLFcmyycV
VezNmzunr21uxd4LLkQbMus907pghkix09TUNF/DulDq14etC0YXgp356aq7S6
MT/SgbBg1oXnaEbpAiZ7Atx+C24y0P5ZDGPF2JrUnM75iW4yf0Ndq4n2XGK8+
+ZsV9az0pltQahzcmsJ/9ulrlfDJo9PZcr0BEJeP5bmAUsQ3DUsljBtMroNZoJVV
9GGWDWOnJ9aGTwqoaoUx3epsL4Ada1zfNBnzxKNgMe4O1BU/2dhgfVV3gB
65q6M27EOCFc0bK6MreNsq9hd78bT19H43hsln7tZNUkWDNUyfSrl1g1pz6XG
++dRh6CynwusJljUbgefM02psX86r5aTodrzSCKD8Tq3yQdU9X0yqweosRqbm
NK2N5G6xMURi10QCzp3AhsfAET9lDgWHyPUa69JS4H7TlqbN2Bq7mNLu9g9
EZoSCWCcHAQkKVJAJe8YUD660Iwx4ZmOFYUbgxQukx84lKALr0GDN2uK21
qEzDmdquKkoGeagB7bsCkaNLd6k1Yg59yHCKLUWFqNTQ1zSeCk4uXULGaQa
zolMfD2nx+5RPB5g5rKjDKCRhDoVDU8Qp0CeV20NTtu9lWa1yrwm/JXC4tQFR
X+cul5BGlKHA4uD/n3amnbgvauT1Wkf3WlBkbJcYpNUCwzPNZR67lWVWlqiah
djVyO2xkfqnx4T0dy7yuXKCjCY762JlQq+2UYRWDNAkN0kr/l1rAsEZxu2Dmj+Hx
ej6Gs1uZa1n3v+N+UHQodrVQmtGacNKMvDV/5Nz+ugZMuvF4mU0r8PNZZq7

Ta650/YW6hnciJywFR/O8HokCUEncneJSPLRlWUXEUpppdW7AXSyD699rk6
G3ALn4ww58C26LLCKF52N+lJso8dSQ4d+SzV8ldODeUtRKpvvBlZAoD+5fy4
yr3JqcvEP0q1k1YRPNGwGXx1cDBArehqAJz/cZg5cHl06ylH5OiYS4TMcwgg9A
N7Q5xr8iQu3ZwtgY8nlPKhCbsOgGTP477JyABioLP9r5fjQUgwjxvfDUKkcCpLp
W2Qg1lz39epwYGr9R06c6zkokyjll1u9HZTDg
lYHo25FUiYji8MJ6yjCF8cPARhkN6PyZPkwzg8l8o4VKQFm520Z8AcLruFWh3w
6X0zKdGzg1bLn5rnlSl4HqBiVT9rhTVtzejFEnAStHGOf8rNDuF/H9cnTPT+N+tfNc
lIJabLcfE0jDTLYtcJB2bm1033AsJQTaSOmlfy+OSttB15v/qjnQZOAfsFFh609ol7l
bFNI+t1GnWDjGDgRQuAMlHu1hDPBiJ6gn3+8/niuC14u5nc/kinBSFDs6+QmW0
9b0lK13gwGftTnTmEJHj7yb+ybxtxM49g4Va4mEhideoRVSLPEyZrdsDUZu7Vh
Q2BrSdDuoJ6FMJBqwcAKxsD4qM6jl9XJOGx+PYWaHnoH/sELqTVNweCU42c
hlg/pOYhdN4sLRk/lphTyBd2ZMsnX68mcW8dDZ9yxCB+Nrbuu2+wcylvSSzLH6
aLwOmXS1aYi1eND1eZM8dHMJXykfV3R5VZHNdtJwe1bCR+cMzuEEnhJKe3la
2K70+m0RQ1zewZmUP8Rkxy3kQGkljhb7AQD56vjgPDiKTAWPSRsVSK4Wr9p+
vuvVJpMly5eDtQjNbVFGN3Yq5Lv3ny8uJ2tZ7cXH2/GuaHqkHjPC9Wc5WzVX
WDDGEk9xYomGPhkO7BdMDBuhUwQZAgT8Vxj2N3dlsMqsl/3q9oOwYsNwSJ
Pd1uuDLrBdRtUyMDl7aY7KKg3YMz+drHakYkvNYM1Nm1C3n55q3+/OQsHLoO
BR8unaTUX3s/iVj99g/Pbm8n1Zr3FRDDkPR5EjNyEwHqPbkq+o5+s6ET1E7060
XPbtUjC3qYgbDO4CgbhEbZV72gebKFkPc4myYFLk4tu7sl2MCvbHgO7WN2Nx
Yr1rqWFieu5Hb62TZOwgasSekTyJ0Rl2ye07SgnC3E06E/WAp9TEU0Ym4c/tvT
NJGPg0Tc3Z3v9CyzEbhSdlWMweJ41muRQ4aEvH+xE6yhklrcCHyiYk3afjTWl6
fU9KgRaU1P7A3wifhx8ujGFl8Olh3h0lTkwVZo10fqKw6xljaZm6/GV89pVQa22C
Ncm9jAbuChVsLuwFkj8jGixC12zz9lC5+/QSTuStqh3gq/qsAMBlf4HVepKV85+Q
1dYtU+KRRnYjduPx8tgWipPMPJAdccmtN/jpgCnGKcKoT1coS+48tVD1qz8NN
wsYHcXqSHYzqlF+J+RAfDANkBac0zx5tKTy/0hG+4jZ/PlkNMGx6bv7gyBnQEi3
yN3NmB3dH94djt61rHrV9u7m4iQtNp/c5gFkrhAl2cbhSe3Ek9zM82tAMaKO+zp
ETm2o1928xJyg3AwDHfQla2TWatYa+4iCQG7/xS1s52hi/q8AuuRGiXWW4kKf7
h5TOR6AvAPesOGBLPKkS3ZLvQmqdtUJNTLzmcE5poNFklLhh4WxoQr6E6Cy
n3O6YlhpP8kT5lWxh3tWAM5McAkK+JcHSoWGWJtlOKwCD1R41vJOQcpp4D
8cPk0c3gKvxqj5yk9QqL4iVrveZ1bjCAsFtMmvspE7opUnQflF/DjlA8NH1F5HcMP
sc1sKLoSYS80lYz/4CAvqJzGifb/XqGqHajJ/zuMu3tg8xp7oZOnrT1uTSND1DTzg
NWV53G3ByyqGD8qkvPJ0GZp4XQBze14WuuuM+1lmr46zToBapvfy6A8hbu8B
dg2A3xyncNrlVa+3bJYO0WU4cnzaB15v++eqhlefiWq6ckmZf4tww4r05u/Mw0

TpWvt3Frg0cL/2WEpKUN2rYYm84SJZZLuUkzasZ09M96Mp9xaDQkPMEWd5
XbMF7ubhPFeoqjJ7N5CHPbW7rFyXDMyar2EEOB2SqNvAhnxj1AGQcrHI1i8KCl
QgLnukNL+v6dlqVKGjhlVqVCLzVcHaYKJ28lQor7VM8mK2+d0wYN1p8i6LzX1jo
GRBC/h9KHmGuPm+cVsfbacbT6qbhUdCWW7UhTw7e/ry+vxzcXJzEMKDq41R
+7DPjbjh9a+gxlbojd2PreAJNs3OgOrKmZU2Mao7UStLYwYwrsV74iA373Slpwx
cl21vWoFOhmmF
N0zvGhLefZaT/bgYTGfAi5cvxuPjcSKbp/RddareyzHlGhub9+CyCTy2MDsJQitx9
dq3huW2dW6qTJvZc/7MzCsJDmfC9k0F8j9z3pYjNJ9XQxrMSnd8AY0yJQWi2
2ug4tPvvlkFkGSpsZ+EQNmZRMbn4zz0BapFeQZ8j4S4eCkuZkaUN6fXNxu0XR
et+hhPGIDJLmLaTdCKsaeMw1HuLn5COhGqIssllAmhUgpyQWHKKUJnJ3x18Y
MVMPklYpaifJr/9iCB6WgPnNH1G1+i4TW8H6zHoUJQnq0vP2MSDwOw3xwvQ
8zmsmPsLpmiX5DeZUu79w8Yccpof2B1wMU0Lc8+ZTGj+sVvUv+0sCUWpxno
cJrYVw8rKduqcnwg2McgdPVFeraDaZ1cdCDbUdGpppp4VXMAX8jx5Zq/7Yup
Xqa0XT57M9DEEPYoMGKbCk/6Ugqeid6tO13P52BbWZE/rla6wEnbAgGRmfIO
Od6lNW/jZpsDurzg0CG39yu3CCZ9RoG/i1QEH2sgxv1rcqiYQqPzvqDuiscG3gR
8MoG6eCy5qaSPQRdWf+dlNXAPmzErApvr/s4u97s6WTQNxQL45BjzdqpitTlvy
XZBo4FTpduUlLMuSfocmN68Vec3aD+VoljOHd4H6baVBvSQAgiyQTvYhg91BU
KfZlSUZCCDlTDcwKGKFNLZKOAcS4esXx2HztAsOmikgSsytCO5GxvOoRcsk09
nKf9EYNbJGa6enhbMseBQiD8p0Ju6ck5amurBVqOmu3XqQYhtnrqimyobFtQt
3nOwfbT9cLAVBlZfZuloqweSQaId6FQaspII7mV/2S1Eox67BTGvEFwaZOC0VQy
Bslgth6p9FiglqnRI8r58Q5MA2ZcJwZksJKt+4lZU1cCYmB+XKjj6NVjdLrsJoSRJ
P6ea4PTOo4rQGuLRjOukZ1MgmEr16gyoNZBYxpu7nEEG/Vn7Gxy2wLTk6+oA2
Jj3NZbwYCJ6U78HmO0enha9ZhFxO46wll5eGy6dleyZPlmOm7TCnvebxuE1Te
ozuXBwOdI+wzHyqZLoXSCW8GPwWy5KjitodjKcd9vj8H37nL61Nwh0xfV9KnT
ekkpYOboW4SafEOXUmHVZlXVDnyzpe3wyZzm6bFWXVqsPwW3NyetS9RTQ
0eXyDnJ6bYQRu8gRudmY2OhMviMDpx3sMl89BVswunXlW0Hlq4jpFpuWrEUT
qypTufzlrO/1NlnNNbUSwliNaM3HGd9zRqYqsz+D+37MJ/wrM7J7xKEYofZEKZ
W14cHE4ilOqXYRsvN61GqxxdVGl3n9B1tpBBYnpBpgdj/Pl2SXSvocmlxXs8r38P
JX8AJbhqEaApw0SVDUMWMlbnmfkvaMUuGE6d3Z6VKWA0Ju7eVlnfTnL5/jJ
BYSMlFoauZ9v3mrByVWQKfrVJaf/snF2SC6T8FWlnttgmu5ageoqVjTFNxCaXK
NVz1y+xqtFqtRlezL6rbyENgdhliriDa8wfEi+7NxRoOPAlDf7zx7DxewfnHwCEEe7
v0RHlgGXPBQLuN2tHuvXSEX/HbnqDqquC1/YIuhlkYv3Zc0KRR9jv1YGCn3Shs
+or3RMQAy9DsOk2vlovbq+6cVdihXtKSfahgrJQrxP158Wilf1hCdkhwYA4LCPwc

DOtYLOwj6+yUsVj39TkW7DpJjfizVSz9dR9NiM9tD+imYnzitnE2Z2wRU8+kOC
Br6L7yI6GwJXnhElq/O7QLxouaJNj6FsWVoGL4akxFSSay3PqlhK+I4CXlisI8JxcG
KSzKaqlRvCYinrEHY3MuzM3xwGbQTMjq7j0lVj/xWlTDvYFOohOp9XtDO28a1d
AW6JyG6AtIjO9+3szOrlaT1dVytvns+qgVc3cqSutitZI5Hj3HuoilFVXByNngwKayi
Voz4pYX1eurlHYbUJleXV2nnzsT0T4W1dC/s4Qgk/Atxzh/LWeac4LNNu8cAYV
WDXanpd6NKqlmC5XRnZoRRpZ2uKx2ftfkFxMeZoMezZObwRqqieqSqwjzElsN
VXL1eJHxx
DCqtDg+lNROdNB5K52Qfy1qYcTKsjsRasOeo2XdMba6q2vjht8mCI9v8q2f+mM
lgjJI7ts9zwwH+0KXbrlWWxF1wTjYYD960hUk5wlZvKBLFynrMko7sQUPRPouw
ulESJ2gTPOak95grurubp/BJjdSs0mEKKKuybXTN1DGO8KlctOrB7QVpYjdeXJ+
XGtlkjAZ6edHM+nSeVzxhiVi9g6a9DYkF6IvqTxcGRzhYlPBRkmE8kfBwNTkj/0BC
askqr23UcZ7aj/lP8asvD5sxJ0ixVDnu2jQVg6K+1BT693RDa2oYbr76huQd5lTryx
Vll2cQWpOt14LlGrkkuJkpypftrLsakGY4gVe03OJEbIaEG0ntJhi1lwCFT03OKJBB
glAvo4DrN12/oUFY4yKbmn9msTNN5kPDYV2zxr7KalE8LyvoUeSEHUqY2U0l7
wAK7CEVvA4ntcjxvm2yshW01Tp9sPzTpLSifoh4HU0i7lQVpuwe99Xf/ksMrqzSR
qJAOy7+2cLVzXsC0fgZFmDviEEhCAzBsYTQ1+RFO/P9SSMg5bCznCl5WVO70
EUBc8kBzfSc1Amdbv9yjGZYll89WK5msCnlEwmq8Xm0cWtlY0P4rZbEsMQllb
FXlggEsdD8wVBne8lSbSEx8vLu2+XD80h+lljOlSPDqvyqvq6Dz3CJ4ZlWUVhC/
rOFhMu8o86syzytbC170U8PqrzL1T5xl9Dz0Ds/o/93pP9fhrigqYXr9X49z3pOUj
7H58wSztSPQ5/VHpehcyPTsoNFrMlNB7xhyK82o7c54H8QCRi9TmmkA+R+bGl
xatIffgpBBlmlXCqjX+G+wcSxrPvEFH5H5KMwv5GdM5DRn404udn+bJcOTPfyd
N7C5L13L4U1OhfD1i8Kdn/XeU0W1j9Yb/3BF+XGr8hd5ca/fYW6fd3Je3vdSZC
GVbe5D6L/cck2b04fDgATY0u3J/qzO6oc3+yqlD75rA0VVmd3OdgNX9skvqPF
4tJiQ664NGlyeLisf8vZLfejRjRUS83s9l6NttcqjmPDPtZiFXs9mA4aNvKT+lG/KJf
9lt+0S/6Rb8oQf8fXczgLrmpUEkAAAAASUVORK5CYII=)"
    ],
    "metadata": {
      "id": "sC4KsFDyqyvN"
    }
  },
  {
    "cell_type": "code",
    "source": [

"# importing tensorflow and Keras\n",
"import tensorflow as tf\n",
"tf.random.set_seed(3)\n",
"from tensorflow import keras"
],
"metadata": {
"id": "1WX_ChHfqrV2"
},
"execution_count": 21,
"outputs": []
},
{
"cell_type": "code",
"source": [
"# setting up the layers of Neural Network\n",
"\n",
"model = keras.Sequential([\n",
" keras.layers.Flatten(input_shape=(30,)),\n",
" keras.layers.Dense(20, activation='relu'),\n",
" keras.layers.Dense(2, activation='sigmoid')\n",
"])"
],
"metadata": {
"id": "rL0jvBLY3Lkq",
"colab": {
"base_uri": "https://localhost:8080/"
},
"outputId": "d7311608-3e7d-4e49-e19b-3837ea22881d"
},
"execution_count": 22,
"outputs": [
{
"output_type": "stream",

"name": "stderr",
"text": [
"/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape` / `input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.\n",
" super().__init__(**kwargs)\n"
]
}
]
},
{
"cell_type": "code",
"source": [
"# compiling the Neural Network\n",
"\n",
"model.compile(optimizer='adam',\n",
" loss='sparse_categorical_crossentropy',\n",
" metrics=['accuracy'])"
],
"metadata": {
"id": "DrqX3ZGv48Ef"
},
"execution_count": 23,
"outputs": []
},
{
"cell_type": "code",
"source": [
"# training the Meural Network\n",
"\n",
"history = model.fit(X_train_std, Y_train, validation_split=0.1, epochs=10)"
],

"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "4aullV3v584B",
"outputId": "79a3acbd-0dd8-4aba-a53a-d8b811d4e502"
},
"execution_count": 24,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"Epoch 1/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m1s\u001b[0m 23ms/step - accuracy: 0.6657 - loss: 0.6222 - val_accuracy: 0.8696 - val_loss: 0.4351\n",
"Epoch 2/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 7ms/step - accuracy: 0.8668 - loss: 0.4075 - val_accuracy: 0.8913 - val_loss: 0.3096\n",
"Epoch 3/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 7ms/step - accuracy: 0.9158 - loss: 0.2957 - val_accuracy: 0.9130 - val_loss: 0.2434\n",
"Epoch 4/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 11ms/step - accuracy: 0.9483 - loss: 0.2333 - val_accuracy: 0.9348 - val_loss: 0.2037\n",
"Epoch 5/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 7ms/step - accuracy: 0.9543 - loss: 0.1955 - val_accuracy: 0.9348 - val_loss: 0.1779\n",

"Epoch 6/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 7ms/step - accuracy: 0.9575 - loss: 0.1702 - val_accuracy: 0.9565 - val_loss: 0.1594\n",
"Epoch 7/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 7ms/step - accuracy: 0.9581 - loss: 0.1519 - val_accuracy: 0.9565 - val_loss: 0.1451\n",
"Epoch 8/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 7ms/step - accuracy: 0.9634 - loss: 0.1379 - val_accuracy: 0.9565 - val_loss: 0.1335\n",
"Epoch 9/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 7ms/step - accuracy: 0.9634 - loss: 0.1268 - val_accuracy: 0.9565 - val_loss: 0.1242\n",
"Epoch 10/10\n",
"\u001b[1m13/13\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 7ms/step - accuracy: 0.9716 - loss: 0.1176 - val_accuracy: 0.9565 - val_loss: 0.1163\n"
]
}
]
},
{
"cell_type": "markdown",
"source": [
"Visualizing accuracy and loss"
],
"metadata": {
"id": "QZCz8JJ98guH"
}
},

{
 "cell_type": "code",
 "source": [
  "plt.plot(history.history['accuracy'])\n",
  "plt.plot(history.history['val_accuracy'])\n",
  "\n",
  "plt.title('model accuracy')\n",
  "plt.ylabel('accuracy')\n",
  "plt.xlabel('epoch')\n",
  "\n",
  "plt.legend(['training data', 'validation data'], loc = 'lower right')"
 ],
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/",
   "height": 489
  },
  "id": "pNism-Z55zmw",
  "outputId": "126af21d-dbc1-49cc-eff9-7cdf258519ef"
 },
 "execution_count": 25,
 "outputs": [
  {
   "output_type": "execute_result",
   "data": {
    "text/plain": [
     "<matplotlib.legend.Legend at 0x7c05b67f9280>"
    ]
   },
   "metadata": {},
   "execution_count": 25
  },
  {

"output_type": "display_data",
"data": {
"text/plain": [
"<Figure size 640x480 with 1 Axes>"
],
"image/png": "iVBORw0KGgoAAAANSUhEUgAAAkAAAAHHCAYAAABXx+
fLAAAAOnRFWHRTb2Z0d2FyZQBNYXRwbG90bGlibHZlcnNpb24zLjEwLjAsIGh
0dHBzOi8vbWF0cGxvdGxpYi5vcmcvlHJYcgAAAAlwSFlzAAAPYQAAD2EBqD+
naQAAYGdJREFUeJzt3Xd4FVX+x/H3TE89hBYlBKRlb1mKiooGUH4WBEWUouJ
asLGooDRBRSwsCCq6ghUVRCwrCklUXloE6UgRQocQSEghvdz5/XHN1ZhQUi
fJ/byeJ89m5p47850ky/145pwzFsMwDEREREQciJPZBYhUNQUgERERcTgK
QCliluJwFlBERETE4SgAiYiIMNRABlRGHowAkliliDkcBSERERByOApCpCIlg4HA
UgEalShw8fxmKx8P7775f6vatXr8ZisbB69eoKr0tEHlsCkliliDgcBSARERFxOAp
Ailmy8jlMLsEEYejACTiYKZMmYLFYuH333/nrrvuub
broJPz8/6taty2uvvVbsGKdPn+bee+8lLCwMDw8P2rdvzwcffffCsXUpKCiNGjM
Df35+AgACGDx9OSkpKiXXt3buX2267jaCgIDw8POjSpQvffPNNNNma7xyJEjPPTQ
Q7Ro0QJPT0+MZNGgQhw8fLrLrHGJ554goiICNzd3WnYsCHDhg0bNjMTHR3iY
7O5spU6Zw2WWX4eHhQb169Qb169bj11luZ++Pr6
8vQ4cOBeB///sfgwYNolGjRi7uxMeHs4TTzxBVZWiT+vwYMHExoaiqenJy1atO
DZZ58F4KeffsnVqhUvvfQSy5Yt4ICLmuP3222nVqhUvvfQSy5Yt4ICLmuP3222nVqh
UvvfQSy5Yt4/nnnycOnTp1qoMOGjZsyY168+bb8ufLKKKwHlysqid+/e
HDhwgNGjR9OkSRPHHH

RB3XffffZv3d2dqZLly4YhsG9995r3x8QEECLFi04ePCgfd93331H3bp1GTJkiH2f
q6srjz76KOnp6axZs8bezsXFhQcffLDleR555JEidZw9e5Yff/yRwYMHc+7cORIT
E0lMTCQpKYno6Gj279/PiRMnSnVtnp6e9u/z8vJlSkqiWbNmBAQEsGXLFvtrX3z
xBe3bty+xh8lisdjbhlSEFKv7r23K4q8/l5LqzsjllDExkR49emAYBlu3bgXgzJkz/Pzz
z9xzzz00atTovPUMGzaMnJwclixZYt
+3aNEi8vPzueuuu8pct0htoQAk4qD+/uHp7++Ph4cHlSEhxfYnJyfbt48cOULz5s1
xcir6z0erVq3srxf+b7169fDx8SnSrkWLFkW2Dxw4gGEYTJw4kdDQ0CJfkydPB
mxjjkojKyuLSZMmER4ejru7OyEhlYSGhpKSkkJqaqq9XVxcHG3atLngseLi4mjRo
gUuLhU3YsDFxYWGDRsW23/06FFGjBhBUFAQPj4+hlaGctVVVwHY6y4Moxeru
2XLlnTt2pWFCxfa9y1cuJB//OMfNGvWrKluRaTG0hggEQfl7Ox8SfvANp6nslitVg
DGjh1LdHR0iW1K+4H9yCOP8N577/H444/TvXt3/P39sVgs3HHHHHHfbzVaTz9QQ
VFBSUuN/d3b1YgCwoKOC6667j7NmzPP3007Rs2RJvb29OnDjBiBEjyIT3sGHD
eOyxxzh+/Dg5OTn88ssvzJ8/f8ssvzJ07t9THEamNFlBEpFQaN27Mjh07sFqtRT7E9+7da3+
98H9jYmJlT08v0gu0b9++Isdr2rQpYLuN1qdPnwqpccmSJQwfPrzlDLbs7OxiM9
AilyPZtWvXBY8VGRnJxo0bycvLw9XVtcQ2gYGBAMWOX9gbdil27tzJ77//zgcff
MCwYcPs+1euXFmkXeHP62J1A9xxxx2MGTOGTz/9lKysLFxdXbn99tvvv

ulFEREyVn59P/fr1GTBgAPPnzze7HJFqQ2OARERqsa+++oozZ84UGVgtlqAelBG
RWmjjxo3s2LGDadOmERISUmQBSBFRD5CISK301ltv8eCDD1KnTh0+/PBDs8s
RqXbUAyQiliIORz1Alili4nAUgERERMThaCHEElitVk6ePlmvr2+5nvYsliliVccwDM6
dO0f9+vWLPW/v7xSASnDy5EnCw8PNLkNERETK4NixYzRs2PCCbRSASIC4lP
+xY8fw8/MzuRoRERG5FGlpaYSHh9s/xy9EAagEhbe9/Pz8FlBERERqmE
sZvqJB0CliluJwFlBERETE4SgAiYiliMNRABlRERGHowAkliliDkcBSERERByOApCI
ilg4HAUgERERcTgKQCliluJwFlBERETE4SgAiYiliMNRABlRERGHowAkliliVcYwDP
YnnOPMuRxT69DT4EVERKRSnUzJYt2BRNbHJbHuQCKnz+XwbP9WjLqyqWk1
KQCJilhlhUrJzGVDXBLr4hJZfyCJg4kZRV53d3EiJSvXpOpsFlBERESkXLJyC/j1y
FnWHrAFnl0nUzGMP193skC7hgH0bBZMz2YhdGoUilers3kFowAklilipZRfYGXHi
VTWH0hk7YFEthxJlbfAWqRN8zo+9GwWQo/lYKKaBuPv6WpStSVTABlREZEL
MgyD/afTWXcgkXUHkth4MllzOflF2tTz96BnsxB6NgumR2QlYX4eJlV7aRSARER
EpJgThQOXDySyLi6p2Kwtf09XujcNpmfzEHpGBtMkxBuLxWJStaWnACQiliL2gc
tr/5itdaiEgcvdmgTRlzKEXs1CaF3fD2enmhN4/k4BSERExAFl5Raw6fBZ+0ytkgY
utw8PoGdkCD2aBVeLgcsVSQFlRETEAeQXWNl+PPWPW1oXHrjcs1klUU2D8PO
oXgOXK5lCkliISC1UOHB57f5E1sclsvHg2WlDl+v7e9Cjme2WVo/lYOpU84HLFUk
BSEREpJa42MDlAC/bwOXC0BMR7FWjBi5XJAUgERGRGio5I5cNB5Psj5n4+8B
lD1cnukYE2W5rRdb8gcsVSQFlRESkhrAPXP5jHM9vJ9OKDFx2drLQvqH/HwsQh
tCpcQDuLrVn4HJFUgASEZFalTUrj2NnMzl2NpOjf3wdS87i2NlMEk1+8nhFycorlN
9qFNl3WZiPfWp6VNMgfGvxwOWKpAAkIl1Qm6+lRMpWfaAcyz5z7Bz7GwWq
Vl5ZpdYJRoEeNljMtj+mAlHGrhckRSARESkWjAMg8T03D8CTdGenOPJWcSnZv
G3zo9iQnzcCQ/ypFGQF42CvAgP9Cl8yIswP3ecasFgX3dXJ+r6eTjswOWKpAA
kliJVJjM3n2Nns/4Wbv7sxcnKK7jg+z1cnezhpmGg159BJ8iL8CBPvNz0sSaXRn8
pliJSYQqsBvGpWfaQcyz5L+NxzmaRmH7hsTgWC9T396RhoOffwo3t+xAfN/V+
SlVQABlRqWhWK2SnmF1FpUnLyud4chbHUzl5kZLF8eQsTiTbvo9PzSav4Pz3qf
wBPw8XGgTaQk6DAE8aBHjZvg/0pJ6/J27OJQUcA8iArlwSXpMaycUD3LzMO7
1pZxYRqW0MA/b8F36YAClHzK6m0vgBrf/4Ksb1j6+LSf7jSxxXrzHQZ7Jpp1cAEh
GpCKf3wPdPw6E1ZlcilpdAAUhEpDyyUmD1SxD7DhgF4OwOPR+Fno+Ba+m79
/MLrKRl55OalffnV3YeaVl/fv31tcLttOx8zuXkF1kUryzcnJ3w83DB39MFf09XAr3/
nFVVOKOqYaAGG0tFMHcsl/6CRUTKwloAWz+CmKmQmWTb1/JGuP558v0bcz
Yzl7TUTFKz8ouFlr+Hl79+n5F74VlQF+YEgKerM/6ervj9EWJs37v++b3Hn9/7e/3l
e09X3F2cNMhYHllCklhlKRlHNlCw7ElcTu8E4KxXE74Me4Sf0ttw9N3DnEzZU2y1
3tLycXf5S3BxKRpa/hJc/P4WaPw8XfToA5FLYHoAeuONN3jllVc4deoU7du3Z8

6cOXTr1q3Etnl5eUyfPp0PPviAEydO0KJFC2bMmEHfvn3tbaZMmcJzzz1X5H0t
WrRg7969lXodllK7ZOcVcDw568/VhpMyOXfmCNHx87gmdzUuQJrhxb/zB/LR2
evlP+sCJNrfb7GAr7tLsR6Wwh4YP88S9ttfd8HF2cm0axdxBKYGoEWLFjFmzBj
mzZtHVFQUs2bNljo6mn379lGnTp1i7SdMmMDHH3/Mf/7zH1q2bMmKFSu45ZZ
bWL9+PR07drS3u/zyy1m1apV928XF9JwnltWM1WpwJj3HtkZN0p/r1R
z7Y72aU2nZ9rbu5HKv8/dMcvkKb0sOVsPCooLevO9xN7516/F/QV40DPrronye1
PH10FO3Raoxi2GUd8hc2UVFRdG1a1fmzp0LgNVqJTw8nEceeYRx48YVa1+/fn2
effZZHn74Yfu+gQMH4unpyccffwzYeoC++uortm3bVua60tLS8Pf3JzU1FT8/vzlf
R0TMdS47j2Nns4qsNmwPOclZ5OZbL/h+bzcnBvnu4qGcBdTJPwlAanBH0q5+
gdAW/8DDVbeaRKqT0nx+m9Y1kpuby+bNmxk/frx9n5OTE3369GHDhg0lvicnJ
wcPj6lPffP09GTt2rVF9u3fv5/69evj4eFB9+7dmT59Oo0aNar4ixARU+UVWllPyf
7basN/PkMqOfPCD8d0drJQP8CD8MDiKw5HGCfwXzMRS1yMrbFPXbh+Gv5t
B+GvQcliNZ5pASgxMZGCggLCwsKK7A8LCzveJ3o6GhmzpzJlVdeSWRkJDEx
MSxdupSCgj9nTURFRfH+++/TokUL4uPjee6557jjjjvvv

hp1WWR2qBGrAMkljVPUnoOGw7abmmtO5DE0bNFBy57ujrTrUkQPZsF0yMyh
Nb1/Mo/QLm8cs7Bz6/ChjfAmgdOLvCPB+HKp8BD/4Ej4qgUgETkvDJy8ok9fJb
1BxJZeyCJPSUMXO4QHmAbxxMZTMdGgbi5VJPeFKsVdi6GlZMh/ZRtX7M+ED
0dQi8ztzYRMZ0CkljY5RVY2XYsxTaO50ASW44mF3uqecu6vvSlDKFX82C6NQ
nGx70a/jNyYgt8/zQcj7VtBzaxTWu/LNr2lFlRcXjV8F8uEakqVqvBvoRzf9zSSiT20
NkSBy73am
Ybx2PawOVLlX4GYp6DrR8DBrh6w5VjbSs5u1TjukWkyikAiTiYwoHLaw8ksiEuia
SMEgYuNwuhZ6RterqpA5cvVUEexL4Dq1+CnD9u07W7Hfo8B371zK1NRKolBS
CRWi4pPYf1cUmsj7OFnmNns4q8XjhwubCXp1XdajBwuTQOxNhmdyXus23X62
Cb1t4oytSyRKR6UwASqWUycmwrLhdOT//7wGWXPwYu92gWQq9mlXQlD6g
+A5dL4+whWPEs7Ftm2/YKti1k2PEucKqklaNFpNZQABKpBXadSGXl7gTWxyWy
9WhKiQOX/1xxuZoOXL5UuRnwv5mwfg4U5lDFGaL+CVc9DZ4BZlcnljVEDf5XU
ETSsvOY/t1ePo09WmR/w8DCgcsh9IgMJsSnFgwANgzY9QX8MBHOnbTta9ob
+s6AOi1NLU1Eah4FlJEa6qd9p3lm6U7iU7MBiL48jN4t6tAzMoRGwV4mV1fB4rf
bprUf3WDbDmhkW8+n5Q2a1i4iZaIAJFLDpGTmMvXb3SzdcgKAxsFezBjYjn80
DTa5skqQkQQ/ToPN72Ob1u4FvcZAj9Hg6ml2dSJSgykAidQgK347xYSvdnHmX
A4WC9zbswn/ur4Fnm61bNBvQT78Oh9+egGyU2372gyE66aCf0NzaxORWkEB
SKQGSErPYfI3v/HtjngAlkO9efm29nRuHGhyZZXg4BpYPg5O77Zth7WFfjMgoqe
5dYllraIAJFKNGYbBtzvimfzNb5zNyMXZycL9VzblsWub4+H6R6/P6b2w5iVlOW
ZusRUhPwcSdtq+9wyCayZA5xGa1i4iFU4BSKSaOp2WzcSvd7HitwTANpX95dv
a0a5hgK1BVopt5ePYd8AoOO9xahyLE3S9D3qPB68gs6sRkVpKAUikmjEMg6Vb
TjD1292kZuXh4mRh9DXNeKh3M9uChdYC2PoRxEyFzCTbm1reCB3utlWHmi60
BQQ1NbsKEanlFlBEqpH41CyeWbqTn/adAaBNAz9eua09rer52Roc/QW+f8o2LR
wgpAX0ewkirzGpYhGRmkkBSKQaMAyDzzYd48VleziXk4+bsxOPX9ec+69oiouz
E6SdhJWTYedi2xvc/Wy3iLqNAmdXc4sXEamBFlBETHbsbCbjlu5g3QHb7ayOjQ
J45bZ2NKvjaxsU/L+58PNrkJcBWGzPurp2MviEmlu4iEgNpgAkYhKr1eCjX44wY/
leMnMLcHdx4snoFozs2QRnC7Dve9tTzpMP2d7QsJttOniDTqbWLSJSGygAiZjg
UGlGTy/ZQezhswB0axLEjlHtaBLiDYn7bevgHFhla+xTF657DtoOBqdaMMhZRKQ
aUAASqUlFVoMFaw/x6g/7yMm34uXmzPh+LRka1Rin3HOw4lnYOA+s+eDkCt0f
hivHgruv2aWLiNQqCkAiVWR/wjmeXLKDbcdSAOjVLlTpt7YlPMADti+EVc9Bxml
b48v6QvSLEBxpXsEilrWYApBlJcsrsPLOzweZvWo/uQVWfN1dmHBjKwZ3Ccdy
YjN88RSc2GxrHNwM+r4Eza8zt2gRkVpOAUikEu0+mcaTS7bz28k0AK5uEcqLt7
alnlMafPUQbP/E1tDNF656CqleABc3EysWEXEMCkAilSA338rcnw7w5k8HyLca

+Hu6MnlAa25pF4pl49uw5mXlPWdr3GGobVq7b5i5RYulOBAFlJEKtv1YCk8t2c
G+BFvAib48jGk3t6HOqf/BWzdB0gFbw/qdoP8r0LCLidWKiDgmBSCRCpKdV8
CsVft55+c4rAYEe7sx9aY29K+fgeWbYbB/ha2hdyj0eQ7aD9G0dhERkygAiVSAz
UfO8uSSHRw8kwHA/7Wvz+ToRgRvfh2+egOseeDkYhvjc9VT4OFvcsUilo5NAUi
kHDJz83llxT7eX38Yw4A6vu48f1Nrri/4GRbcAemnbA0jr7XN7gq9zNyCRUQEU
AASKbMNcUk8/cUOjp7NBOC2zg2Z0ikHn5+Gw/FYW6PAJtB3um1dH4vFxGpF
ROSvFlBESik9J5+Xvt/Dx7
8cBaCevwev9q9Pz8NvwEcfAwa4ettWcO7+MLi4m1uwilgUowAkUgprfj/DM0t3
cillC4C7utVnQuhaPL4bCTmptkbtboc+U8CvvnmFiojlBSkAiVyC1Kw8Xli2m8W/H
gcgPMiTed3TuHz7P2HHPlujeu2h38vQ6B8mVioilpdCAUjkllbtTuDZr3aSkJaDxQK
Pd3Ll4bz/4BLzna2BV7BtlcOOd4GTs7nFiojlJVEAEjmP5lxcpvz3N77edhKA1sHOv
Bv5M/V/excKcsDiDFH/hKueBs8Ac4sVEZFSUQASKcF3O+OZ9PUuEtNzcblY/Lt1
HP93+i0sO+JtDZr2hr4zoE5LU+sUEZGyUQAS+Ysz53KY9PUuvt9lW7+nX0gCr
3h/gk/cJluDgEYQPR1a3qBp7SliNZgCkAhgGAZfbzvJlP/+RkpmHiFO55gfvpx2C
V9hSTfAxROu+Bf0GA2unmaXKyli5aQAJA7vVGo2E77ayao9p3GmgKcD13J/wa
c4J6TZGrQZCNdNBf+G5hYqlilVRgFlHJZhGHy++TjTvt3Nuex8rnD5jX/7fUZlZpy
tQVhb6DcDlnqaW6iliFQ4BSBxWHN+PMDMlb/T0HKGt/0W0yN3HWQCnoFwz
UToPELT2kVEaikFlHFl7/7vlG+u3MnjLv9ltNsyXHJzwOlEXe6Fq58BryCzSxQRkUq
kACQO59ONR9jy/Xuscv+EhpZEsAlRV9ie1l63jdnliYhlFVAAEofy45qfilh5hiFuuwE
w/Btiuf55aH2zprWLiDgQBSBxDJlnOfrFBK468AnOTgZ5FjdcrnwCS8/Hwc3L7O
pERKSKKQBJ7WYtgM3vkbdyGo1yU8AC23yvot3IOViCGptdnYilmEQBSGqvw+
vg+6chYSeuwD5rQ75r8DiP3HcvTs5OZlcnlilmUgCS2if1OPwwEX5bats0vHkt/za
ONR3C28OjcFH4ERFxeApAUnvkZcH6OfC/mZCfhYGFJfThxZyBXNYkgvfv7oabi
8KPilgoAEltYBiw91tY8QykHAUgu1437j0ziHXpDWgfHsD8EV3xdNOihiliYqMAJD
Xb6T22cT6H1ti2fetztudEbvyxDifTc2hZ15cPRnbFx11/6ili8id9KkjNlJUCq1+C2HfA
KABnd+jxCKc7PsTg+Ts4mZZJ0xBvPro3igAvN7OrFRGRakYBSGoWawFs/Qhipk
Jmkm1fyxvh+udJdm/A3e/8wuGkTBoEePLxfVGE+rqbW6+liFRLCkBScxz9Bb5/C
uK327ZDWkC/lyDyGs5l5zH83Y3sSzhHHV93PhkVRf0AT3PrFRGRaksBSKq/tJO
wcjLsXGzbdveD3uOh2yhwdiUrt4B73/+VHcdTCfRyZeF9UTQO9ja3ZhERqdYU
gKT6ys+BDXPh59cgLwOwQKe74ZpJ4BMKQE5+Afd/9Cuxh8/i6+7CR/dG0TzM
19y6RUSk2jN9UZQ33niDilglPDw8ilqKljY29rxt8/LymDp1KpGRkXh4eNC+fXuWL
19ermNKNWQYsO97eCPKNtYnLwMadoP7f4L/m2MPP3kFVh75ZCv/25+lp6sz7
9/TlTYN/E0uXkREagJTA9CiRYsYM2YMkydPZsuWLbRv357o6GhOnz5dYvsJE

ybw9ttvM2fOHHbv3s0DDzzALbfcwtatW8t8TKlmzvwOHw+ET++A5EPgUxdue
Qfu/QHqd7Q3s1oNnvx8Oz/sTsDNxYl3h3ehc+MgEwsXEZGaxGlYhmHWyaOiouj
atStz584FwGq1Eh4eziOPPMK4ceOKta9fvz7PPvssDz/8sH3fwlED8fT05OOPPy
7TMUuSlpaGv78/qamp+Pn5lfcy5VJkp8Kal2HjPLDmg5MrdH8YrhwL7kVvaRmG
wTNf7uLT2KO4OFmYd1dn+rQOM6lwERGpLkrz+W3aGKDc3Fw2b97M+PHj7fuc
nJzo06cPGzZsKPE9OTk5eHh4FNnn6enJ2rVry3zMwuPm5OTYt9PS0sp0TVlGV
its/wRWTYGMM7Z9l/WF6BchOLJYc8MweGHZHj
6NPYrFAjNv76DwlylipWbaLbDExEQKCgolCyv64RUWFsapU6dKfE90dDQzZ8
5k//79WK1WVq5cydKlS4mPjy/zMQGmT5+Ov7+//Ss8PLycVyeX5Piv8O618PXD
tvAT3AyGLoE7F5UYfgBmrdrPu2sPATDj1nb8X/v6VVmxiljUEqYPgi6N2bNn07x5
c1q2blmbmxujR49m5MiRODmV7zLGjx9Pamqq/evYsWMVVLGU6Nwp+PJBW/
g5uQXcfOG6afDgBmh+3Xnf9s7PccyO2Q/A5AGtGdxVQVVERMrGtFtglSEhODs
7k5CQUGR/QklCdevWLfE9oaGhfPXVV2RnZ5OUlET9+vUZN24cTZs2LfMxAdz
d3XF314rBlS4/Fza+ZRvrk5tu29dhKFw7GXwvfBvr41+O8OJ3ewEYe/1ljOzZpLKr
FRGRWsy0HiA3Nzc6d+5MTEyMfZ/VaiUmJobu3btf8L0eHh40aNCA/Px8vvjiC
2666aZyH1Mq2f6V8FZ3WDnJFn7qd4L7YuDmNy8afr7cepyJX+8C4MHekTx8d
bOqqFhERGoxUxdCHDNmDMOHD6dLly5069aNWbNmkZGRwciRlwEYNmwY
DRo0YPr06QBs3LiREydO0KFDB06cOMGUKVOwWq089dRTl3xMqWJJcbB8P
OxfYdv2rgN9pkD7lXAJty6X74pn7Oc7MAwY3r0xT0W3wGKxVG7NliJS65kagG
6//XbOnDnDpEmTOHXqFB06dGD58uX2QcxHjx4tMr4nOzubCRMmcPDgQXx8f
Ojfvz8fffQRRAQEBl3xMqSI55+DnV2HDG2DNAycX+MeDcOVT4HFpSwus3neaRz
7dSoHV4LbODZk84HKFHxERqRCmrgNUXWkdoHKwWm3P7Fo5GdL/mHnXrA
9ET4fQyy75MBsPJjFsQSw5+VZuaFuP14d0xNlJ4UdERM6vRqwDJLXQiS3w/d
Nw/l9HjwQ2gb4vwWXRUlqem23HUrj3g1/JybdyTcs6/Pv2Dgo/liJSoRSApPzSz
0DMc7D1Y8AAV2/bCs7dHwaX0s2u2xOfxvAFsaTn5NO9aTBvDu2Em0uNWq1B
RERqAAUgKbuCPlh9B1a/BDl/rJ7d7nbbIGe/0i9QePBMOnfP30hqVh4dGwXw7v
AueLg6V2zNliliKABJWcX9CN+Pg8R9tu167aHfy9DoH2U63LGzmQx9dyOJ6bm
0rufH+yO64e2uP08REakc+oSR0jl7CFY8C/uW2ba9gm0LGXa8C5zK1ltzOi2bu+
ZvJD41m8hQbz68txv+Xq4VWLSliEhRCkByaXLSYe1MWD8XCnLA4gzd7ofe48
AzoMyHPZuRy9B3N3lkKZPwlE8W3vcPQny0KreliFQuBSC5MMOAnUtsKzifO2n
b17Q39J0BdVqW69Bp2XkMW7CR/afTCfNz55P7/kFdf4/y1ywilnlRCkByfvHbbd
Paj26wbQc0sq3n0/KGUk1rL0lmbj73vLeJXSfSCPZ2Y+F9/yA8yKsCihYREbk4BS
ApLiMJfpwGm9/HNq3dC3qNgR6jwdWz3lfPzivg/g838+uRZPw8XPjw3m40q+

NT7uOKilhcKgUg+VNBPvw6H356AbJTbfvaDlTrpoJ/wwo5RV6BldGfbGHtgUS
83Jx5/55uXF7fv0KOLSlicqkUgMTm4BpYPg5O77Zth7WFfjMgomeFnaLAajBm8
XZW7TmNu4sT7w7vQqdGgRV2fBERkUulAOToko/ADxNgzze2bc8guGYCdB5R
5mntJbFaDZ5ZupP/bj+Ji5OFt+7qRI/lkAo7voilSGkoADmq3ExYNwvWzYb8bLA
4Qdf7oPd48Aqq0FMZhsG0ZbtZ9OsxnCww+46OXNMyrELPlSliUhoKQl7GMGD
3V7BiAqQdt+2LuMJ2uyvs8ko55cyVv/PeusMAzBjYjhva1auU84iliFwqBSBHcmq
XbVr7kbW2bf9wuP55aH1Tuae1n89bq+OY8+MBAKbedDmDuoRXynlERERKQw
HlEWSehZ9etM3wMqzg4gG9noAej4Jb5a298+GGw8xYvh
eAp/q2YFj3iEo7l4ilSGkoANVm1gLY/B78+DxkJdv2tb7J1usT0KhST71k83Emff0b
AKOvbsZDvZtV6vlERERKQwGotjq8zna7K2GnbbtOa9s4nyZXVvqpv9sZz1NLtg
MwokcE/7r+sko/p4ilSGkoANU2qcfhh4nw21LbtkcAXP0sdLkHnCv/1/3T3tM8+ul
WrAbc3iWcSTe2xlJJ44tERETKSgGotsjLgvVz4H8zlT8LsECXkXD1BPAOrplSNsQ
l8cDHm8m3GgxoX58Xb22Lk5PCj4ilVD8KQDWdYcDeb2HFM5By1LavUQ/b7a5
67aqsjC1Hk7n3g03k5Fvp06oOMwe3x1nhR0REqikFoJrs9B7bOJ9Da2zbvvXh+
mm253dV4W2n306mMmJBLJm5BfRsFszcOzvh6uxUZecXEREpLQWgmigrBVa
/BLHvgFEAzu7Q81Hb1HY37yot5cDpdlbNjyUtO5/OjQP5z7AueLhW3CM0RERE
KoMCUE1iLYCtH0HMVMhMsu1reaNtWntQkyov59jZTO56dyNJGblcXt+PBSO64
uWmPykREan+9GlVUxz9Bb5/CuJt08sJaQH9XoLla0wp51RqNne++wun0rJpXs
eHj+6Nwt/T1ZRaRERESksBqLpLOwkrJ8POxbZtd3+4erztwaXO5gSO1Kw8hr77
C8fOZtE42luP74siyNvNlFpERETKQgGousrPgQ1z4efXlC8DsECnu+GaSeATamp
pX245TtyZDOr6efDxvVGE+XmYWo+liEhpKQBVN4YBvy+H5eMh+ZBtX8Nu0P
9lqN/R3Nr+EHv4LAB3d29MeFDlPUtMRESksigAVSdnfofl4yAuxrbtU9c2rb3toC
qd1n4hhmEQe8gWgLo1CTK5GhERkbJRAKoOslNhzcuwcR5Y88HZDbo/DFf8C
9x9za6uilOJGSSm5+Lm4kS7hv5mlyMilllmCkBmslph+yewagpknLHtu6wfRL8A
wZGmlnY+hb0/HclDcHfRej8illlzKQCZ5fiv8N2TcHKLbTu4GfR9CZpfZ25dF7Hpjw
AUpdtfliJSgykAVbVzCbYen+2f2LbdfKH309Dtn+BS/aeSb9T4HxERqQUUgKrSp
vm2NX1yz9m2OwyFayeDb5i5dV2i48mZnEjJwtnJQqdGgWaXlyliUmYKQFXJsN
rCT4PO0O9laNjF7lpKZdMf09/b1PfD211/OiliUnPpU6wqdR4J3iHQ6iZwqnlPS9f0
dxERqS0UgKqSswtcfovZVZTZnwEo2ORKREREyqdM3RA//fRTRdch1Vxieg5xZ
zlA6Bqh8T8illKzlSkA9e3bl8jlSJ5//nmOHTtW0TVJNVQ4/b1lXV8CvKr/bDUREZE
LKVMAOnHiBKNHj2bJkiU0bdqU6OhoFi9eTG5ubkXXJ9WEpr+LiEhtUqYAFBlSw
hNPPMG2bdvYuHEjl112GQ899BD169fn0UcfZfv27RVdp5iscPxP1wgFlBERqfnKP
RWpU6dOjB8/ntGjR5Oens6CBQvo3LkzV1xxBb/99ltF1CgmS8vOY8+pNEA9QC

IiUjuUOQDI5eWxZMkS+vfvT+PGjVmxYgVz584lISGBAwcO0LhxYwYNGlSRtYpJ
Nh9OxjAgItiLMD8Ps8sREREptzJNg3/kkUf49NNPMQyDu+++m5dffpk2bdrYX/f
29ubVV1+Ifv36FVaomEfjf0REpLYpUwDavXs3c+bM4dZbb8Xd3b3ENiEhlZouX
0vEHkoCtP6PiljUHmUKQDExMRc/slsLV111VVkOL9VlVm4BO46nAtBNA6BFRKS
WKNMYoOnTp7NgwYJi+xcsWMCMGTPKXZRUH1uPJZNvNajr50F4kKfZ5YiliFSl
MgWgt99+m5YtWxbbf/nllzNv3rxyFyXVx1+f/2WxWEyuRkREpGKUKQCdOnWK
evXqFdsfGhpKfHx8uYuS6kMPQBURkdqoTAEoPDycdevWFdu/bt06zfyqRXLzr
Ww5mgxAlAKQiljUlmUaBD1q1Cgef/xx8vLyuOaaawDbwOinnnqKf/3rXx
VaoJhn54lUsvOsBHq50qyOj9nlililVJgyBaAnn3ySpKQkHnroIfvzvzw8PHj66acZP
358hRYo5tl0+M/HX2j8j4il1CZlCkAWi4UZM2YwceJE9uzZg6enJ82bNz/vmkBS
M2n8j4il1FZlCkCFfHx86Nq1a0XVltVlgdWw9wBFaQFEERGpZcocgH799VcWL1
7M0aNH7bfBCi1durTchYm59p5K41x2Pj7uLrSq52t2OSlilhWqTLPAPvvsM3r06M
GePXv48ssvycvL47fffuPHH3/E39+/omsUExTe/urUOBAX5zl/M1dERKRaKtMn2
4svvsi///1v/vvf/+Lm5sbs2bPZu3cvgwcPPP

RUZGBiNHjgRg2LBhNGjQgOnTpwMwYMAAZs6cSceOHYmKiuLAgQNMnDiRA
QMG2lPQ2LFjGTBgAI0bN+bkyZNMnjwZZ2dnhgwZUpZLdTiFt7+0/o+liNRmZQp
Ad9xxBwCPPvqofZ/FYsEwDCwWCwUFBZd0nNtvv50zZ84wadlkTp06RYcOH
Vi+fLl9YPTRo0eL9PhMmDABi8XChAkTOHHiBKGhoQwYMlAXXnjB3ub48eMM
GTKEpKQkQkND6dWrF7/88guhoaFluVSHs/HQn0+AFxERqa0sxvnuHV3AkSNHL
vh648aNy1xQdZCWloa/vz+pqan4+fmZXU6VSUzPocvzqwDYNuk6ArzcTK5IRE
Tk0pXm87tMPUA1PeBlyTb90fvTsq6vwo+liNRqZQpAH3744QVfHzZsWJmKEX
Nt1PR3ERFxEGUKQl899liR7by8PDlzM3Fzc8PLy0sBqlbS+j8iluloyjQNPjk5uchXe
n
o6+/bto1evXnz66acVXaNUgdSsPPacsj0CpJsGQluISC1XpgBUkubNm/PSSy8V
6x2SmmHLkWQMAyKCvajj52F2OSlilpWqwgIQ2FaJPnnyZEUeUqqIxv+lilgjKdMY
oG+++abltmEYxMfHM3fuXHr27FkhhUnVij2UBEC3JsEmVylillL5yhSAbr755iLbF
ouF0NBQrrnmGl577bWKqEuqUFZuATuOpwJaAVpERBxDmQKQ1Wqt6DrERFuP
JpNvNajr50HDQE+zyxEREal0FToGSGqm2MN/jv+xWCwmVyMillL5yhSABg4cy
lwZM4rtf/nllxk0aFC5i5KqpfV/RETE0ZQpAP3888/079+/2P5+/frx888/l7soqTq5
+Va2HE0GNP5HREQcR5kCUHp6Om5uxZ8V5erqSlpaWrmLkqqz80Qq2XlWgrz
daFbHx+xyREREqkSZAlDbtm1ZtGhRsf2fffYZrVu3LndRUnUKb391aRyo8T8ilulw
yjQLbOLEidx6663ExcVxzTXXABATE8Onn37K559/XqEFSuXadFjjf0RExPGUKQA
NGDCAr776ihdffJElS5bg6elJu3btWLVqFVdddVVF1yiVpMBq2ANQlBZAFBERB1
KmAARwww03cMMNN1RkLVLF9p5K41x2Pj7uLrSq52t2OSlillWmTGOANm3ax
MaNG4vt37hxl7/++mu5i5KqUTj+p3PjQFyctSSUilg4jjJ96j388MMcO3as2P4TJ0
7w8MMPl7soqRpa/0dERBxVmQLQ7t276dSpU7H9HTt2ZPfu3eUuSiqfYRgaAC
0ilg6rTAHl3d2dhlSEYvvj4+NxcSnzsCKpQgcTM0hMz8XNxYl2Df3NLkdERKRKlS
kAXX/99YwfP57U1FT7vpSUFJ555hmuu+66CitOKk/h7a+O4QG4uzibXl2liEjVKl
N3zauvvsqVV15J48aN6dixlwDbtm0jLCyMjz76qEIlIMpRGID0+AsREXFEZQpA
DRo0YMeOHSxcuJDt27fj6enJyJEjGTJkCK6urhVdo1SCPwdAa/0fERFxPGUesO
Pt7U2vXr1o1KgRubm5AHz//fcA/N///V/FVCeV4nhyJidSsnB2stCxUYDZ5YiliFS5
MgWggwcPcsstt7Bz504sFguGYRR5jlRBQUGFFSgVr3D2V5sG/ni7a9C6ilg4njlN
gn7sscdo0qQJp0+fxsvLi127drFmzRq6dOnC6tWrK7hEqWga/yMilo6uTP/5v2H
DBn788UdCQkJwcnLC2dmZXr16MX36dB599dB599FG2bt1a0XVKBdpYOP4nQgFlRE
QcU5l6gAoKCvD1tT07KiQkhJMnTwLQuHFj9u3bV3HVSYU7cy6Hg2cysFigqwK
Qilg4qDL1ALVp04bt27fTpEkToqKiePnll3Fzc+Odd96ldccQXBwcEsWrSoQguUirV

Rz/8SEREpWwCKjo62f9+sWTP27t3L2bNnCQwMLDIbTKofPQBVRESkHOsA/V
1QkD5Qq7vUrDz2nEoDNABaREQcW5kGQUvNtPnlWQwDmoR4U8fPw+xyRE
RETKMA5EBiDyUD0DUi0ORKRERERzKUA5EBiDyUBev6XiliIApCDyMotYMfxVE
ArQluliCgAOYitR5PJtxrU8/egYaCn2eWliiYSgHlQfx1/R8tVSAilo5OAchBFD4BXo
+/EBERUQByCLn5VrYctc0A0/gfERERBSCHsPNEKtl5VoK83WhWx8fsckRERREy
nAOQACh9/0TVCjyoRERERBBSCHoPV/RERERiIlAquUKrAa/HrGN/9Hzv0RERGwU
gGq5vafSOJedj4+7C63q+ZpdjoiISLWgAFTLFY7/6dw4EBdn/bpFRERAAajWi/3
LAogiliJiowBUixmGYQ9AWv9HRETkTwpAtdjBxAySMnJxc3GibUN/s8sRERGpN
hSAarHC3p+O4QG4uzibXl2liEj1oQBUi+n2l4iISMkUgGqxPwdAawFEER
GRv1IAqqWOJ2dyliULFycLnRoHmF2OilhltaIAVEttOmzr/bm8gT9ebi4mVyMiIlK9
KADVUhr/lylicn4KQLXUxsLxP3r+l4iISDGmB6A33niDilgIPDw8ilqKljY29oLtZ82a
RYsWLfD09CQ8PJwnnniC7Ozsch2ztjlzLoeDZzKwWKCrApCliEgxpgagRYsWM
WbMGCZPnsyWLVto37490dHRnD59usT2n3zyCePGjWPy5Mns2bOH+fPns2jRI
p555pkyH7M2Khz/0yLMF38vV5OrERERqX5MDUAzZ85k1KhRjBw5ktatWzNv3
jy8vLxYsGBBie3Xr19Pz549ufPOO4mliOD6669nyJAhRXp4SnvM2kjjf0RERC7Mt
ACUm5vL5s2b6dOnz5/FODnRp08fNmzYUOJ7evTowebNm+2B5+DBg3z33Xf
079+/zMcEyMnJlS0trchXTVYYgLoqAlmliJTltPnRiYmJFBQUEBYWVmR/WFgYe
/fuLfE9d955J4mJifTq1QvDMMjPz+eBBx6w3wlryzEBpk+fznPPPVfOK6oeUrPy2
HPKFuA0AFpERKRkpg+CLo3Vq1fz4osv8uabb7JlyxaWLl3KsmXLmDZtWrmOO
378eFJTU+1fx44dq6CKq97ml2cxDGgS4k0dPw+zyxEREamWTOsBCgkJwdnZ
mYSEhCL7ExlSqFu3bonvmThxlnfffTf33XcfAG3btiUjl4P777+fZ599+fZ599tkzHBHB3d8f
d3b2cV1Q9aPq7iljlxZnWA+Tm5kbnzp2JiYmx77NarcTExNC9e/cS35OZmYmTU9
GSnZ1tTzk3DKNMx6xtNtmf/6UAJClicj6mPiNhzJgxDB8+nC5dutCtWzdmzZpFR
kYGI0eOBGDYsGE0aNCA6dOnAzBgwABmzpxJx44diYqK4sCBA0ycOJEBAwbY
g9DFjlmbZeUWsON4KqAAJCliciGmBqDbb7+dM2fOMGnSJE6dOkWHDh1YvNy
5fRDz0aNHi/T4TJgwAYvFwoQJEzhx4gShoaEMGDCAF1544ZKPZpMvt
Wgnr8HDQM9zS5HRESk2rlYhmGYXUR1k5aWhr+/P6mpqfj5+ZldziX798rfmR2z
n5s61Gf2HR3NLkdERKRKlebzu0bNApMLi9X4HxERkUuiAFRL5OZb2XosGdAK0
CliIhejAFRL7DyRSnaelSBvNyJDfcwuR0REpFpTAKol7I+/iAjEYrGYXl2liEj1pgBUS8
QeSgKgW5NgkysRERGp/hSAaoECq8GvhzX+R0RE5FlpANUCe0+lcS4nHx93F
1rVqznT9kVERMyiAFQLFl7/6dw4EGcnjf8RERG5GAWgWkDr/4iliJSOAlANZxiGP
QBp/I+liMilUQCq4eLOZJCUkYu7ixNtG/qbXY6liEiNoABUw206bOv96dgoAHcX
Z5OrERERqRkUgGo4+/ifCN3+EhERuVQKQDXcnwOgtQCiiljlpVlAqsGOJ2dyliUL

FycLnRoHmF2OilhljaEAVlMV9v60aeCPl5uLydWliljUHApANVjhAGhNfxcRESkdB
aAabKP9CfAKQCliIqWhAFRDnTmXw8EzGVgsCkAiliKlpQBUQxXe/moR5ou/l6vJ
1YiliNQsCkA1lB5/lSliUnYKQDWU1v8REREpOwWgGig1K489p9lA6Nok0ORqRE
REah4FoBpo85GzGAY0CfGmjq+H2eWliljUOApANdBGPf9LRESkXBSAaqA/x/8
oAlmliJSFAlANk5VbwM7jqYACklilSFkpANUwW48mk281qOfvQcNAT7PLERERq
ZEUgGqYjX+5/WWxWEyuRkREpGZSAKphNP5HRESk/BSAapDcfCtbjiYDWgF
aRESkPBSAapCdJ1LJybcS5O1GZKiP2eWliljUWApANUjh7a+uEYEa/yMillIOCkA1
SOyhJEDP/xlRESkvBaAaosBq8Othjf8RERGpCApANcSe+DTO5eTj4+5Cq3p+Z
pcjliJSoykA1RCbDtvG/3SJCMTZSeN/REREykMBqlbQ+j8ililVRwGoBj
AM488ApCfAi4illJsCUA0QdyaDplxc3F2caNvQ3+xyREREajwFoBqgsPenY6MA
3F2cTa5GRESk5lMAqgEKB0Br/R8REZGKoQBUAxT2AGn9HxERkYqhAFTNHU/O
5ERKFi5OFjo2CjC7HBERkVpBAaiaK+z9adPAHy83F5OrERERqR0UgKo53f4SE
RGpeApA1VzsYS2AKCliUtEUgKqxM+dyOHgmA4sFujRWABlREakoCkDVWOH0
9xZhvvh7uZpcjYilSO2hAFSNafyPilhl5VAAqsY2HtlCiCliIpVBAaiaSs3KY++pNAC6
Ngk0uRoREZHaRQGomtp85CyGAU1DvKnj62F2OSliIrWKAlA1VXj7q2uExv+lilhU
NAWgair2kNb/ERERqSwKQNVQZm4+O4+nAgpAlililaFaBKA33niDilglPDw8ilqKlj
Y29rxte/fujcViKfZ1ww032NuMGDGi2Ot9+/atikupENuOppBvNajv70HDQE+zyxE
REal1TH+65qJFixgzZgzz5s0jKiqKWbNmER0dzb59+6hTp06x9kuXLiU3N9e+nZ
SURPv27Rk0aFCRdn379uW9996zb7u7u1feRVSwjX+5/WWxWEyuRkREpPYxv
Qdo5syZjBo1ipEjR9K6dWvmzZuHl5cXCxYsKLF9UFAQdevWtX+tXLkSLy+vYgHl
3d29SLvAwJozlbxw/E9X3f4SERGpFKYGoNzcXDZv3kyfPn3s+5ycnOjTpw8bN
my4pGPMnz+fO+64A29v7yL7V69eTZ06dWjRogUPPvggSUlJ5z1GTk4OaWlpR
b7MkptvZcvRZEArQluliFQWUwNQYmliBQUFhlWFFdkfFhbGqVOnLvr+2NhYdu3
axX333Vdkf9++ffnwww+JiYlhxowZrFmzhn79+lFQUFDicaZPn46/v7/9Kzw8vO
wXVU47T6SQk28lyNuNyFAf0+oQERGpzUwfA1Qe8+fPp23btnTr1q3l/jvuuMP+f
du2bWnXrh2RkZGsXr2aa6+9tthxxo8fz5gxY+zbaWlppoWg2EO23p9uERr/lyliUll
MDUAhlSE4OzuTkJBQZH9CQgJ169a94HszMjL47LPPmDp16kXPU0JCQjh
w4ECJAcjd3b3aDJKOPWS7Vafp7yJSUxUUFJCXl2d2GVlLubq64uzsXCHHMjU
Aubm50blzZ2JiYrj55psBsFqtxMTEMHr06Au+9/PPPycnJ4e77rrrrrrouc5fvw4SUlJ1
KtXryLKrjQFVoNfD//RA6QAJCI1jGEYnDp1ipSUFLNLkVosICCAunXrlvsuiem3wM
aMGcPw4cPp0qUL3bp1Y9asWY9asWWRkZDBy5EgAhg0bRoMGDZg+fXqR982fX
bb76Z4OCiT0pPT0/nueeeY+DAgdDAgdStW5e4uDieeuopmjVrRnR0dJ3dJVdV1nsiU/jXE
4+Pu4utKrnZ3Y5liKlUhh+6tSpg5eXl27jS4UyDlPMzExOnz4NUO5ODdMD0O23

386ZM2eYNGkSp06dokOHDixfvtw+MPro0aM4ORUdq71v3z7Wrl3LDz/8UOx4
zs7O7Nixgw8++ICUlBTq16/P9ddfz7Rp06rNba7zKZz+3iUiEGcn/cMhljVHQUGB
Pfz8/T9MRSqKp6dtceDTp09Tp06dct0OMz0AAYwePfq8t7xWr15dbF+LFi0wD
KPE9p6enqxYsaliy6symw7r+V8iUjMVjvnx8vlyuRKp7Qr/xvLy8soVgExfCFFsDM
Ow9wBp/R8Rqal020sqW0X9jSkAVRNxZzJlysjF3cWJtg0CzC5HRETKKCliglmz
Zl1y+9WrV2OxWEwZPP7+++8TEBBQ5eetDhSAqonC3p+OjQJwc9GvRUSkqvT
u3ZvHH3+8wo63adMm7r///ktu36NHD+Lj4/H396+wGipTaQNedVUtxgDJX9f/0
eBBEZHqxjAMCgoKcHG5+MdmaGhoqY7t5uZ20bXvpOKpq6Ga2HRYz/8SEalqI
0aMYM2aNcyePRuLxYLFYuHw4cP221Lff/89nTt3xt3dnbVr1xlXF8dNN9
1EWFgYPj4+dO3alVWrVhU55t97SCwWC++++y633HlLXl5eNG/enG+++cb++t
9vgRXellqxYgWtWrXCx8eHvn37Eh8fb39Pfn4+jz76KAEBAQQHB/P0008zfPhw
+5p65/P+++/TqFEjvLy8uOWWW4o9J/Ni19e7d2+OHDnCE088Yf95ASQlJTFky
BAaNGiAl5cXbdu25dNPPy3Nr6LKKQBVA8eTMzmRkoWLk4WOjQLMLkdEpElY
hkFmbr4pX+ebKfx3s2fPpnv37owaNYr4+Hji4+OLPApp3LhxvPTSS+zZs4d27dq
Rnp5O//79iYmJYevWrfTt25cBAwZw9OjRC57nueeeY/DgwezZsYsYP+/fszdOhQzp
49e972mZmZvPrqq3z00Uf8/PPPHD16dKWl8d
VXX12who0bN3LvvfcyevRotm3bxtVXX83zzz9fpM3Frm/p0qU0bNiQqVOn2n9
eANnZ2XTu3Jlly5axa9cu7r//fu6++2ZMvWJOZdAusGigc/9OmgT9ebvqViEjtk
JVXQOtJ5ixLsntq9CX9e+rv74+bmxteXl4l3oaaOnUq1113nX07KCil9u3b27enTZv
Gl19+yTfffPHPBJxiMGDGCIUOGAPDiiy/y/y+uuvExsbS9++++ExsbS9
y8X89dFPc+bMYfz48dxyyy0AzJ07l0+++eaaa6+9dsqUKVNmzZ4dHBwcHBwcHBwc

pn99ldEoMmVilg4prFjx+Ls7Ezr1q0JDQ294LiVmTNnEhgYSI8ePRgwYADR0dF0
6tSpCqu1efrppxkyZAjDhg2je/fu+Pj4EB0djYeHx3nf849//IP//Oc/zJ49m/bt2/PDD
z8wYcKElm0u5fqmTp3K4cOHiYyMtK95NGHCBDp16kR0dDS9e/embt26F52Sb
zaLURU3FWuYtLQ0/P39SU1Nxc/Pr1LP1W/2/9gTn8Ybd3bihnb1KvVcliKVJTs7m
0OHDtGkSZMLfghL5bBarbRq1YrBgwczbdo0s8upVBf6WyvN53f1vzlbi6Vm5bH
3VBoAXZuoB0hERC7NkSNH+OGHH7jqqqvlyclh7ty5HDp0iDvvvNPs0moM3Qlz
0eYjZzEMaBriTR1f/ReTilhcGicnJ95//326du1Kz5492blzJ6tWraJVq1Zml1ZjqAfIRB
s1/V1ERMogPDycdevWmV1GjaYelBNp/R8RERFzKACZJDM3n53HUwHoGqEA
JCliUpUUgEyy9WgK+VaD+v4eNAz0NLscERERh6
IAZJK/3v660MJVliliUvEUgEzyZwAKNrkSERERx6MAZlLcfCtbjiYDGgAtliJiBgUg
E+w8kUJOvpVgbzciQ70v/gYREanWlilimDVrln3bYrHw1Vdfnbf94cOHsVgsbNu2r
VznrajjlMWIESOq/eMuLkQByASF6/90jdD4HxGR2ig+Pp5+/fpV6DFLChzh4eHEx
8fTpk2bCj1XZTAzrJVECyGaYJPW/xERqdXq1q1bJedxdnausnPVNuoBqmIFVoNf
D2v8j4hldfDOO+9Qv359rFZrkf033XQT99xzDwBxcXHcdNNNNhlWF4ePjQ9euX
Vm1atUFj/v3W2CxsbF07NgRDw8PunTpwtatW4u0Lygo4N5776VJkyZ4Z4enrSok
ULZs+ebX99ypQpfPDBB3z99ddddYLBYsFgurV68usVdlzZo1dOvWDXd3d+rVq8e
4cePlz8+3v967d28effRRnnrqKYKCgqhbty5Tpky54PUUFBQwZswYAgICCA4O
5qmnnuLvz1Jfvnw5vXr1sre58cYbiYuLs7/epEkTADp27ljFYqF3794AbNq0iAQ
6QkBD8/f256qqr2LJlywXrqQgKQFVsT3wa53Ly8XV3oVW9yn3SvlilqQwDcjPM
+frbh/P5DBo0iKSkJH766Sf7vrNnz7J8+XXKGDh0KQHp6Ov379ycmJoatWW7fSt2
9fBgwYwNGjRy/pHOnp6dx44420bt2azZs3M2XKFMaOHVKFMaOHVukjdVqpWHDhnz++
efs+efs3r2bSZMm2bZ3M2XKFMaOHVukjdVqpWHDhnz++efs3r2bSZMm8cwzz7B48WIA
CwAq1atlj4+nqVLlwJw7tw5hg8fPll19o3rw5/fv359y5c5y5c5f08y0r3QKrYoXT
3ztHBOLspPE/lIKL5WXCi/XNOfczJ8Ht4pNMAgMD6devH5988gnnzJ8+fPx8
CEkJlSrr74agPbt29O+fXv7e6Zm8aXX37JN998+fPnz8+fPx8
PDg8ssv5/jx4zz7JN998w+jRoy96jk8++QSr1cr8+fPx8
ycnAve8nrzzTcJDw9n7ty5WCwWCwMGD8fHxwdPTk5
AJo3b87cuXOxYnhuuuuK/G4s2bNYvz48dx6660AzJs3jxUrVhRpM3DgwCLbC
xYslDQ0lN27d9OmTRtCQ0MBCA4OLnlN11xzTZH3vfPOOwEBAQEBAQEBAQEBAQEBAQ
815reakHqlrp+V8iltXL0KFD+eKLL8jJyQFg4cKFjB49mqzMzMzM

13Lvkcfz1X9+7di0yw6dmzJ+np6Rw/fty+r127dkXeV69ePU6fPl3iMVNTU4mPjy
cqKsq+z8XFhS5duhRpt3//foYMGULTpk3x8/MjliIC4KLXkJCQwKhRo2jevDn+/v7
4+fmRnp5e6msvLfUAVSHDMNh02BaAohSARKS2c/Wy9cSYde5LNGDAAAzD
YNmyZXTt2pX//e9//Pvf/7a/PnbsWFauXMmrr75Ks2bN8PT05LbbbiM3N7fCyv3
ss88YO3Ysr732Gt27d8fX15dXXnmFjRs3Vtg5/srV1bXItsViKTYOqrQGDBhA48a
N+c9//mMfV9WmTZuL/pyGDx9OUlISs2fPpnHjxri7u9O9e/cK/fmWRAGoCsWdy
SApIxd3FyfaNggwuxwRkcplsVzSbSizeXh4cOutt7Jw4UIOHDhAixYt6NSpk/31de
vWMWLECG655RbA1iN0+PDhSz5+q1at+Oijj8jOzrb3Av3yyy9F2qxbt44ePXrw
0EMP2ff9dQAxgJubGwUFBRc91xdffIFhGPZeoHXr1u
Hr60vDhg0vuea/8vf3p169emzcuJErr7wSgPz8fDZv3mz/OSUIJbFv3z7+85//cM
UVVwCwdu3aYvUDxa5h3bp1vPnmm/Tv3x+AY8eOkZiYWKZaS0O3wKpQ4e2v
To0CcXPRj15EpLoYOnQoy5YtY8GCBfbBz4WaN2/O0qVL2bZtG9u3bz+OO+8s
VW/JnXfeicViYdSoUezezvZvvvuOV199tdsdg5fv31V1asWMHvv//OxkT2bRpU5E2
ERER7Nixg3379pGYmEhIeXl6xcz300EMcO3aMRx55hL179/L1118/L1118zefJkxowZY7+
lVxaPPfYYL730El999999RV79+7loYceeiUlxf56YGAgwcHBvPPOOxw4cIAff/zxyRMWP
GFDlGnTp18PT0ZPny5SQkJJCammq/9o8++og9e/bwzceNGjad+
+PQ888AD33nsvEyZMKMVPo7h//etf3H333XHHHHTTTQDMnz+f5ORkOnXqx
N9OmTRueeOlJXnnllSLHcHFx4fXXX+ftt9+ftt9+mfv363mHTTTQDMnz+f5ORkOnXqx
N13382jjz5KnTp1ylXvpbAYf5/lL6SlpeHv709qaip+fhU7VT0330pOfgG+Hq4Xby
wiUkNkZ2dz6NAhmjRpUmSwr0hFu9DfWmk+vzUGqlq5uTjp9peliljJ9EksliiDkcB
SERERByOApCliIg4HAUgERERcTgKQCliUmE0sVgqW0X9jSkAiYhluRU+WiEzM
9PkSqS2K/wb+/vjPEpL0+BFRKTcnJ2dCQgIsD9Q08vLq8gDOUXKyzAMMjMzO
X36NAEBATg7O5freApAliJSlerWrQtw3qeKi1SEgIAA+99aeSgAiYhlhbBYLNSrV4
86deqU+JwqkfJydXUtd89PlQUgERGpUM7OzhX2lSVSWTQIWkRERByOApCliI
g4HAUgERERcTgaA1SCwkWW0tLSTK5EREREllXh5/alLJaoAFSCc+fOARAeH
m5yJSlillJa586dw9/f/4JtLlbWLS/GarVy8uRJfH19K3whr7S0NMLDwzl27Bh
VemwpPf0+qhf9PqoX/T6qF/0+Ls4wDM6dO0f9+vvVxcrrwKB/1AJXAycmJhg0b
Vuo5/Pz89Adcjej3Ub3o91G96PdRvej3cWEX6/kppEHQliIi4nAUgERERMThKAB
VMXd3dyZPnoy7u7vQ3+n1UL/p9VC/6fVQsDYIWERERh6MelBEREXE
4CkAiliLicBSAREREXOEoAlmliljDUQCqQm+88QYRERF4eHgQFRVFbGys2Gys2SU5p

OnTp9O1a1d8fX2pU6cON998M/v27TO7LPnDSy+9hMVi4fHHHze7Fld24sQJ7rrr
LoKDg/H09KRt27b8+uuvZpflkAoKCpg4cSJNmjTB09OTyMhlpk2bdknPu5LzUw
CqIosWLWLMmDFMnjyZLVu20L59e6Kjozl9+rTZpTmcNWvW8PDDD/PLL7+wc
uVK8vLyuP7668nlyDC7NIe3adMm3n77bdq1a2d2KQ4tOTmZnj174urqyvfff8/u3
bt57bXXCAwMNLs0hzRjxgzeeust5s6dy549e5gxYwYvv/wyc+bMMbu0Gk3T4K
tlVFQUXbt2Ze7cuYDteWPh4eE88sgjjBs3zuTqHNuZM2eoU6cOa9as4corrzS7Hl
eVnp5Op06dePPNN3n++efp0KEDs2bNMrsshzRu3DjWrVvH//73P7NLEeDGG2
8kLCyM+fPn2/cNHDgQT09PPv74YxMrq9nUA1QFcnNz2bx5M3369LHvc3Jyok
+fPmzYsMHEygQgNTUVgKCgIJMrcWwPP/wwN9xwQ5H/n4g5vvnmG7p06cK
gQYOoU6cOHTt25D
//+Y/ZZTmsHj16EBMTw++//w7A9u3bWbt2Lf369TO5sppND0OtAomJiRQUFBA
WFlZkf1hYGHv37jWpKgFbT9zjjz9Oz549adOmjdnlOKzPPvuMLVu2sGnTJrNLEeD
gwYO89dZbjBkzhmeeeYZNmzbx6KOP4ubmxvDhw80uz+GMGzeOtLQ0WrZsib
OzMwUFBbzwwgsMHTrU7NJqNAUgcWgPP/wwu3btYu3atWaX4rCOHTvGY48
9xsqVK/Hw8DC7HMH2HwZdunThxRdfBKBjx47s2rWLeFPmKQCZYPHixSxcuJBP
PvmEyy+/nG3btvH4449Tv359+/nn3/7T7KQQGoCoSEhODs7ExCQkR/QkJCdStW9e
kqmT06NF8++23/PzzzzzzzRs2NDschzW5s2bOX36NJ06dbLvKygo4Oeff2Lnk
5OTg7OxsYoWOp169erRu3brIvlatWvHFF1+YVJFje/LJJxk3bhx33HHHHHHAG3btuXIk
SNMnz5dAagcNAaoCri5udG5c2diYmLs+6xWKzExMXTv3t3EyhyTYRiMHj2aL7/
8kh9//JEmTZqqYXZJDu/baa9m5cyfbtm2zf3Xp0oWhQ4eybds9OzZs9jSE
L///juNGzc2qSLHlpmZiZNT0Y9rZ2dnrFarSRXVDuoBqiJjxoxh+PDhdOnShW7duj
Fr1iwyMjlYOXKk2aU5nlcffphPPvmEr7/+GI9fX06dOgWAv78+eiiJfneHx9fYuN
v/L29iY4OFjjskzyxBNP0KNHD1588cXXXXXUUGDx5MbGws77zzzZpTmkAQMG
8MlLL9CoUSMuv/xytmdysyZM7nnnnnvMLq1G0zT4KjR37lxeeeUVTp06RYcOH
Xj99deeeeeeUVTp06RYcOH
Zo0YcyYMYwaNcrsshzSuXPnmDhxll9++SWnT5+mfv36ZZ+mfv36DBkyZ
TWWApCliIg4HI0BEhEREYejACQiliIORwFIREREHI4CkliliDgcBSARERFxOApAlili4
nAUgERERMThKACJiFyC1atXY7FYSEIJMbsUEakACkAiliLicBSAREREXOEoAlIlj
WC1Wpk+fTpNmjTB09OT9u3bs2PJEuDP21PLli2jXbt2eHh48I9//Ll//Ndu3YVOcYX
X3zB5Zdfjru7OxEREbz22mtmtFXs/JyeHpp58mPDwcd3d3mjVrxvz584u02bx5M12
6dMHLy4sePXoNMCkAiUiNMnz6dDz/8kHnz5vxvHbb7/xxBNPPcNdd7FmzRp
mzRp7myeffJLXXXXnuNTZs2ERoayoABA8jLywNswWXw4MHccccccd7Ny5kylTpjBx
4kTef/9/8+bNm3HXXXfZ3Xfffc3bt2/fvn3ZsmXLzJ9/9tlnzZ9/9tlnzZ9/9tlnzZ
4kTef/99+bNm3HXXXfZ3Xfffc3bt2/fvn3ZsmXLzJ9/9tlnzZ9/9tlnzZ9/9tlnzZ

nsgtUkPpYagiUu3l5OQQFBTEqlWr6N69u33/fffdR2ZmJvfffz9XX301n332Gbfffj
sAZ8+epWHDhrz//vsMHjyYoUOHcubMGX744Qf7+5966imWLVvGb7/9xu+//06
LFi1YuXllffr0KVbD6tWrufrqq1m1ahXXXXnstAN999x033HADWVlZeHh4VPJPQU
QqknqARKTaO3DgAJmZmVx33XX4+PjYvz788EPi4uLs7f4ajoKCgmjRogV79uw
BYM+ePfTs2bPlcXv27Mn+/fspKChg27ZtODs7c9VVV12wlnbt2tm/r1evHgCnT58
u9zWKSNVyMbsAEZGLSU9PB2DZsmU0aNCgyGvu7u5FQlBZeXp6Xll7V1dX+/
cWiwWwjU8SkZpFPUAiUu21bt0ad3d3jh49SrNmzYp8hYeH29v98ssv9u+Tk5P5
/fffadWqFQCtWrVi3bp1RY67bt06LrvsMpydnWLrvsMpydnWnbti1Wq7XlmClRqb3UAyQi1Z6
vry9jx47liSeewGq10qtXL1JTU1m3bh1+fn40btwYgKlTpxlcHExYWBjPPvssslSEh3H
zzzQD861//omvXrkybNo3bb7+dDRs2AlliGD58OP
fccw+vv/467du358iRI5w+fZrBgwebdekiUkkUgESkRpg2bRqhoaFMnz6dgwcPE
hAQQKdOnXjmmWfst6BeeuklHnvsMfbv30+Hdh473473//i5ubGwCdOnVi8eLFTJo
0iWnTplGvXj2mTp3KiBEj7OdZZ3jooYdlSkqiUaNGPPPMM2ZcrohUMs
0CE5Ear3CGVnJyMgEBAWaXlyl1gMYAiYiliMNRABlRERGHo1tglili4nDUAyQiliIO
RwFlRERERHl4CkliliDgcBSARERFxOApAlili4nAUgERERMThKACJiliIw1EAEhEREY
ejACQiliIO5/8BjW1a47lCIfYAAAAASUVORK5CYII=\n"
      },
      "metadata": {}
    }
  ]
},
{
  "cell_type": "code",
  "source": [
    "plt.plot(history.history['loss'])\n",
    "plt.plot(history.history['val_loss'])\n",
    "\n",
    "plt.title('model loss')\n",
    "plt.ylabel('loss')\n",
    "plt.xlabel('epoch')\n",
    "\n",
    "plt.legend(['training data', 'validation data'], loc = 'upper right')"
  ],

"metadata": {
"colab": {
"base_uri": "https://localhost:8080/",
"height": 489
},
"id": "ZrgfNlbk9EZK",
"outputId": "8364cf48-ea01-4733-8af0-5196abb191c4"
},
"execution_count": 26,
"outputs": [
{
"output_type": "execute_result",
"data": {
"text/plain": [
"<matplotlib.legend.Legend at 0x7c05c8c5ed20>"
]
},
"metadata": {},
"execution_count": 26
},
{
"output_type": "display_data",
"data": {
"text/plain": [
"<Figure size 640x480 with 1 Axes>"
],
"image/png": "iVBORw0KGgoAAAANSUhEUgAAAjcAAAHHCAYAAABDUn
kqAAAAOnRFWHRTb2Z0d2FyZQBNYXRwbG90bGlIHZlcnNpb24zLjEwLjAsIGh
0dHBzOi8vbWF0cGxvdGxpYi5vcmcvlHJYcgAAAAlwSFlzAAAPYQAAD2EBqD+
naQAAaCtJREFUeJzt3Xd4VNXaxuHfTHoP6ZAEQokQWuhlUVFBFEVFFDtFxXP
siqhgAQUVGx5UUJRPxWNFpXpQEFBAEAHpvZcQ0iEd0ma+PyYZiAmhJdmT5
Lmvay5m9uyZeQeQPK717rVMVqvVioiliEgtYTa6ABEREZHKpHAjlilitYrCjYiliNQq
CjciliJSqyjciliISK2icCMiliK1isKNiliI1CoKNylillKrKNylilhlraJwlylO7+DBg5hMJqZPn3

7er126dCkmk4mlS5dWeN706dMxmUwcPHjwgmoUEcehcCMiliK1isKNilil1CoKN
ylillKrKNylyFm9/PLLmEwmdu/ezT333lOfnx/BwcG89NJLWK1W4uLiuOmmm/D1
9SUsLlyJEyeWeY/k5GTuv/9+QkNDcXd3JzY2li+++KLMeenp6QwdOhQ/Pz/8/f0
ZMmQl6enp5da1c+dObr31VgICAnB3d6dTp07MmzevUr/7hx9+SKtWrXBzc6NB
gwY88sgjZerZs2cPAwcOJCwsDHd3dylilrjjjjvlyMiwn7No0SJ69uyJv78/3t7eNG/
enOeff75SaxURG2ejCxCRmuP2228nJiaGN954g/nz5/Pqq68SEBDAxx9/
cWbb77J119/zciRl+ncuTOXX345ACdOnKBXr17s99lMaNG/PDDz8wdOh
Q0tPTeeKJJwCwWq3cdNNNrFixgn//+9/ExMQwe/ZshgwZUqaWbdu20aNHD8
LDwxk1ahReXl58//333HzzzcycOZMBAwZc9Pd9+eWXeeWVV+jduzcPPfQQ
t4qOPPmLt2rWsXLkSFxcX8vPz6eTz22GOEhYURHx/P//73vPz6du3L3l5eTz22GOEh
Pbtm3ccMMNtG3blnHjxuHm5sbevXtZuXLlRdcoluWwioicxdixY62A9cEHH7QfK
ywstEZERFhNJpP1jTfesB8/fvy41cPDwzpkyBD7sUmTJlkB61dffWU/lp+fb+3WrZ
vV29vbmpmZabVardY5c+ZYAetbb71V6nMuu+wyK2D9/PPP7cevvvpqa5s2ba
wnT560H7NYLNbu3btbo6Oj7cd+//13K2D9/fffK/yOn3/+uRWwHjhwwGq1Wq3
JyclWV1dX6zXXXGMtKiqynzd58mQrYP3ss8+sVqvUmHDBitg/eGHH8743v/5z
3+sgDUlJaXCGkSkcmhaSkTO2QMPPGC/7+TkRKdOnbBardx///3/4/7+/jRv
+/fbj/3888+EhYVx55LFz+/vz8Ac+fOxWKxXFRdlnJ2
Cjcics4aNmxY6rGfnx/u7u4EBQWVOX78+HH740OHDhEdHV0qJADExMTYny/5
tX79+nh7e5c6r3nz5qUe7927F6vVyqvVyksvvURURwcHCp29ixYwFbj8/FKKnpn5/t6upK
kyZN7M83btyYESNG8H//938/PDx8fHx8fHx8fHx8fHx8fHx8fHx8fHx8fHx8fHx8fHx
cdd/D9998r6IhUEfXciMg5c3JyOqdjYOufqSoloWDkyJH07du33HOaNWtWZZ//
TxMnTmTo0KHMnTuXXX3/9lccff5wJEybMnTuXXX3/9lccff5wJEybw9122mrr76at59
M7OLnXerl27Sj1u0qQJYJva6t27d7d7k3Hx
+fi665vM/Oz8/nwlED9udLtGnThhdffJHly5fzx9/EB8fz9SpU+3Pm81mrr76at59
9122b9/Oa6+9xm+//cbvvvvvUXWKSFkKNyJS5fr160diYilzZsywHyssLOSDDz7A
29ubK664wn5eYWEhH330ER9fXx8fHx8fHx8fHx8fHx8fHx8fHx8fHx8fHx8fHx
XXXPv3r1xdXXl/fffLzUK9emnn5KRkcHx8fHx8fHx8fHx8fHx8fHx8fHx8fHx8fHx
XXXl/fffLzUK9emnn5KRkcH1118PQGZmJoMHDx4zZ8ccMHTqUdevERUVxY8//jhz
mSfZSlf//+9OjRg1GjRg1GjRnHw4EFatmzZ2NjY2NjY2NjY2NjY2NjY2NjY2NjY
2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY2NjY
mpCUlMSqVas4cuQlmzZtuqiag4ODGT16NK+88grXXnstN954l7t27eLDDz+kc+fO

3HPPPQD89ttvPProo9x2221ccsklFBYW8uWXX+Lk5MTAgQMBGDduHMuXL+f
666+nUaNGJCcn8+GHHxlREUHPnj0vqk4RKUvhRkSqnleHB0uXLmXUqFF88cU
XZGZm0rx5cz7//HOGDh1qP89sNjNv3jyefPJJvvrqK0wmEzfeeCMTJ06kffv2pd6
zZcuW/P3337zyyitMnz6dtLQ0QkJCaN++PWPGjKmUul9++WWCg4OZPHkyTz
31FAEBATz44lO8/vrruLi4ABAbG0vfvn356aefiI+Px9PTk9jYWH755RcuvfRSAG6
88UYOHjzlZ599RmpqKkFBQVxxxRW88sor9qutRKTymKxV2fUnlliUs3UcyMiliK1i
sKNiliI1CoKNylillKrKNylilhlraJwlylilrWKwo2liljUKnVunRuLxcLRo0fx8fHBZDlZXY6li
licA6vVSlZWFg0aNCizCe8/1blwc/ToUSljl40uQ0RERC5AXFwcERERFZ5T58JN
yTLvcXFx+Pr6GlyNiliInlvMzEwilyPPaVNcw8PNlClTePvtt0lMTCQ2NpYPPviALl26
nPH89PR0XnjhBWbNmsWxY8do1KgRkyZNol+/fuf0eSVTUb6+vgo3liliNcy5tJQ
YGm5mzJjBiBEjmDp1Kl27dmXSpEn07duXXbt2ERlSUub8/Px8+vTpQ0hlCD/++
CPh4eEcOnQlf3//6i9eREREHJKhe0t17dqVzp07M3nyZMDW7BsZGcljjz3GqFGjy
pw/depU3n77bXbu3GnftO58ZWZm4ufnR0ZGhZuREREaojz+flt2KXg+fn5rFu3
jt69e58qxmymd+/erFq1qtzXzJs3j27duvHll48QGhpK69atef311ykqKjrj5+Tl5ZGZ
mVnqJiliIrWXYdNSqampFBUVERoaWup4aGgoO3fuLPc1+/fv57fffffffuPuu+/m559
/Zu/evTz88MMPPP

ABpKWlIXr+bN+vV69eHDp0iKeeesr++wWQlpbGnXfeSXh4OJ6enrRp04Zvv/32
fP4oqp3CTSXp0KgeABsOp7MvJdvgakREqpbVaiU3v9CQ27leB/Pee+/RrVs3hg
8fTkJCAgkJCaVWqB81ahRvvPEGO3bsoG3btmRnZ9OvXz+WLFnChg0buPbaa
+nfvz+HDx+u8HNeeeUVBg0axObNm+nXrx933303x44dO+P5ubm5vPPOO3z5
5ZcsX76cw4cPM3LkSPvzb775Jl9//TWff/45K1euJDMzkzlz5lRYw+rVq7n//vt59
NFH2bhxl1deeSWvvvpqXPO9v1mzZpFREQE48aNs/9+ga3Jt2PHjsyfP5+tW7f
y4lMPcu+997JmzZoKazJSjdp+wZGF+LhzeXQQv+9KYdb6lzzTt4XRJYmlVJkTB
UW0HLPQkM/ePq4vnq5n//Hl5+eHq6srnp6e5a58P27cOPr06WN/HBAQQGxsrP
3x+PHjmT17NvPmzTvj4rJgGyG68847AXj99dd5//33WbNmDdd5//33WbNmDddee2255xcUFDB
16lSaNm0KwKOPPsq4cePsz3/wwQeMHj2aAQMGADB58mR+/vnnnnC/re++9x7X
XXsuzzz4LwCWXXMKff/7JggUL7OfExsZW+P0CAgJwcnLCx8en1O9XeHh4qfD
12GOPsXDhQr7//nu6dOlSYV1G0chNJRrYMQKA2evjsVjq1BX2lil1TqdOnUo9zs7O
ZuTlkcTExODv74+3tzc7duw468hN27Zt7fe9vLzw9fUlOTn5jOd7enragw3YthYqO
T8jl4OkpKRSoaFkL8WK7Nixg65du5Y61q1bt0r5fkVFRYwfP542bdoQEBCAt7c3
CxcuPOvrjKSRm0rUOyYUX3dnjmac5K/9aXRvFmR0SSliVcLDxYnt4/oa9tmVwc
vLq9TjkSNHsmjRIt555x2aNWuGh4cHt95661I3Qf/nNgEmk6nCzUXLO786lpy70
O/39ttv89577zFp0iTatGmDl5cXTz75ZGRERERER

mbvhqNHliljleYqKimLSpEn2xyaTiTlz5pzx/lMHD2lymdi4ceNFfW5lvc+FGDp0qM
NvsVARhZtqMLB49EZXTYml1HwJCQlcd911lfqe5YWJyMhlEhlSaN26daV+VlUw
MoiVR+GmGvRv2wBXJzM7E7PYdjTD6HJEROQihlWF4ebmVuWf4+TkRFhYGM
7OWm/3fCncVAM/Txd6twwBYOY6bccgllldPvnkExo0alDFYil1/KabbuK+++4DY
N++fdx0002Ehobi7e1N586dWbx4cYXv+89pqTVr1tC+fXvc3d3p1KkTGzZsKHV
+UVER999/P40bN8bDw4PmzZvz3nvv2Z9/+eWX+eKLL5g7dy4mkwmTycTSp
UvLHQ1ZtmwZXbp0wc3Njfr16zNq1CgKCwvtz/fq1YvHH3+cZ599loCAAMLCwn
j55Zcr/D5FRUWMGDECf39/AgMDefbZZ/nnntoLFiygZ8+e9nNuuOEG9u3bZ3+
+cePGALRv3x6TyUSvXr0AWLt2LX369CEoKAg/Pz+uuOlK1q9fX2E9lUHhppqU
TE3N3RhPQZHlLGeLiDg4qxXyc4y5/eMH75ncdtttpKWl8fvvv9uPHTt2jAULFnD
33XcDkJ2dTb9+/ViyZAkbNmzg2muvpX///vb4cOZJBgwZx7bXXk
pCQQEJCAt27dy/zWfHx8fTr14/OnTuzadMmMmPvroIz799FNeffXVUud98cUXeHl5
sXr1at566y3y9YsWlFx44dY/bs2aXOycnJYcSlE
fz9998sWbIEs9nMgAED7MFxzZo1ACxevJiEhARmzZoFQFZWFkOGDGHFihX89
ddfREdH069fP7Kyss7p9/dCaayrmlx+STBB3q6kZuezbFcKvVuGGl2SiMiFK8iF1xs
Y89nPHwVXr7OeVq9ePa677jq++eYbg7agB+/PFHgoKCuPLKKwGljY0lNjbW/p
rx48cze/Zs5s2bx6OPPPnrWz/jmm2+wWCx8+umnnn

/Hx8SEiluKcaz6dn58f9evXZ/Xq1Vx++eUAFBYWsm7dOvvvU1paGrt27WLatGlc
dtllAKxYsaJM/UCZ77By5Uo+/PBD+vXrB0BcXBypqakXVOv50LRUNRuo7RhER
KrV3Xffzfz58/nss8/sjcQloqOjmTVrFhs3bmTTpk3cdddd5zXKcdddd2EymRg+fD
jbt2/n559/5p133inzGX///TcLFy5k9+7dvPTSS6xdu7bUOVFRUWzevJldu3aRmp
pKQUFBmc96+OGHiYuL47HHHmPnzp3MnTuXsWPHMmLECPs024V44okneOO
NN5gzZw47d+7k4YcfJj093f58vXr1CAwM5JNPPmHv3r389ttvjBgxotR7hlSE4O
HhwYIFC0hKSiljl8P+3b/88kt27NjB6tWrufvuu/Hw8LjgWs+Vwk01u6ldOE5mExvj
0tmXkm10OSlitd5VV11FQEAAu3bt4q677ir13Lvvvku9evXo9r07/fv3p2/fvqVGds
7G29ubn376iS1bttC+fXteeOEF3nzzzVLn/Otf/+KWW27h9ttvbtvp2vXrqSlpZUaxQE
YPnw4zZs3p1OnTgQHB7Ny5coynxUeHs7PP//MmjVrl2N5d///jf3338/L7744nn8
bpT19NNPc++99zJkyBD7tFnJSBaA2Wzmu+++Y926dbRu3ZqqnnqKt99+u9R7
ODs78/777/Pxxx/ToEEDbrrpJgA+/fRTjh8/TjoOHbj33nt5/PHHHHHCQkJuah6z4XJ+
s+L2Wu5zMxM/Pz8yMjlwNfX15Aa7pu+lt92JvNwr6Y8e20LQ2oQETkfJ0+e5M
CBAzRu3LhU86xlZaro79n5/PzWyI0BSta8mb0hniJLncqWlilVU7hxgBXx4Tg6+5
MQsZJVu1LM7ocERGRWkXhxgDuLk70j7UtfqU1b0RERCqXwo1BBna0TU0t2Jpl
dl7hWc4WERGRc6VwY5D2kf40CfLiREERv2xJMLocEZFzUseuQZFqVll/vxRuD
Glymbilg7ZjEJGawcnJCaDKV5aVuq3k71fJ37cLpRWKDTSgQwQTF+3mr/3HiDu
WS2TAuS8pLiJSnZydnfH09CQlJQUXF5eLWjROpDwWi4WUlBQ8PT1xdr64eKJ
wY6Bwfw+6NQnkz31pzN4Qz+NXRxtdkohluUwmE/Xr1+fAgQMcOnTl6HKkljKbz
TRs2LDU5qAXQuHGYAM7RPDnvjRmrT/CY1c1u+g/UBGRquLq6kp0dLSmpqTK
uLq6VsqooMKNwa5tHcZLc7dyMC2XdYeO0ykqwOiSRETOyGw2a4VicXiaNDW
Yl5sz17WuD8DM9fEGVyMillLzKdw4glEdbVdN/W/zUU4WVLzlvYiliFRM4cYBX
No4kHB/D7JOFrJoe5LR5YiliNRoCjcOwGw2MaC91rwRERGpDAo3DqJkQb/lu1
NIzjxpcDUiliI1l8KNg2gS7E2Hhv5YrDBnoxqLRURELpTCjQMp2Uxz5rp47d8iliJyg
RRuHMgNbRvg6mxmV1IW245mGl2OilhljaRw40D8PFzo0zlUUGOxiljlhVK4cTC3
drBNTc3beJSCIovB1YiliNQ8CjcO5rLoIlK83UjLyW
fprhSjyxEREalxFG4cjLOTmZvbNQBg5jpNTYmliJwvhRsHVHLV1JKdSRzP0e67lili
50PhxgHF1PelZX1fCoqs/LT5qNHliIil1CgKNw7KvuaNdgoXERE5Lwo3Duqmdg1
wNpvYFJfO3uRso8sRERGpMRRuHFSQtxu9mgcDWvNGRETkfDhEuJkyZQpRU
VG4u7vTtWtX1qxZc8Zzp0+fjslkKnVzd3evxmqrz8DiNW9mr4+nyKLtGERERM6
F4eFmxowZjBgxgrFjx7J+/XpiY2Pp27dvb7vcvycnJZ3yNr68vCQkJ9tuhQ4eqseLqc1V
MCH4eLiRmnuTPfalGlyMillljGB5u3n33XYYPH86wYcNo2bllU6dOxdPTk88+++
MrzGZTlSFhdlvoaGh1Vhx9XFzdqJ/bH1Aa96lilicK0PDTX5+PuvWraN37972Y2az

md69e7Nq1aozvi47O5tGjRoRGRnJTTfdxLZt26qjXEOUTE0t2JZl1skCg6sRERFx
fIaGm9TUVlqKisqMvlSGhpKYmFjua5o3b85nn33G3Llz+eqrr7BYLHTv3p0jR8of
2cjLyyMzM7PUrSZpF+lPk2AvThZY+GVL+b8nlilicorh01Lnq1u3bgwePJh27dpxxR
VXMGvWLlKDg/n444/LPX/ChAn4+fnZb5GRkdVc8cUxmUz20RtdNSUiInJ2hoa
boKAgnJycSEpKKnU8KSmJsLCwc3oPFxcX2rdvz969e8t9fvTo0WRkZNhvcXFx
F113dbulQzgmE6w+cly4Y7lGlyMiluLQDA03rq6udOzYkSVLltiPWSwWlixZQrdu3
c7pPYqKitiyZQv169cv93k3Nzd8fX1L3Wqa+n4e9GgaBMAsrVgsliJSlcOnpUaM
GMG0adP44osv2LFjBw899BA5OTkMGzYMgMGDBzN69Gj7+ePGjePXX39l//7
9rF+/nnvuuYdDhw7xwAMPGPUVqsUtHclBmLXhCFar1rwRERE5E2ejC7j99ttJS
UlhzJgxJCYm0q5dOxYsWGBvMj58+DBm86kMdvz4cYYPH05iYiL16tWjY8eO/P
nnn7Rs2dKor1Atrm0dxktztnloLZe/Dx2nc1SA0SWJilg4JJO1jg0DZGZm4ufnR0Z
GRo2bohr5wyZ+XHeEOzpH8sbAtkaXlyliUm3O5+e34dNScu5KrpqavzmBkwVF
BlcjliLimBRuapCujQMl9/cgK6+QX7cnnf0FlilidZDCTQ1iNpsYWNxYrO0YREREyq
dwU8PcUjw19ceeFJlyTxpcjYiliONRuKlhooK86NioHhYrzNmgNW9ERET+SeGmB
jp9O4Y6drGbiljlWSnc1EDXt62Pq7OZ3UnZbI2vWRuBioilVDWFmxrlz8OFa1raFjn
UZpoiliKlKdzUUAM72qam5m06Sn6hxeBqREREHlfCTQ11WbMggn3cOJaTz9Jd
yUaXlyli4jAUbmooZyczA9oXr3mjqSkRERE7hZsarOSqqd92JnM8J9/gakRERBy
Dwk0N1jzMh1YNfCkosjJv01GjyxEREXElCjc13Olr3oililJCTY13U7sGOJtNbD6SwZ
6kLKPLERERMZzCTQ0X6O1Gr+YhAMxcr+0YREREFG5qgVs72q6amr3hCEUW
bccgliJ1m8JNLXBlixD8PV1lysxj5d5Uo8sREExlMJNLeDm7MSNsQ0ANRaLilgo
3NQStxRfNbVwWyJZJwsMrkZERMQ4Cje1RGyEH02DvThZYOHnLQlGlyMilmlY
hZtawmQy2TfTnLlOV02JiEjdpXBTiwxoH47JBGsOHuNwWq7R5YiliBhC4aYWqe
/nQc9mQQDM2qDGYhERqZsUbmqZku0YZq2Px2rVmjcilIL3KNzUMn1bheHl6s
ThY7msPXjc6HJERESqncJNLePh6kS/NvUBmLlOU1MilIL3KNzUQiVXTc3fksCJ/
CKDqxEREaleCje1UJeoACLqeZCdV8iv2xONLkdERKRaKdzUQmazyb5i8Y+amh
lRkTpG4aaWGtjBtlP4yr2pJGacNLgaERGR6qNwU0
s1CvSic1Q9LFaYs1ErFoulSN2hcFOLlax5M3PdEa15lylidYbCTS3Wr2193JzN7EnO
Zkt8htHlililVAuFm1rM192Fa1qFAVrzRkRE6g6Fm1qupLF43qaj5BdaDK5GRESk6in
c1HKXRQcT4uPG8dwCftuZbHQ5liliVU7hppZzMpsY0N42ejNrvaamRESk9lO4q
UxFBXDC8TarLNmO4fddyRzLyTe4GhERkaqlcFNZDq2CDy+F+U8bXUkZl4T60
Cbcj4liK/O05o2liNRyCjeVxdUL0vbB1plw5G+jqymjpLF45nqFGxERqd0UbipL/b
bQ7i7b/YUvglMtmtc/tgHOZhNb4jPYnZRldDkiliJVRuGmMl31ljh7QNxfsOMno6s
pJdDbjStbhABa80ZERGo3hZvK5NsAuj9mu794LBQ6VvNuyXYMszfEU1ikNW9

ERKR2UripbD0eB68QOLYf/v7U6GpKuapFCPU8XUjOymPlvjSjyxEREakSCjeVzc
0HrnrBdn/Zmw51abirs5kbYxsAmpoSEZHaS+GmKrS7B4JjbMFm+TtGV1NKyZo
3C7clknmywOBqREREKp/CTVVwcoZrXrXdX/MJHDtgbD2naRPuR3SlN3mFFn7
enGB0OSlilpVO4aaqNLsamlwJRfmw5BWjq7EzmUz20ZuZ2o5BRERqlYWbqml
yFY/emGDbblhbY3RFdje3C8dsgrUHj3MoLcfockRERCqVwk1VCmsN7e+23Xeg
hf3C/Nzp0Swl0lrFliJS+yjcVLUrXwQXTziyBrbPNboau1uLp6ZmrT+CxelYoUtERK
QyKNxUNd/60P1x2/3FY6Ewz9h6il3TMgxvN2eOHD/B2oPHjC5HRESk0ijcVlfuj4
F3KBw/CGv/z+hqAPBwdeL6NvUBNRaLiEjtonBTHdy8bftOASx7C3ldY6Sk5Kqp
n7ckciK/yOBqREREKofCTXVpdzeEtlKT6Q6zsF/nqHpEBniQnVflwm2JRpcjliJSKR
RuqovZCa4Zb7u/5hPb3lMGM5lM3NJea96liEjtonBTnZpdDU2vBksBLH7Z6GqA
UzuFr9ibSkLGCYOrERERuXgKN9XtmlfBZLZdFn54tdHV0DDQky5RAVitMGfDU
aPLERERuWgKN9UttCW0v8d2/1fHWNhvYMdwwDY1ZXWAekRERC6Gwo0Rr
nwBXLzgyFrb1gwG69emPu4uZvYmZ7P5SlbR5YiliFwUhwg3U6ZMlSoqCnd3d
7p27cqaNee2D9N3332HyWTi5ptvrtoCK5tPGPR4wnZ/8cuGL+zn4+5C31ZhgB
qLRUSk5jM83MyYMYMRI0YwduxY1q9fT2xsLH379iU5ObnC1x08eJCRI0dy2W
WXVVOllaz7o+BTH9lP2a6eMlhJY/G8TUfJK9SaNyliUnMZHm7effddhg8fzrBhw2
jZsiVTp07F09OTzz777lyvKSoq4u677+aVV16hSZMm1VhtJXL1OrWw3/K3DV/Yr
0ezlEJ93UjPLeD3nRUHSxEREUdmaLjJz89n3bp19O7d237MbDbTu3dvVq1adc
bXjRs3jpCQEO6///6zfkZeXh6ZmZmlbg4j9k4lbQ0nM2wrFxvlyWzi5va2xulf12mn
cBERqbkMDTepqakUFRURGhpa6nhoaCiJieWvmLtixQo+/fRTpk2bdk6fMWHCB
Pz8/Oy3yMjli6670pidbJeGA6ydBmn7DC3n1uKpqaW7kknLdowNPkVERM6X4d
NS5yMrK4t7772XadOmERQUdE6vGT16NBkZGfZbXFxcFVd5nppeCc36gKXQt
mu4gaJDfWgb4Uehxcq8TVrzRkREaiZnlz88KCgIJycnkpKSSh1PSkoiLCyszPn79
u3j4MGD9O/f337MYrEA4OzszK5du2jatGmp17i5ueHm5lYF1Veia8bDviWw4yc4
tAoadTOslIEdlth8JlMZa+MY0i0Ks9lkWC0iliIXwtCRG1dXVzp27MiSJUvsxywWC
0uWLKFbt7l/4Fu0aMGWLVvYuHGj/XbjjTdy5ZVXsnHjRseacjofIT
HQYbDt/q8vQHFgM8KNsQ3wdHViZ2lWn608YFgdliliF8rwaakRI0Ywbdo0vvjiC
3bs2MFDDz1ETk4Ow4YNA2Dw4MGMHj0aAHd3d1q3bl3q5u/vj4+PD61bt8bV1
dXlr3Jxej0Prt4Qvw62zTKsjHperrx4fUsA3lq4i91JWYbVliliciEMDze3334777zzD
mPGjKFdu3Zs3LiRBQsW2JuMDx8+TEJCgsFVVgOfUOjxpO3+4leg4KRhpdzZJZ
lrmweTX2jhqRkbyS80biRJRETkfJmsdWwzoczMTPz8/MjlyMDX19fockrLz4UPO
kLWUegz7tQqxgZlzjzJNZOWk55bwGNXNePpa5obVouliMj5/Pw2fORGTuPqCV
e/ZLu/fCLkpBlWSoivO6/d3AaAKb/vZf3h44bVlilicj4UbhxN2zsgrA3kZcCyNw0t

5fq29bmpXQMsVnj6+02cyNe2DCIi4vgUbhyN2QzXvGa7//enkLrX0HLG3diaMF9
3DqTmMOGXHYbWlilici4UbhxRkysguq9DLOzn5+nC27e1BeC/qw6xfHeKofWlili
cjcKNo7pmPJicYOf/4OBKQ0u5LDqYld0aAfDsj5vJyC0wtB4REZGKKNw4quDm
0HGI7b7BC/sBjLouhiZBXiRmnmTMvK2G1ililIlRhRtH1mu0bWG/oxtg60xDS/Fwd
WLioFiczCbmbjzK/zZr7ykREXFMCjeOzDsEej5lu7/E2lX9ANo3rMcjvWx7d704Zy
vJmcbWlylilUh6FG0d36cPgGw4ZcbD6I6Or4dGromkd7kt6bgHPztxMHVsDUkR
EagCFG0fn6gIXj7Hd/+NdyEk1thxnM/8Z1A5XZzNLd6Xw7Zo4Q+sRERH5J4Wb
mqDNIKgfC3mZsPQNo6shOtSHZ/vatmN4df52DqXIGFyRiljIKQo3NYHZDNe8ar
v/92eQusfYeoD7ejSma+MAcvOLePr7TRRZND0lliKOQeGmpmh8OVxyHViLYNE
Yo6vBbDYxcVAs3m7O/H3oOJ8s3290SSIiloDCTc3SZ5xtYb9dP8OBP4yuhoh6n
ozt3xKAdxftYkdCpsEViYilKNzULMGXQKdhtvsOsLAfwK0dI+jTMpSClitPzdhlXqE
21xQREWMp3NQ0V4wCVx9I2ARbfjC6GkwmExNuaUOglys7E7P4zyLj+4FERKR
uU7ipabyD4bIRtvtLxkHBCWPrAYK83ZhwSxsAPl6+j7UHjxlckYil1GUKNzXRpQ+B
XyRkHoG/PjS6GgCuaRXGrR0jsFphxPcbyc4rNLokERGpoxRuaiIXj9MW9vsPZKc
YW0+xMf1bEu7vQdyxE7w2f4fR5YilSB11QeHmiy++YP78+fbHzz77LP7+/nTv3p1
Dhw5VWnFSgda3QoP2kJ8FSycYXQ0Avu4uvHNbLADfrjnMbzuTDK5IRETqogs
KN6+//joeHh4ArFq1iilTpvDWW28RFBTEU089VakFyhmcvrDfuumQssvQckp0ax
rI/T0bA/Dsj1s4lpNvcEUiIlLXXFC4iYuLo1mzZgDMmTOHgQMH8uCDDzJhwgT+
+MP49VfqjKie0Px6h1nYr8QzfZvTLMSb1Ow8XpyzRZtriohltbqgcOPt7U1aWhoAv
/76K3369AHA3d2dEyeMv3qnTunzCpidYfcC2L/M6GoAcHdx4j+D2uFsNvHzlkT
mbjxqdEkiIlKHXFC46dOnDw888AAPPPAAu3fvpl+/fgBs27aNqKioyqxPziYoGjrd
Z7v/64sOsbAfQJslPx6/OhqAl+ZuJSFDoVdERKrHBYWbKVOm0K1bN1JSUpg5c
yaBgYEArFu3jjvvvLNSC5RzcMVz4OYLiZth8wyjq7F7uFdTYiP9yTpZyDM/bMaiz
TVFRKQamKx1rCEiMzMTPz8/Mjly8PX1NbqcyrNiEiweC77h8Ojf4OppdEUA7EvJ
5vr3/+BkgYVXbmzFkO5RRpcklil10Pn8/L6gkZsFCxawYsUK++MpU6bQrl077rrrL
o4fP34hbykXq+u/wa8hZMbDX1OMrsauab
A3o6+LAWDCLzvYI5JtcEUiIlLbXVC4eeaZZ8jMtO0AvWXLFp5++mn69evHgQ
MHGDFiRKUWKOfIxR16j7XdXzEJspMNLed0917aiMuigzhZYGHE95soLHKMviA
REamdLijcHDhwgJYtWwlwc+ZMbrjhBI5//XWmTJnCL7/8UqkFynlodQs06AD52
fD760ZXY2c2m3jr1rb4uDuzKS6dD5fuM7okERGpxS4o3Li6upKbmwvA4sWLue
aaawAICAiwj+iIAcxm6Pua7f76LyB5p7H1nKa+nwfjb2oNwPtL9rDISIbBFYmISG11
QeGmZ8+ejBgxgvHjx7NmzRquv/56AHbv3k1EREIFijnqVF3aHEDWC2w6CWjq
ynlpnYNuL5NfQotVp76fiMnC4qMLklERGqhCwo3kydPxtnZmR9//JGPPvqIJGPPvqI8PB

wAH755ReuvfbaSi1QLkDv4oX99vwK+343uho7k8nE+JtbE+zjxt7kbN5e6BhbRo
ilSO2iS8Frq1+eg9VTlbQN/GsZmJ2Mrsjut51J3Df9bwC+Gd6V7k2DDK5IREQcX
ZVfCg5QVFTEzJkzefXVV3n11VeZPXs2RUWaZnAYlz8Lbn6QtAU2fWd0NaVc1S
KUO7s0BOCZHzaTdbLA4lpERKQ2uaBws3fvXmJiYhg8eDCzZs1i1qxZ3HPPbR
q1Yp9+3QljEPwCoTLR9ru/zYe8nONrecfXrw+hoYBnsSnn+CVn7YbXY6liNQiFxR
uHn/8cZo2bUpcXBzr169n/fr1HD58mMaNG/P4449Xdo1yobo8CP4NlSsBVk02u
ppSvNycmTgoFpMJflx3hlXbEo0uSUREaokLCjfLli3jrbfeliAgwH4sMDCQN954g
2XLHGNnaqF4Yb+XbfdXTlKsJCOrKaNzVAD/urwpAM/P2kJqdp7BFYmlSG1wQe
HGzc2NrKysMsezs7NxdXW96KKkErW6Bcl7QUEO/P6a0dWU8VSfaFqE+ZCWk
8/oWVuoY/3tliJSBS4o3Nxwww08+OCDrF69GqvVitVq5a+//uLf//43N954Y2XX
KBfDZDq1sN+GLyHJsfpb3JydeHdQO1ycTCzansSP644YXZKliNRwFxRu3n//fZo
2bUq3bt1wd3fH3d2d7t2706xZMyZNmlTJJcpFa3gpxNzokAv7AbRs4MulPs0Be
OWn7cQdc6zmZxERqVkuap2bvXv3smPHDgBiYmJo1qxZpRVWVerMOjf/lLYPp
nQFSwHcMwuaXW10RaUUWawM+ngV6w4dp2vjAL4dfilms8noskRExEGcz8/v
cw4357Pb97vvvnvO51a3OhtuABaMhr8+hNDW8K/lDrWwH8ChtByue+8PcvOL
ePH6GB64rlnRJYmlilM4n5/fzuf6phs2bDin80wm/d+2w7r8Gdj4NSRthY3fQld7ja
6olEaBXrx4fUuen72Ftxbu4vJLgrkk1MfoskREplbR9gt1zZ+T4dcXwDsMHl8Prl5G
V1SK1Wrlvulr+X1XCq0a+DL74R64Ol/wQtoillJLVMv2C1JDdRkO9algOxH+/MDo
asowmUy8ObAt/p4ubDuayeTf9hhdkoil1DAKN3WNs9uphf1WvgdZjrcyclivO6/d
3AaAKUv3seHwcYMrEhGRmkThpi5qeTNEdlaCXPjtVaOrKdf1betzU7sGFFmsjPh
+EyfytSmrilicG4WbushkgmtKFvb7ChK3GlvPGYy7sTVhvu4cSM3hjV92GF2OiljU
EAo3dVXDrrYRHKwOubAfgJ+nC2/f1haAL1Yd4o89KQZXJCliNYHCTV3WeyyY
XWDfb7B3sdHVlOuy6GAGd2sEwDM/bCYjt8DgikRExNEp3NRlAU2g679s9399
CSyO2dcy+roYmgR5kZh5krHzHHMKTUREHlfCTV132dPg7g/J2239Nw7lw9WJi
YNiMZtgzsajzN+cYHRJliLiwBRu6jrPALjiOdv931+DvGxj6zmD9g3r8ciVtr3LXpizh
eTMkwZXJClijkrhRqDzA1CvMWQnwZ/vG13NGT12VTStw31Jzy3guZ
mbqWOLa4ulyDlSuBFwdoU+r9jur3wfMo8aW88ZuDqbeXdQO1ydzfy+K4Vv18
QZXZKliDgghRuxibkRlrtC4Qn47TWjqzmjS0J9eLZvcwBenb+dQ2k5BlckliKORuF
GbE5f2G/j15C4xdh6KnBfj8Z0bRxAbn4RT3+/iSKLpqdEROQUhRs5JblztLoFsM
LPz0JhntEVlctsNvHObbF4uznz96HjfLJ8v9EliYilA1G4kdJ6jwVndzj8J3x7B+TnGl
1RuSlDPBnTvyUA7y7axY6ETlMrEhERR6Fwl6XVi4l7vwMXT9vKxV8NhJOOGRxu
6xhBn5ahFBRZeWrGRvlKHXMRQhERqV4KN1JW0yvh3jng5mcbwfnvjZB7zOiqyj
CZTEy4pQ2BXq7sTMziP4v2GF2Silg4AlclN1OmTCEqKgp3d3e6du3KmjVrznjurF

mz6NSpE/7+/nh5edGuXTu+/PLLaqy2jmjYFYbMA89AOLoBPu8HWYlGV1VGkL
cbr9/SBoCPl+9j7UHHC2EiIlK9DA83M2bMYMSlEYwdO5b169cTGxtL3759SU5
OLvf8gIAAXnjhBVatWsXmzZsZNmwYw4YNY+HChdVceR3QoB0M/Rm8wyBlB
3x+HaQfNrqqMvq2CuPWjhFYrfD095vIzis0uiQRETGQyWrwMq9du3alc+fOTJ4
8GQCLxUJkZCSPPfYYo0aNOqf36NChA9dffz3jx48/67mZmZn4+fmRkZGBr6/v
RdVeZxw7YJuaSj8MvhEweC4ENTO6qllyTxZw3aQ/iE8/wZ1dGjKheDRHRERqh/
P5+W3oyE1+fj7r1q2jd+/e9mNms5nevXuzatWqs77earWyZMkSdu3axeWXX17u
OXl5eWRmZpa6yXkKaAzDFkBgNGQesY3gJG0zuqpSfN1deOe2WAC+XXOY33
eWP/lnliK1n6HhJjU1laKilkJDQ0sDw0NJTHxzP0dGRkZeHt74+rqyvXXX88HH3x
Anz59yj13woQJ+Pn52W+RkZGV+h3qDL9wGPYLhLaBnGRbD86RdUZXVUq3
poHc37MxAM/O3MyxnHyDKxlRESMY3nNzlXx8fNi4cSNr167ltddeY8SlESxdurTc
c0ePHk1GRob9Fhen/YgumHcwDP0JljrDyXTbVNXBFUZXVcozfZvTLMSblKw8X
pyzRZtriojUQYaGm6CgIJycnEhKSip1PCkpibCwsDO+zmw206xZM9q1a8fTTz/N
rbfeyoQJE8o9183NDV9f31l3uQge9WyXiTe+HPKzbevg7FlsdFV27i5O/GdQO5z
NJn7eksi8TY65CailiFQdQ8ONq6srHTt2ZMmSJfZjFouFJUuW0K1bt3N+H4vFQl
6eY24VUCu5ecNdP0B0Xyg8aVvJePtco6uyaxPhx+NXRwPw/Kwt/L5L/TcilnWJ
4dNSl0aMYNq0aXzxxRfs2LGDhx56iJycHlYNGwbA4MGDGT16tP38CRMmsGj
RIvbv38+OHTuYOHEiX375Jffcc49ffcc49RX6FucnGH27+CVgPAUgA/DlWN3xpdld3D
vZrSo1kgOflF3D99LZ+uOKApKhGROsLZ6AJuv/12UlJSGDNmDlmJibRr144FCx
bYm4wPHz6M2Xwqg+Xk5PDwww9z5MgRPDw8aNGiBV999dXXX

dm7OTrx1a1ue79cCkwm+WX2YIZ+tIT1XjcYiIjWJwo1Ur6BmcN8vUK8xpB+yjeCk
7DK6KjuTycSDlzfl/wZ3wsvViT/3pXHzlJXsTXaMRmgRETk7hRupfv4N4b4FEBw
DWQm2gJOwyeiqSrk6JpSZD3cnop4HB9NyGfDhSpar0VhEpEZQuBFj+ITBsJ+h
QXvlTYPp/SFujdFVldIizJc5j/SgU6N6ZJ0sZNj0tXzx50E1GoulODiFGzGOZ4DtKq
qG3SAvA/57M+xfanRVpQR5u/H18K7c2jGCIouVsfO28alajUVEHJrCjRjL3de2Dk
7Tq6AgB74eBLt+MbqqUtycnXj71raMvs7WaPy1Go1FRByawo0Yz9UL7vwOWtw
ARXm2rRq2/Gh0VaWYTCb+dUVTpt1butF4X4oajUVEHl3CjTgGZze47QtoeztYC
m2bba7/r9FVldG7pa3RONzf1mh885SV/LFHjcYilo5E4UYch5Mz3DwVOt0HWG
HeY/DXR0ZXVUaLMF/mPnqq0Xjo52v576qDRpclliLFFG7EsZjNcP270P0x2+MF
o2DZ2+BgVyiVNBoP7GBrNB4zdxsvztmiRmMREQegcCOOx2SCPuOh1/O2x7+/
CovHOlzAcXN24p3bTjUaf/XXYYZ+rkZjERGjKdylYzKZoNdz0Pd12+OV78H8p8H
iWCMjJY3GnxQ3Gq/cm8aAD/9Uo7GlilEUbsSxdXsE+r8HmODvT2Huw1BUaHR
VZfRpGcqPD9kajQ+k5jBAjcYiloZRuBHH13EoDPw/MDnBpm/hx6FQmGd0VWX
E1Lc1GndsVl9MNRqLiBhG4UZqhja3wu1fgZMr7PgJvrsL8nONrqqMlG83vhnelVs
6hNsbjV/SisYiltVK4UZqjhb94K7vwcUT9i6Gr2+Fk5lGV1WGm7MTE2+LZVRxo/
GXfx1i6OdryMgtMLo0EZE6QeFGapamV8K9s8HNFw6thP/eBLnHjK6qDJPJxL+L
G4097Y3GK9mvRmMRkSqncCM1T8NLYchP4BEAR9fD9OshK8noqsrVp2UoM
4sbjfen5nDzlJWs2JNqdFkilrWawo3UTA3awbBfwDsMkrfD59dBepzRVZUrpr4v
cx451Wg85PM1fKlGYxGRKqNwlzVXSAu4bwH4N4Rj+2wBJ22f0VWVK9jHja8f6
Mot7W2Nxi/N3caYuVspVKOxiEilU7iRmi2gMQxbAlHRkBEHn10LSduMrqpc7i5OT
BwUy7PXNsdkgv+uOsTQz9eq0VhEpJlp3EjN5xdum6lKbQM5ybYenPj1RldVLpP
JxMO9mjH1no54ujqxYm+qGo1FRCqZwo3UDt7BMPQnCO8EJ47DFzfCoT+Nru
qM+rYK48d/d6eBn7u90XjlXjUai4hUBoUbqT086sHgORB1GeRnwZe3wJ5FRld1
Ri0b+DLn0R60b+hP5slCBn+2hq/+OmR0WSliNZ7CjdQubj5w9w8Q3RcKT9gW
+pv9b8g8anRl5QrxcefB4ZcyoLjR+MU5WxmrRmMRkYuicCO1j4uHbauGDkNsjz
d9Cx90hKVvOuSWDe4uTrw7KJZn+jYH4ltVhxg2fS0ZJ9RoLCJylRRupHZydoU
b34cHfoPlrlCQC0tfh8mdYPP3YHGskRGTycQjV9oajT1cnPhjj63R+EBqjtGliYjUO
Ao3UrtFdl
T7FsKtn4FfQ8iMh1nD4dM+ELfW6OrKuLZ1GD8+1M3WaJxiazT+U43GliLnReFG
aj+TCVoPhEfXwFUvgas3xP8Nn/aGH+93uJWNWzXwY86jPWgX6U/GiQlGf7aG
/646iMViNbo0EZEawWS1WuvUv5iZmZn4+fmRkZGBr6+v0eWlEbIS4bfxsOFrw
ArO7tD9MejxJLh5G12d3cmClkbN3MycjbZm6Jj6vjzT9xKubB6CyWQyuDoRkep
1Pj+/FW6k7krYBAueh0MrbI+9w+DqMRB7J5gdY1DTarXy6YoDvLd4D1I5hQB0al

SPZ/o2p2uTQlOrExGpPgo3FVC4kVKsVtj5P/j1RTh+0Hasfiz0nQBRPQwt7XTHc/
KZumwf0/88SF6hrRn6ikuCeaZvc1qH+xlcnYhl1VO4qYDCjZSrMA9WfwzL34a8T
NuxmBuhzzjb/lUOlinzJO8v2cOMtXEUFvfg9GsTxog+zWkW4jhTaililU3hpgIKN1Kh
7BTbJePrpoPVAk6ucOlDcNllcHecvy+H0nKYtHgPczbGY7WC2QQDO0TwRO9o
lup5Gl2eiEilU7ipgMKNnJOkbbDwBdj/u+2xVzBc9SK0vxfMTsbWdpqdiZlM/HU3i
7YnAeDqZOaurg155MpmBPu4GVydiEjlUbipgMKNnDOrFfb8Cgufh7S9tmOhraH
va9Ckl6Gl/dOGw8d5e+Eu/tyXBoCHixP39Yziwcub4ufhYnB1lilXT+GmAgo3ct6K
CmDt/8HSN+Bkuu1Y835wzasQ2NTQ0v5p5d5U3lq4i01x6QD4ujvz715NGdo9C
k9XZ2OLExG5CAo3FVC4kQuWewyWvQlrpoG1CMwu0OVBuOlZ247kDsJqtfLr9
iQm/rqL3UnZAAT7uPHYVc24o3NDXJ0d4zJ3EZHzoXBTAYUbuWgpu2yXju/51f
bYIwCufB46DgMnxxkdKbJYmbsxnv8s3k3csRMARNTz4MnelzCgfThOZi0EKCl1
h8JNBRRupNLsXWxrOk7ZaXsc1Bz6vg7RvY2t6x/yCy3M+DuOD5bslTkrD4Bml
d6MvOYS+rYK02rHllljKNxUQOFGKlVRlayfDr+9BieO2Y416w3XvAYhLQwt7Z9O5
BfxxaqDfLR0HxknCgBoG+HHM32b07NZkEKOiDg0hZsKKNxllTiRblsAcPXHYCk
AkxN0vh96jQbPAKOrKyXjRAH/98d+Pl1xgNz8lgAubRLAM31b0LGR4/QOiYicTu
GmAgo3UqXS9sGiMbYtHQDc/eCK56DzcHB2Nba2f0jNzuPD3/fx1V+HyC+ybe
nQOyaEp69pTkx9/bchlo5F4aYCCjdSLQ4st23KmbTF9jigqW19nEuuBQeb/olPP8
H7i/fww7o4LFZbeTfGNuCp3pcQFeRldHkiloDCTYUUbqTaWlpg49ewZDzkJNu
ONb7C1nQc1trY2sqxLyWbdxftZv7mBACczCYGdYrkiaujCfNzN7g6EanrFG4qoH
Aj1S4vC/54F1ZNgal8MJmhw2C48kXwDja6ujK2xmfwzq+7WLorBQA3ZzODuzX
ioV7NCPByrKk1Eak7FG4qoHAjhjl+CBaPhW2zbY9dfeDykbaNOZ0dbx+oNQeO8
fbCnaw9eBwAbzdnHrisMff3blyPu7Z0EJHqpXBTAYUbMdyhVbBwNBzdYHtcLw
r6jlOYGx2uH8dqtbJ0dwpvL9jF9oRMAOp5uvDllc2459JGuLs4ziaillK7KdxUQOF
GHlLFAptnwJJXlMvW40KjHrZ+nAbtDC2tPBaLlZ+3JvDur7vZn5oDQJivO49fHc1
tnSJwcdKWDiJStRRuKqBwlw4lPwdWvgcr34fCE4AJ2t0FV70EvvWNrq6MwilL
s9bHM2nxbo5mnAQgKtCTp/pcQv+2DTBrSwcRqSlKNxVQuBGHlHEEFr8CW76
3PXbxgp5PQfdHwcXD2NrKcbKgiG9WH2bK73tJy8kHoEWYD8/0bc5VLUK02r
GlVDqFmwoo3lhDO/l3LBgNR9bYHvtFQu+XofVAh+vHAcjJK+SzFQf4ZPl+sv
lKAejQ0J9n+ragW9NAg6sTkdrkfH5+O8RE+ZQpU4iKisLd3Z2uXbuyZs2aM547b
do0LrvsMurVq0e9evXo3bt3heeL1CgRneD+X2Hgp7ZgkxEHM++HaVfCuum2bR
4ciJebM49dHc0fz13Jv65ogruLmfWH07lz2l/c++lqNh9JN7pEEamDDB+5mTFjB
oMHD2bq1Kl07dqVSZMm8cMPP7Br1y5CQkLKnH/33XfTo0cPunfvjru7O2+++Sa
zZ89m27ZthleHn/XzNHljNUbBCVg1Gf74DxTYmnhxcoMW19v6cppcCU7Oxtb4

D0mZJ5n8216+XXOYQovtn5brWofx9DWX0CzEx+DqRKQmq1HTUl27dqVz585
MnjwZAlvFQmRkJl899hijRo066+uLioqoV68ekydPZvDgwWc9X+FGapzsZNj0L
Wz8FlJ2nDruHQptbrMFndBWxtVXjsNpuUxavJvZG+OxWsFsggHtl3iydzSRAZ5
GlyciNVCNCTf5+fl4enry448/cvPNN9uPDxkyhPT0dObOnXvW98jKyilkJlQffviBG
264oczzeXl55OXl2R9nZmYSGRmpcCM1j9UKCZtsQWfLD5Cbduq5sDYQe5ct7
DjQqse7k7J4Z+Euft2eBNi2dLgsOogB7cO5pmUYHq5aJ0dEzk2NCTdHjx4lPDyc
P//8k27dutmPP/vssyxbtozVq1ef9T0efvhhFi5cyLZt23B3L7v/zcsvv8wrr7xS5rjCjd
RohfmwdzFs+gZ2LQBLge242Rma9YHYO6D5dQ6z8vHGuHTeWbiLFXtT7ce83
Zy5tnUYt3Ql59LGgbqMXEQqVGfCzRtvvMFbb73F0qVLadu2bbnnaORGar3cY7
B1pm1EJ37dqePu/rarrGLvtDUqO8DVVvtTspmzlZ7ZG+OJO3bCfryBnzs3tQ/nlvb
hRleqN0dEyqox4eZipqXeeecdXn31VRYvXkynTp3O+TPVcyO1WsouW8jZNAOyj
p46HhhtG82JvQP8loyrr5jVauXvQ8eZtf4l/9ucQNbJQvtzrcN9GdA+ghtjGxDs4x
gjTyJivBoTbsDWUNylSxc++OADwNZQ3LBhQx599NEzNhS/9dZbvPbaayxcuJB
LL730vD5P4UbqBEsRHFhuCzo7foKC3OlnTND4Mlt/Tkx/cPM2tEywLQj4285kZ
q2PZ+muZPtVVk5mE5dHBzGgQwTXtAzVPlYidVyNCjczZsxgyJAhfPzxx3Tp0oV
Jkybx/fffs3PnTkJDQxk8eDDh4eFMmDABgDfffJMxY8bwzTff0KNHD/v7eHt74+1
99n+oFW6kzsnLgu3zbEHn4B+njrt4QcsbbdNWUZeB2fhlr9Ky8/jf5gRmbYhnU1
y6/bi3mzP92oQxoH0EXRsHqD9HpA6qUeEGYPLkybz99tskJibSrl073n//fbp27
QpAr169ilqKYvr06QBERUVx6NChMu8xduxYXn755bN+lsKN1Gnph21TVpu+gW
P7Tx33jYDY221BJyjauPpOs6+4P2fW+nji00/154T7e3BTuwbc0iFca+el1CE1LtxU
J4UbEWyXlcetsY3mbJ0FeRmnngvvBO3uhFa3gGeAcTUWs1isrD14jNkb4pm/p
XR/TtslPwa0D6d/bAOCvNWfl1KbKdxUQOFG5B8KTsKun2HTd7bLy61FtuNOrrb
LyWPvhGa9wcnF2Dqx9ecs2ZHMrPVHWLY7pVR/zhWXBHNLh3B6x6g/R6Q2U
ripgMKNSAWykmwLBG76FpK2njruFWxblDD2Dghr6xCXladl5/HTpqPM3hDPpi
OnRp583Jzp16Y+AzqE0yVK/TkitYXCTQUUbkTOUelW25YPW76HnJRTx0Na2a
at2gwCn1Dj6jvN3uRsZm84wpwNR8v059zcvgED2kfQLMT4K8NE5Mlp3FRA4
UbkPBUVwL7fYOM3tumronzbcZMZml5tCzrN+4GLh7F1YuvPWXPwGLPXx/Pzl
gSy8k7158Se1p8TqP4ckRpH4aYCCjciF+HEcdg22zaic2TNqeNuftDqZtsmnp
FdHWLa6mRBEYu2JzF7QzzLdqdQVNyf42zvz4ng6pgQ9eel1BAKNxVQuBGpJKl
7bb05m2dARtyp4/Ua25qQY++Aeo2Mq+80qdl5zNto68/ZEn9af467M9e3qc+
A9uF0Vn+OiENTuKmAwo1IJbNY4NAK22jO9rlQkHPquUY9bSGn5U3g7hj/ve1Nz
mLW+njmbljnaMZJ+/Gleh4MaB/OgPbhNAlWf46lo1G4qYDCjUgVys+xbfew8Rv
b9g8U//Pi7GHb7qHtlljq6TD9OX8dSGP2+nh+2Zpl9un9OZH+3FLcnxPg5WpglS

JSQuGmAgo3ItUk44htymrjt5C259RxJzdo2BWa9LLd6rcDs7F9Lyfyi1i0l4nZ64+
wfE9qqf6cXs1DuKVDOFe1UH+OiJEUbiqgcCNSzaxWiF9v2/Jh53zlSij9vLsfNL68
OOxcCQFNDG1ITsnKY96mo8zecISt8Zn2477uzlzf1rbtQ6dG9TA5QNO0SF2icFM
BhRsRA1mtkLoH9i+13Q7+AXmZpc/xi4QmV9iCTuPLwTvEiEoB2JOUxawNtv6ch
NP6c8L9PbiyRTA9mwXTrWkgfh7Gr94sUtsp3FRA4UbEgRQVwtENp8JO3GqW
FJQ+J6TVqSmsRt3BrfqbfS0WK3/tT2PWhnh+2ZJATn6R/Tkns4nYCD96Rgdze
XQQsZH+uDgZv8O6SG2jcFMBhRsRB5afA4dXnQo7iVtKP292hogup8JOeldq3/
PqRH4RK/amsmJPCn/sSWV/ak6p573dnOnWNJDLooO4LDqYqEBPTWGJVAK
Fmwoo3IjUIDmpcGCZLejsWwoZh0s/7+pju/qqJOwEN6/2fp0jx3NZsSeVP/amsn
JvKum5pUeewv09uPySIHo2C6ZHs0D8PXX1lciFULipgMKNSA1ltcLxA6dGdQ4s
t62YfDrvsOJ+nV7Q+ArwC6/WEossVrYdzeCPPan8sSeFdYeOU1B06p9Ykwnah
vvRs3hUp0PDerg6awpL5Fwo3FRA4UaklrBYIHHzqbBzeBUUnix9TtAlp0Z1onrar
syqRrn5hazef8wedvYkZ5d63tPViUubBNKzWRCXXxJE02BvTWGJnlHCTQUUb
kRqqYKTtobkkrBzdAP2RQTBttFneMdTYSeiMzhX7waaiRkn+WNPCiuKp7BSs/N
LPR/m685I0UH0jA6iZ7MgbfApchqFmwoo3IjUESeOw4E/TvXspO0t/byzh+3qq
5KwE9oazNU3RWSxWNmRmGnr19mTypqDx8gvtJQ6p1UDX3pGB3F5dDAdG
9XTIoJSpyncVEDhRqSOSo87FXT2L4WclNLPewba+nRKwk41b/p5sqCItQdLprB
S2ZFQev0fdxczXRoHclkz28hOizAfTWFJnaJwUwGFGxHBaoXk7actJriy9IafYNv
dvCToNL4cPAOqtcSUrDxW7k1I+Z4UVuxJJTkrr9TzwT5u9GwWZJvGahZEiK97t
dYnUt0UbiqgcCMiZRTmQ/zfsL94ZOfIWrAWnXaCCeq3PRV2Gnar1s0/rVYru5O
y+aN4bZ3VB9I4WVB6CqtFmI8t7FwSTJeoADxcNYUltYvCTQUUbkTkrE5mwqE
/T43spOwo/bzZBUJb2RYRbNAeGnSA4Bbg5Fwt5eUVFrHu0HH+2JPKij2pbD2
awen/krs6mencuB49mwVzWXQQLev7YjZrCktqNoWbCijciMh5y0o8NaqzfylkH
S17jrOHbXSnJOw0aA+BzaqlSflYTj4r99ouN1+xJ5WjGaUviQ/wcqVH8RTWZdFB
1PervlEnkcqicFMBhRsRuShWKxw/aLvU3H7bCPlZZc9184X6sbagUzLK49+oSld
RtIqt7E/N4Y/dtkvOV+1LK7UXFkDTYC8ui7aN6rRvWI8AL62aLI5P4aYCCjciUuks
FtuI5kc3wNH1tI8TNkPhibLnegQUj+6cFnh8G1RZafmFFjbGpdv7dTYfScfyj3/1Gw
Z4EhvpT7tIf9pF+tGqgZ8uOxeHo3BTAYUbEakWRYWQsvNU2IlfD0nbyu56DrZtI
04POw
06gFdgIZSVkVvAn/tse2H9tS+tzMafYNvpvEWYjy3wRPgTG+lPsxBvnNS3IwZSu
KmAwo2IGKYwD5K2Foed4imtlB1gtZQ9168hhJ/Wv9OgXZVsH5GRW8Dm+HQ
2xaWzMS6DjXHppGbnlTnP09WJNuF+tIv0t4/y1Pdz11o7Um0UbiqgcCMiDiU/Bx
K32EZ2Sqa1/rmaconAZqfCTngHCGsDrI6VWo7VaiUh46Qt7BxJZ+PhdLbEZ5D7

j74dsK21Exthm8qKjfSnbbg/fp4ulVqPSAmFmwoo3liIwzuZYWtStjcsr4f0w2XPM
5khOKY47BT38YS2rvQ9s4osVvYmZ9sDz6a4dHYmZlH0z+YdoEmQF7GR/sRG
2AJPTH1f9e9lpVC4qYDCjYjUSDlppcPO0Q2QlVD2vGpag+dEfhHbEzLYGJfBprh
0Nh1J51BabpnzXJxMxNT3JTbi1HRWkyAvrbsj503hpgIKNyJSa2QmIA478evhxL
Gy51XTGjzHcvLZfCSdTXEZbIw7zqYjGRzLyS9zno+bM20j/UoFnlBtHyFnoXBTAY
UbEam1rFbb9NXpYSdhE+RIlj3X1ce2Bk9oSwhubhvdCW4BXkGVWI6VI8dPsDE
u3T66syU+o8zWEQBhvu7EFvfutIvwp02EHz7u6t+RUxRuKqBwlyJ1isUCx/adCjt
HN9gCT3lr8IBtd/TgmNMCT/Gv3iGVsvhgYZGF3UnZbDpScoVWOruTssqsvWM
yQbNgb1v/TnHgaR7mg6tz1a/4LI5J4aYCCjciUucVFULqLlvlSdkJKbtsvx4/BJzhR
4JHvdJhJ7i5LQT5hF106MnNL2RrfKY97GyMSyc+vWz4cnU206qBb/EVWrbQE
xXoqcvR6wiFmwoo3liInEF+LqTuPhV27KHnQPlr8QC4+RUHndOmtkJagG/4RY
WelKy84v6ddDYesTUtZ5wouwCin4cLbSP8iKnvS3SIN83DfGgW4o2na/VsYirV
R+GmAgo3liLnqeCEbe0de+jZCck74dh+sJZd/wYAV++yozzBzcEv8oIama1WK4
fSctI0JN3ew7P1aCb5hWVDI8kEEfU8aB7qQ3SoD5eEenNJqA9Ng711WXoNpn
BTAYUbEZFKUpgHaftKj/Kk7LQFlUth+a9x8YSgS06FnpDi0OPfCMznFzzyCy3sTs
pi05F09iRIsysxiz3JWaRml71CC8BsgkaBXvawEx3qQ/NQHxoHeamXpwZQuKm
Awo2lSBUrKrCN6pSEnuQdtl/T9kBR+cEDZ3clij41tVVyqxd13mv0pGXnsTspmz3
JWexOymJ3Yja7k7NIzy1nXy/A2WwiKsireKTHFnwuCfUhKtATZyeFHkehcFMBhR
sREYMUFcLxg6dGeEpuqXug8GT5r3FyhcDo0qM8wS0goAk4nful4larlZTsPFvQ
SbKN8OxKzGJPUjZZeeWPMrk6mWkS7FUcdrztIz2RAZ7aRNQACjcVULgREXE
wliJlP1R6lCdIp625uaDsqscAmJ1tCxGWTG8FNIF6jW0jPedx2brVaiUx86Q96Ox
OyioOP9nl7qcF4OZsplmIt72np3mYN9EhPoT7e2jl5SqkcFMBhRsRkRrCYoGMuO
Kws6P0VVz52Wd+nYunLeSUuhUHH/+G4HL21ZAtFivx6SeKw86p0LM3OZu8cp
qYwbZzenSoD5eEFE9thdlGfMJ8tXt6ZVC4qYDCjYhlDWe1Qma87YqtlJ22NXu
OH4RjByHzyJkvWwfABL4Nyg8+9aJsKzRXEESKLFYOH8u1je4kZbErKZs9SVnsS
8mmoKj8H6c+7s6nprZCfGgeZuvtCfZ2U+g5Dwo3FVC4ERGpxQrzbaM9xw/YA
k/J7dhB27GKRnzAdgl7RaM+zq7lvqygyMKhtJxSozy7k7I5kJpT7u7pAP6eLvbQc
0moD9EhPjQN9iLYR6GnPAo3FVC4ERGpo6xWyD12WvA5UBx6im+Z8ZxxhWY
ATOAXUX74CWhsW8X5H6Ekr7CIA6m20LMnqbiJOTmbQ2k5ZbacKOHh4kSjQE
8aBngSFe
RFo0BPGgXYfm3g71Fnm5kVbiqgcCMiluUqOFk86nMQjv1j5Of4gTM3N5dw84
V6jcpOdQU0ti1eeNrVXScLitibXHK5eja7i0PPkeO5Zww9AC5OJiLredIw0JOowO
LgE+hJo0AvIup54OZcexcpVLipgMKNilicN6sVclLKCT7F97MSKn69yVw86tO49

KhPQPFjj3qAbWHC+PQTHEzL4XBabqlf446dIL/ozP1EJhM08PMgKsiThgFeRJ0
WfBoFetb4LSkUbiqgcCMilpWu4lRt49F/hp6S25nW8Snh7mcLOX6Rtn25/MKLf
42w/epTnyKTE4mZJzmUmsOhY6cHn1wOp+WQc4ZL10sE+7gRFXgq+JSM/kQ
FeuHnee5rBhlF4aYCCjcillKtLBbITio/+Bw7ADnJZ38Pkxm8w2xXevmFg29EqQBk
9W1AqtWfQ8dPcigtl0NpJQHldv9MqzOX8PNwKQ48JSM+p6a8HOWqLoWbCij
ciliIQ8nPOTXqk3HEdjl75lHliC++nwCWisMJYFvY0Kd+uSM/WW6hHC6qx/4cD3
voKZnuSs7Kq/BtPV2daBhgCzq2Pp9Twae+X/U1OCvcVEDhRkREahSLxTa6Yw8
7R4tDUHzxsXhbz0+F6/sUc3K1jf6cNvKT71WfJHMQcYX12HPSl10ZLhw+Zuv7O
Zp+osIGZ1cnMxEBHkQFetmu7ioe9Wkc5EVUkFfl/R6gcFMhhRsREal1igohO7Gc
4HPkVADKTqbiS92LuXgWB6BwinzCyXQNIckURFyhP3tO+rEtx4cdx00cqaDBuX
moDwufurxSv+L5/Pyu2a3TliliYts53S/CdovsUv45hfm2EZ7ygk/JsdxU2yXvaXsh
bS9OQL3iWwugT8l7ufpgbRBOnmcY6S4hJBNEXFE99p70Y0u2N/VC61XHtz4jh
RsREZG6wNm1eB2eRmc+p+Bk6bBTqv8n3jYqdDld8rMwpezEnZ2EAWFA29Pfx
z0G+Ksqv02FFG5ERETExsUdApvabmeSn1NO8DltFCgj3tbPYyCFGxERETl3rl4
QfIntdiaFFV+BVdXMhn66ilil1D7OboZ+vMKNilil1CoKNylillKrKNylilhlraJwlylilrWK4
eFmypQpREVF4e7uTteuXVmzZs0Zz922bRsDBw4kKioKk8nEpEmTqq9QERERq
REMDTczZsxgxlgRjB07lvXr1xMbG0vfvn1JTi5/h9Tc3FyaNGnCG2+8QVhYWDV
XKylilJWBoeHm3XffZfjw4QwbNoyWLVsydepUPD09+eyzz8o9v3Pnzrz99tvccc
cduLkZe5mZililOCbDwk1+fj7r1q2jd+/ep4oxm+nduzerVq0yqiwRERGp4QxboTg1
NZWioiJCQ0NLHQ8NDWXnzp2V9jl5eXnk5Z1aKTEzM7PS3ltEREQcj+ENxVVtw
oQJ+Pn52W+RkZFGlyQiliJVyLBwExQUhJOTE0lJSaWOJyUlVWqz8OjRo8nlyLD
f4uLiKu29RURExPEYFm5cXV3p2LEjS5YssR+zWCwsWbKEbt26VdrnuLm54evr
W+omlilitZehu4KPGDGCIUOG0KlTJ7p06cKkSZPlyclh2LBhAAwePJjw8HAmTJg
A2JqQt2/fbr8fHx/Pxo0b8fb2plmzZoZ9DxEREXEchoab22+/nZSUFMaMGUNiYi
Lt2rVjwYlF9ibjw4cPYzafGlw6evQo7du3tz9+5513eOedd7jiiitYunTpOX2m1WoF1
FgsliJSk5T83C75OV4Rk/VczqpFjhw5oqZiERGRGiouLo6lilgKz6lz4cZisXD06FF
8fHwwmUyV+t6ZmZlERkYSFxen3h4HoD8Px6l/D8eiPw/Hoz+TilmtVrKysmjQoE
GpWZ3yGDotZQSz2XzWxHex1LjsWPTn4Vj05+FY9OfhePRncmZ+fn7ndF6tX+d
GRERE6haFGxEREalVFG4qkZubG2PHjtWmng5Cfx6ORX8ejkV/Ho5HfyaVp841
FluliEjtppEbERERqVUUbkRERKRWUbgRERGRWkXhRk
RERGoVhZtKMmXKFKKionB3d6dr166sWbPG6JLqrAkTJtC5c2d8fHwlCQnh5pt
vZteuXUaXJcXeeOMNTCYTTz75pNGl1Fnx8fHcc889BAYG4uHhQZs2bfj777+NL
qtOKioq4qWXXqJx48Z4eHjQtGlTxo8ff077J8mZKdxUghkzZjBixAjGjh3L+vXriY2

NpW/fviQnJxtdWp20bNkyHnnkEf766y8WLVpEQUEB11xzDTk5OUaXVuetXbu
Wjz/+mLZt2xpdSp11/PhxevTogYuLC7/88gvbt29n4sSJ1KtXz+jS6qQ333yTjz76i
MmTJ7Njxw7efPNN3nrrLT744AOjS6vRdCl4JejatSudO3dm8uTJgG3/qsjlSB577
DFGjRplcHWSkpJCSEgly5Yt4/LLLze6nDorOzubDh068OGHH/Lqq6/Srl07Jk2a
ZHRZdc6oUaNYuXllf/zxh9GlCHDDDTcQGhrKp59+aj82cOBAPDw8+OqrrwysrG
bTyM1Fys/PZ926dfTu3dt+zGw207t3b1atWmVgZVlilyMDgICAAlMrqdseeeQRrr/
++lL/rUj1mzdvHp06deK2224jJCSE9u3bM23aNKPLqrO6d+/OkiVL2L17NwCbN
m1ixYoVXHfddQZXVrPVuY0zK1tqaipFRUWEhoaWOh4aGsrOnTsNqkpKWCwW
nnzySXr06EHr1q2NLqfO+u6771i/fj1r1641upQ6b//+/Xz00/nrVr1/
L444/j6urKkCFDjC6vzhk1ahSZmZm0aNECJycnioqKeO2117j77ruNLq1GU7iRWu
2RRx5h69atrFixwuhS6qy4uDieeOlJFi1ahLu7u9Hl1HkWi4VOnTrx+uuvA9C+fXu2
bt3K1KlTFW4M8P333/P111/zzTff/PPPP

p08fvL297bf//ve/7Nu3z37e6cEnlCCA5s2bs2PHDgB27NhBjx49Sr1vjx492LNnD
0VFRWzcuBEnJyeuuOKKCmtp27at/X79+vUBSE5OvujvKCLVy9noAkSkbsvOzg
Zg/vz5hleHl3rOzc2tVMC5UB4eHud0nouLi/2+yWQCbP1AllKzaORGRAzVsmVL3
NzcOHz4MM2aNSt1i4yMtJ/3119/2e8fP36c3bt3ExMTA0BMTAwrV64s9b4rV67
kkksuwcnJiTZt2mCxWEr18lhl7aWRGxExll+PDyNHjuSpp57CYrHQs2dPMjlyWLly
Jb6+vjRq1AiAcePGERgYSGhoKC+88AJBQUHcfPPNAD z99NN07tyZ8ePHc/vtt
7Nq1SomT57Mhx9+CEBUVBRDhgzhvvvu3/333c2NpZDhw6RnJzMoEGDjPrqI
lJFFG5ExHDjx48nODiYCRMmsH//fvz9/enQoQPPP/+8ADz99NN07tyZ8ePHc/vtt
eOn376CVdXVwA6dOjA999/9/z/z5gxYxg/fjz169dfX4mlknnmDPnj20a9
eOn376CVdXVwA6dOjA999/z5gxYxg/fjz169dn3LhxDB061P4ZH330Ec8//zwPP
/wwaWlpNGzYkOeff96lrysiVUxXS4mlQyu5kun48eP4+/sbXY6l1ADquREREZFa
ReFGREREahVNS4mliEitopEbERERqVUUbkRERKRWUbgRERGRWkXhRkRERGRWFGxERUFZRuBEREZFaReFGREREahWFGxEREalV/h+w5iTs
MJwnEwAAAABJRU5ErkJggg==\n"
      },
      "metadata": {}
    }
  ]
},
{
  "cell_type": "markdown",
  "source": [
    "Accuracy of the model on test data"
  ],
  "metadata": {
    "id": "ycbLKSDl-SJX"
  }
},
{
  "cell_type": "code",
  "source": [
    "loss, accuracy = model.evaluate(X_test_std, Y_test)\n",
    "print(accuracy)"
  ],

"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "D3RgBzQF-B8W",
"outputId": "be7daea4-4ce1-40d9-a6b7-a1f765063642"
},
"execution_count": 27,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"\u001b[1m4/4\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 21ms/step - accuracy: 0.9735 - loss: 0.1128\n",
"0.9649122953414917\n"
]
}
]
},
{
"cell_type": "code",
"source": [
"print(X_test_std.shape)\n",
"print(X_test_std[0])"
],
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "OWyJPf9O-oJT",
"outputId": "252ed0a2-7bbd-4098-cff9-ec9faca91ec5"

},
"execution_count": 30,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"(114, 30)\n",
"[-0.04462793 -1.41612656 -0.05903514 -0.16234067 2.0202457 -0.1132
3672\n",
" 0.18500609 0.47102419 0.63336386 0.26335737 0.53209124 2.6276
3999\n",
" 0.62351167 0.11405261 1.01246781 0.41126289 0.63848593 2.8897181
5\n",
" -0.41675911 0.74270853 -0.32983699 -1.67435595 -0.36854552 -0.387
67294\n",
" 0.32655007 -0.74858917 -0.54689089 -0.18278004 -1.23064515 -0.62
68286]\n"
]
}
]
},
{
"cell_type": "code",
"source": [
"Y_pred = model.predict(X_test_std)"
],
"metadata": {
"id": "XWAZk1SK-3qf",
"colab": {
"base_uri": "https://localhost:8080/"
},
"outputId": "4eed0c9e-8443-48ce-cad3-4a1c665e99d7"

},
"execution_count": 31,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"\u001b[1m4/4\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 16ms/step\n"
]
}
]
},
{
"cell_type": "code",
"source": [
"print(Y_pred.shape)\n",
"print(Y_pred[0])"
],
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "umNl8qUX_GeO",
"outputId": "fb7afd88-c517-437a-dea7-1024af588f12"
},
"execution_count": 32,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"(114, 2)\n",

"[0.3911283 0.7049195]\n"
]
}
]
},
{
"cell_type": "code",
"source": [
"print(X_test_std)"
],
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "0lsaXHZr_mTT",
"outputId": "f02bd988-b222-435f-8524-c2e4cd5caade"
},
"execution_count": null,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"[[-0.04462793 -1.41612656 -0.05903514 ... -0.18278004 -1.23064515\n",

" -0.6268286]\n",
" [0.24583601 -0.06219797 0.21802678 ... 0.54129749 0.11047691\n",
" 0.0483572]\n",
" [-1.26115925 -0.29051645 -1.26499659 ... -1.35138617 0.269338\n",
" -0.28231213]\n",
" ...\n",
" [0.72709489 0.45836817 0.75277276 ... 1.46701686 1.19909344\n",
" 0.65319961]\n",

```
      " [ 0.25437907  1.33054477  0.15659489 ... -1.29043534 -2.22561725\n",
      "  -1.59557344]\n",
      " [ 0.84100232 -0.06676434  0.8929529  ...  2.15137705  0.35629355\n",
      "   0.37459546]]\n"
    ]
  }
 ]
},
{
 "cell_type": "code",
 "source": [
  "print(Y_pred)"
 ],
 "metadata": {
  "colab": {
   "base_uri": "https://localhost:8080/"
  },
  "id": "cmaeRBc7_M59",
  "outputId": "4cc8d8ed-9b8a-4a18-ed2a-337d1d894f3e"
 },
 "execution_count": 33,
 "outputs": [
  {
   "output_type": "stream",
   "name": "stdout",
   "text": [
    "[[0.3911283  0.7049195 ]\n",
    " [0.5164513  0.5410615 ]\n",
    " [0.04336363 0.8943092 ]\n",
    " [0.99922407 0.00427999]\n",
    " [0.3611054  0.5433798 ]\n",
    " [0.9550043  0.06278113]\n",
    " [0.18345568 0.6151417 ]\n",
```

" [0.08052514 0.90846545]\n",
" [0.12616669 0.8190895]\n",
" [0.15915196 0.8931857]\n",
" [0.4868551 0.4661419]\n",
" [0.15479487 0.6536912]\n",
" [0.20664826 0.79865897]\n",
" [0.32411543 0.62896466]\n",
" [0.06087842 0.8601217]\n",
" [0.9652394 0.24043068]\n",
" [0.10668699 0.8861862]\n",
" [0.08097368 0.8705285]\n",
" [0.13319834 0.9222683]\n",
" [0.9252814 0.09910237]\n",
" [0.01349126 0.08834455]\n",
" [0.09537797 0.93914974]\n",
" [0.15204243 0.851396]\n",
" [0.06239184 0.96219647]\n",
" [0.21940611 0.760679]\n",
" [0.91869956 0.14567868]\n",
" [0.15302996 0.8425631]\n",
" [0.22005162 0.8294395]\n",
" [0.9130257 0.2760873]\n",
" [0.9346131 0.16852579]\n",
" [0.18752375 0.8461464]\n",
" [0.16387205 0.6634572]\n",
" [0.13986309 0.8925306]\n",
" [0.9845656 0.08467065]\n",
" [0.9234556 0.03942275]\n",
" [0.26875693 0.7937962]\n",
" [0.06765838 0.94004077]\n",
" [0.27962473 0.68683314]\n",
" [0.03580508 0.85851765]\n",
" [0.12074795 0.73851806]\n",

" [0.9918112 0.00544042]\n",
" [0.7836475 0.4228762]\n",
" [0.0340062 0.90062976]\n",
" [0.04896436 0.79250795]\n",
" [0.52313185 0.12821601]\n",
" [0.10785364 0.8457893]\n",
" [0.02761351 0.8303093]\n",
" [0.02000181 0.88016623]\n",
" [0.9876164 0.09092868]\n",
" [0.9037055 0.16100125]\n",
" [0.13733742 0.9098595]\n",
" [0.7044874 0.3308058]\n",
" [0.41120523 0.56744397]\n",
" [0.0953822 0.922956]\n",
" [0.02912947 0.87210387]\n",
" [0.44844854 0.46464044]\n",
" [0.2263031 0.7110021]\n",
" [0.03291021 0.8040876]\n",
" [0.39624515 0.0267334]\n",
" [0.12627457 0.7809259]\n",
" [0.31291652 0.6084102]\n",
" [0.54701006 0.08340784]\n",
" [0.09912547 0.92183304]\n",
" [0.9313995 0.05583052]\n",
" [0.8615376 0.21408364]\n",
" [0.27397817 0.23180664]\n",
" [0.97463083 0.10631539]\n",
" [0.93085295 0.16497684]\n",
" [0.26127833 0.7541054]\n",
" [0.24892722 0.1908311]\n",
" [0.7419908 0.13440447]\n",
" [0.9758   0.2590311]\n",
" [0.12390636 0.7783192]\n",

" [0.82916945 0.19656679]\n",
" [0.03731575 0.93089193]\n",
" [0.71693546 0.17407367]\n",
" [0.05627083 0.6256016]\n",
" [0.07991214 0.8449547]\n",
" [0.434865   0.7486748]\n",
" [0.5635281 0.15433535]\n",
" [0.9789559 0.06717476]\n",
" [0.76706076 0.29586586]\n",
" [0.8392746 0.05256142]\n",
" [0.21517153 0.6349469]\n",
" [0.10904227 0.85879076]\n",
" [0.42488673 0.48982525]\n",
" [0.04876341 0.42984095]\n",
" [0.08822696 0.7943996]\n",
" [0.20059806 0.48233542]\n",
" [0.9893586 0.18230177]\n",
" [0.09869318 0.71248657]\n",
" [0.23000379 0.8167128]\n",
" [0.06718588 0.83381754]\n",
" [0.96439433 0.20374772]\n",
" [0.6875367 0.35703582]\n",
" [0.07121131 0.8486421]\n",
" [0.9604786 0.06068479]\n",
" [0.8212932 0.07128309]\n",
" [0.24191433 0.75566053]\n",
" [0.05943422 0.9472607]\n",
" [0.06641878 0.9503383]\n",
" [0.66023666 0.42217234]\n",
" [0.9937631 0.08209572]\n",
" [0.9880941 0.01360018]\n",
" [0.19373627 0.7758272]\n",
" [0.04747728 0.82231355]\n",

" [0.03998263 0.97610146]\n",
" [0.04456213 0.6420242]\n",
" [0.0023429 0.8494351]\n",
" [0.20866181 0.80812997]\n",
" [0.97156584 0.08482962]\n",
" [0.97291386 0.12018061]\n",
" [0.38877603 0.62668073]\n",
" [0.90034616 0.07521999]]\n"
]
}
]
},
{
"cell_type": "markdown",
"source": [
"model.predict() gives the prediction probability of each class for that data point"
],
"metadata": {
"id": "dx5ZqN_W_6U-"
}
},
{
"cell_type": "code",
"source": [
"# argmax function\n",
"\n",
"my_list = [0.25, 0.56]\n",
"\n",
"index_of_max_value = np.argmax(my_list)\n",
"print(my_list)\n",
"print(index_of_max_value)"
],

"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "pPF0aJ0sAbML",
"outputId": "7af3b1a0-f452-4a1c-a5ef-a894fc9f0945"
},
"execution_count": 34,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"[0.25, 0.56]\n",
"1\n"
]
}
]
},
{
"cell_type": "code",
"source": [
"# converting the prediction probability to class labels\n",
"\n",
"Y_pred_labels = [np.argmax(i) for i in Y_pred]\n",
"print(Y_pred_labels)"
],
"metadata": {
"colab": {
"base_uri": "https://localhost:8080/"
},
"id": "67EUrfSf_rZi",
"outputId": "bb5f25f4-67f3-46bc-ddb1-b6c1a5aaba42"

},
"execution_count": 35,
"outputs": [
{
"output_type": "stream",
"name": "stdout",
"text": [
"[np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(0)]\n"
]
}
]
},
{
"cell_type": "markdown",
"source": [
"**Building the predictive system**"

```
    ],
    "metadata": {
      "id": "PBvyhDoEBVk8"
    }
  },
  {
    "cell_type": "code",
    "source": [
      "input_data = (11.76,21.6,74.72,427.9,0.08637,0.04966,0.01657,0.01115,0.149
5,0.05888,0.4062,1.21,2.635,28.47,0.005857,0.009758,0.01168,0.007445,0.02
406,0.001769,12.98,25.72,82.98,516.5,0.1085,0.08615,0.05523,0.03715,0.2433,
0.06563)\n",
      "\n",
      "# change the input_data to a numpy array\n",
      "input_data_as_numpy_array = np.asarray(input_data)\n",
      "\n",
      "# reshape the numpy array as we are predicting for one data point\n",
      "input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)\n",
      "\n",
      "# standardizing the input data\n",
      "input_data_std = scaler.transform(input_data_reshaped)\n",
      "\n",
      "prediction = model.predict(input_data_std)\n",
      "print(prediction)\n",
      "\n",
      "prediction_label = [np.argmax(prediction)]\n",
      "print(prediction_label)\n",
      "\n",
      "if(prediction_label[0] == 0):\n",
      "  print('The tumor is Malignant')\n",
      "\n",
      "else:\n",
      "  print('The tumor is Benign')"
```

```
    ],
    "metadata": {
      "colab": {
        "base_uri": "https://localhost:8080/"
      },
      "id": "rXQi6eStBJxb",
      "outputId": "a8bc4c3b-7118-4d20-8c3c-fb46171d99e1"
    },
    "execution_count": 36,
    "outputs": [
      {
        "output_type": "stream",
        "name": "stdout",
        "text": [
          "\u001b[1m1/1\u001b[0m \u001b[32m━━━━━━━━━━━━━━━━━━━━\u
001b[0m\u001b[37m\u001b[0m \u001b[1m0s\u001b[0m 35ms/step\n",
          "[[0.07750479 0.9005814 ]]\n",
          "[np.int64(1)]\n",
          "The tumor is Benign\n"
        ]
      },
      {
        "output_type": "stream",
        "name": "stderr",
        "text": [
          "/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: U
serWarning: X does not have valid feature names, but StandardScaler was fitted
with feature names\n",
          "  warnings.warn(\n"
        ]
      }
    ]
  },
```

```
  {
   "cell_type": "code",
   "source": [],
   "metadata": {
    "id": "W0jvu0aNDDP3"
   },
   "execution_count": null,
   "outputs": []
  }
 ]
}
```